

Designing a universal homomorphic instruction set and applications to transciphering with TFHE

Daphné Trama

▶ **To cite this version:**

Daphné Trama. Designing a universal homomorphic instruction set and applications to transciphering with TFHE. Cryptography and Security [cs.CR]. Université Paris-Saclay, 2025. English. ⟨NNT : 2025UPASG071⟩. ⟨tel-05409543⟩

HAL Id: tel-05409543

https://theses.hal.science/tel-05409543v1

Submitted on 10 Dec 2025

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

université PARIS-SACLAY

Designing a universal homomorphic instruction set and applications to transciphering with TFHE

Conception d'un jeu d'instructions homomorphe universel et applications au transchiffrement avec TFHE

Thèse de doctorat de l'université Paris-Saclay

École doctorale n°580 Sciences et Technologies de l'Information et de la Communication (STIC)
Spécialité de doctorat: Informatique mathématique
Graduate School : Informatique et sciences du numérique
Référent : Faculté des Sciences d'Orsay

Thèse préparée dans l'unité de recherche **Laboratoire d'Intégration de Systèmes et des Technologies du CEA**,
sous la direction de **Renaud Sirdey**, directeur de recherche au CEA-LIST
et le co-encadrement d'**Aymen BOUDGUIGA**, ingénieur chercheur au CEA-LIST

Thèse soutenue à Paris-Saclay, le 13 novembre 2025, par

Daphné TRAMA

Composition du jury

Membres du jury avec voix délibérative

Anne Canteaut Directrice de recherche, INRIA Paris	Présidente
Philippe Gaborit Professeur des universités, Université de Limoges	Rapporteur & Examinateur
Melek Önen Professeure, EURECOM	Rapportrice & Examinatrice
Sonia Belaïd Experte en cryptographie, CryptoExperts	Examinatrice
Mireille Fouquet Maîtresse de conférence, Université Paris Cité	Examinatrice
Louis Goubin Professeur des universités, Université de Versailles-St-Quentin	Examinateur

THESE DE DOCTORAT

NNT : 2025UPASG071

université PARIS-SACLAY

ÉCOLE DOCTORALE
Sciences et technologies de l'information et de la communication (STIC)

Titre: Conception d'un jeu d'instructions homomorphe universel et applications au transchiffrement avec TFHE

Mots clés: Chiffrement homomorphe, calculs sécurisés, cryptographie

Résumé: Avec l'émergence d'internet et le flux grossissants de communications que cela entraîne, conserver la confidentialité des données communiquées reste un enjeu majeur pour la vie privée des utilisateurs. Jusqu'à présent, la confidentialité est assurée lors du transfert, et parfois du stockage de nos données à des serveurs distants. En revanche, dès que nous souhaitons faire appel à un serveur distant pour appliquer des calculs à nos données (par exemple appliquer un filtre sur une photo), celui-ci doit les déchiffrer pour procéder aux manipulations souhaitées. Un attaquant qui aurait accès aux serveurs de tels fournisseurs de service aurait donc un accès aux données non protégées des utilisateurs.

Une solution pour répondre à cette faille de sécurité est d'utiliser le chiffrement homomorphe. Ce nouveau type de chiffrement permet en effet d'effectuer des calculs directement sur les données chiffrées: plus besoin de déchiffrer pour calculer, nos données sont donc protégées de bout en bout ! Cependant, aussi idyllique que paraisse cette solution, quelques freins subsistent à son déploiement. En effet, les chiffrés homomorphes sont très lourds (plusieurs milliers d'octets pour seulement quelques bits d'information) ce qui a des conséquences sur le temps d'exécution des calculs et la quantité de mémoire nécessaire à leur bon déroulement.

Dans cette thèse, nous étudions un procédé appelé "transchiffrement" qui permet de réduire la borne passante utilisée pour l'envoi de données homomorphes, ce qui a pour effet secondaire d'également réduire le délai d'attente lors de l'envoi de données. En particulier, nous nous concentrons sur les chiffrements par bloc tels que l'AES. Dans un deuxième temps, nous proposons un jeu d'instruction universel pour processeur homomorphe. C'est-à-dire, nous construisons un jeu d'instruction qui fonctionne sur données chiffrées. Notre processeur permet d'effectuer n'importe quel algorithme avec des latences compétitives avec l'état de l'art.

En résumé, nous proposons des améliorations du cryptosystème homomorphe TFHE en étudiant différentes méthodes de transchiffrement, différents chiffrements à bloc et en créant un jeu d'instructions universel compétitif avec l'état de l'art en terme de latence et qui permet l'évaluation de n'importe quel algorithme de façon facilité par rapport aux approches standards.

université PARIS-SACLAY
ÉCOLE DOCTORALE
Sciences et technologies de l'information et de la communication (STIC)

Title: Designing a universal homomorphi instruction set and applications to transciphering with TFHE

Keywords: Homomorphic encryption, secure computations, cryptography

Abstract: With the developpement of the internet and the increasing flow of communications it facilitates, maintaining the confidentiality of transmitted data remains a significant concern for user privacy. Until now, confidentiality has primarily been ensured during the transmission and, in some cases, the storage of our data on remote servers. However, when we request a distant server to perform calculations on our data (for example, applying a filter to a photo), the server must decrypt the data to perform the desired operations. This presents a vulnerability, as an attacker who gains access to the servers of such service providers could potentially access unprotected user data.

One solution to address this security flaw is homomorphic encryption. This innovative type of encryption allows for calculations to be conducted directly on encrypted data, meaning there is no need to decrypt it in order to perform calculations. As a result, our data remains protected throughout the entire process!

Despite its promising nature, the deployment of homomorphic encryption still faces several challenges. For instance, homomorphic ciphertexts are known to be very large (often several thousand bytes for just a few bits of information), which impacts both the execution time of calculations and the memory required for their proper functioning.

In this thesis, we explore a process called "transciphering", which reduces the bandwidth needed for transmitting homomorphic data, thereby also decreasing latency when sending this data. In our work, we especially focus on block ciphers such as AES.

Additionally, we propose a universal instruction set for a homomorphic processor working on 8-bit encrypted inputs. This means we developed an instruction set that directly operates over ciphertexts. Our processor can execute any algorithm with latencies that are competitive with the state of the art.

In summary, we propose improvements to the TFHE homomorphic cryptosystem by examining various transciphering methods and block ciphers. We also design a universal instruction set that outperforms other traditional approaches while facilitating the evaluation of any algorithm over encryptions of bytes.

Résumé Étendu

Avec l'émergence d'Internet et l'explosion du volume de communications numériques qu'il génère, la question de la confidentialité des données échangées et traitées constitue aujourd'hui un enjeu central pour la protection de la vie privée des utilisateurs. Chaque jour, d'innombrables informations personnelles transitent entre nos appareils et des serveurs distants : messages, photographies, documents administratifs, données médicales, historiques de navigation, etc. Jusqu'à présent, la confidentialité est assurée lors du transfert, et parfois du stockage de nos données à des serveurs distants. En revanche, dès que nous souhaitons faire appel à un serveur distant pour appliquer des calculs à nos données (par exemple appliquer un filtre sur une photo), celui-ci doit les déchiffrer pour procéder aux manipulations souhaitées. Un attaquant qui aurait accès aux serveurs de tels fournisseurs de service aurait donc un accès aux données non protégées des utilisateurs.

Une solution pour répondre à cette faille de sécurité est d'utiliser le chiffrement homomorphe. Ce type de chiffrement permet d'effectuer des opérations directement sur les données chiffrées, sans jamais avoir besoin de les déchiffrer durant les calculs intermédiaires. En d'autres termes, les données demeurent protégées de bout en bout, même lorsqu'elles sont manipulées par un serveur tiers. Le résultat produit, également chiffré, peut ensuite être transmis à l'utilisateur qui, lui seul, détient la clé de déchiffrement nécessaire pour obtenir la réponse en clair. Le chiffrement homomorphe promet ainsi un modèle de sécurité révolutionnaire dans lequel l'on peut déléguer des calculs potentiellement complexes sans jamais céder la confidentialité de ses données. Cependant, aussi idyllique que paraisse cette solution, quelques freins subsistent à son déploiement. En effet, les chiffrés homomorphes sont très lourds (plusieurs milliers d'octets pour seulement quelques bits d'information) ce qui a des conséquences sur le temps d'exécution des calculs et la quantité de mémoire nécessaire à leur bon déroulement.
Ce facteur d'expansion est dû au formatage des chiffrés homomorphes : en effet, pour assurer la sécurité des chiffrés, un bruit, aussi appelé « erreur » est ajouté aux messages pendant le processus de chiffrement.

Dans cette thèse, nous étudions principalement un cryptosytème homomorphe appelé TFHE, connu pour l'efficacité de son opération de réduction de bruit des chiffrés, appelée bootstrapping. En plus d'être efficace, le bootstrapping de TFHE est fonctionnel. Cela signifie que l'on peut évaluer une fonction sur les chiffrés en même temps que nous en réduisons le bruit. Cet opérateur de bootstrapping fonctionnel est le composant principal de la boîte à outils développée dans cette thèse.

En particulier, nous étudions un procédé appelé « transchiffrement » qui permet de réduire la bande passante utilisée pour l'envoi de données homomorphes, ce qui a pour effet secondaire de réduire le délai d'attente lors de l'envoi de données. Ainsi, nous proposons deux nouvelles méthodes pour évaluer le circuit de déchiffrement de l'AES avec TFHE.

Dans un deuxième temps, nous proposons un jeu d'instruction universel pour processeur homomorphe. C'est-à-dire, nous construisons un jeu d'instruction qui fonctionne sur données chiffrées. Notre processeur permet d'effectuer n'importe quel algorithme avec des latences compétitives avec l'état de l'art.

En résumé, cette thèse propose des contributions améliorant l'état de l'art du cryptosystème homomorphe TFHE. Nous analysons et améliorons différentes méthodes de transchiffrement, explorons l'intégration de divers chiffrements par bloc et introduisons un jeu d'instructions universel capable d'évaluer n'importe quel algorithme de manière plus simple et plus performante que les approches actuelles. Ces avancées rapprochent le chiffrement homomorphe d'une utilisation pratique à grande échelle, ouvrant la voie à des services réellement sécurisés, privés et fiables, même dans le contexte de serveurs honnêtes mais curieux.

Remerciements

En débutant la thèse, j'avais bien en tête les témoignages de nombreux docteurs assurant à quel point le doctorat est une période éprouvante, stressante et harassante. J'avais même discuté avec un étudiant de troisième année m'assurant que son doctorat l'avait dégoûté des mathématiques.
Si pour moi ces trois dernières années ont été passionnantes, exaltantes et surtout heureuses, c'est parce que j'ai pu travailler au sein d'une équipe bienveillante et motivante et ai tout du long été encouragée par tous mes proches. C'est à vous tous que je dois la sérénité qui m'a habitée durant ces trois années. Merci.

En particulier, je tiens à exprimer ma profonde gratitude à mon directeur de thèse, Renaud, et à mon encadrant Aymen, pour leur accompagnement constant, leurs conseils éclairés et la confiance qu'ils m'ont accordée tout au long de ces années. Merci à tous les deux d'avoir été des papas de thèse si présents et patients. Vos conseils et vos encouragements permanents ont contribué à créer une dynamique de travail dans laquelle je me sentais légitime. Enfin, merci de m'avoir toujours poussée à donner le meilleur de moi-même (même quand cela impliquait de réimplémenter entièrement l'opérateur de division !). J'ai l'impression que les mots me manquent pour exprimer toute la gratitude qui m'habite ; alors simplement, merci.

Je remercie aussi chaleureusement mes rapporteurs, Philippe Gaborit et Melek Önen, qui m'ont fait l'honneur d'évaluer ce manuscrit et d'accepter de participer à la soutenance. Merci pour tout le temps investi et l'intérêt que vous avez porté à mes travaux. Un grand merci aussi à tous les autres membres du jury : Sonia Belaïd, Anne Canteaut, Mireille Fouquet et Louis Goubin, pour avoir accepté de prendre part au jury de ma soutenance. Vos remarques, vos questions et votre regard extérieur ont élargi mon champ de vision et m'ont permis de lier mes travaux à d'autres domaines, créant de nouvelles pistes qu'il me tarde d'explorer. Grâce à vous tous, la séance de torture que j'imaginais être la séance de questions s'est en réalité avérée être un moment d'échanges agréables. Je remercie Anne d'avoir accepté de présider ce jury. J'en profite pour également exprimer ma reconnaissance pour les interviews et interventions de vulgarisation auxquelles elle participe régulièrement ; car c'est en découvrant ses travaux dans un magazine de vulgarisation scientifique que j'ai eu envie de faire de la cryptographie mon métier.

Je remercie Isabelle Aubineau-Lanièce et Mireille Fouquet d'avoir accepté de prendre part à mes deux réunions de Comité de Suivi Individuel, au cours desquelles nos échanges et vos encouragements m'ont permis de prendre petit à petit confiance en moi et d'apprécier justement la valeur de mon travail. Mireille, un immense merci pour tes encouragements et ton suivi depuis le master. Merci d'avoir été une enseignante si pédagogue et toujours à l'écoute, et de toujours avoir eu foi en mes capacités.

Je remercie aussi toutes les chercheuses dont j'ai croisé la route et qui m'ont montré

qu'en tant que femmes, nous avons aussi notre place dans le monde de la recherche en informatique et mathématiques. À Adeline, Anne, Caroline, Ilaria, Melek, Mireille, Oana, Sihem, Sonia et à toutes les doctorantes de crypto, merci.

À mes collègues du LCYL Antoine, Jean-Paul, Nicolas Y., Oana, Olive et Pierre-Emmanuel, merci pour les nombreuses discussions passionnantes que nous avons eues, les déjeuners partagés et les pauses café animées qui m'ont fait me sentir chez moi au CEA. Je remercie aussi Benoît pour son écoute et son souci du bien-être des membres du laboratoire. Merci à Étienne et Alexandre pour les discussions au sortir de la navette ou autour d'un bon chocolat chaud réconfortant face aux intempéries du plateau. Une pensée particulière pour tous les doctorants avec qui j'ai pu partager notre (petit) bureau : Antonina, Guillaume, Jules, Marc, Marina, Robin, Thomas, Valentin. Ce petit cocon que nous avons créé va me manquer. Merci à tous d'avoir rendu ma vie de doctorante plus légère et plus humaine.

Je remercie Arnaud, Akram et Nicolas Q. pour nos échanges en conférences ou dans les couloirs de l'étage. Baptiste, Axel, merci pour votre bonne humeur à toute épreuve et désolée pour les croissantages à répétition. Mais c'était de bonne guerre.

Merci également à Nicolas B. et à l'équipe de CryptoExperts, pour les échanges fructueux, j'ai beaucoup aimé collaborer avec vous tous.

Merci à Henri, Jonathan, Pierre-Augustin, Virginio et tous les autres déjà cités d'avoir toujours répondu présent lors des soirées rugby. Vivement le prochain tournoi des Six Nations !

Marc, merci pour ta générosité, ta patience et ton écoute. Merci d'avoir été comme un grand frère pour moi pendant ces trois années ; j'ai adoré partir en conférence au Canada et en Bulgarie avec toi, ta présence rendait ces rendez-vous bien moins effrayants. J'espère que nous aurons très prochainement le plaisir de repartir ensemble.

Marina, si j'avais su, il y a huit ans que nous allions poursuivre toutes nos études, y compris le doctorat, côte à côte, je n'aurais pu imaginer la chance qui m'était donnée. Merci pour toutes les heures de cours passées parfois à bavarder, mais bien souvent à s'aider, pour tous les projets informatiques qui auront été bien sportifs (on se souvient notamment du robot et des bouches de l'enfer et de la souffrance qui portaient si bien leurs noms...), pour toutes les séances de révisions, mais aussi pour toutes les sorties pancakes, au zoo ou au cinéma. Sans parler de nos quelques vacances, puis conférences, partagées. Merci pour ton amitié infaillible qui a contribué à rendre ces trois années si agréables.

Merci à Alexandra, Clara, Hélène, Jeanne et Léa pour nos nombreuses sorties, seules ou en groupe. Ces moments partagés à discuter de tout sauf de mathématiques étaient toujours des points lumineux dans mon emploi du temps. Merci à vous toutes, et à Aloïs,

pour plus d'une décennie d'une amitié qui m'est si chère et qui a su rester forte malgré la distance et nos évolutions différentes. Merci à tous d'avoir répondu présent pour ma soutenance alors que je sais que quitter les mathématiques à la sortie du lycée était un soulagement pour plusieurs d'entre vous.

À Lucien, Maria et Veronica, merci de m'avoir accueillie à bras ouverts et de m'avoir instantanément fait me sentir à ma place parmi vous. J'ai adoré découvrir le ski et la montagne, ainsi que la culture autrichienne et la Suisse avec vous tous. J'ai beaucoup de chance de vous avoir pour deuxième famille.

Merci à mes grand-parents d'avoir été parmi mes plus grands supporters depuis ma naissance, mais aussi et surtout pendant la thèse. En particulier, merci à ma grand-mère Ginette d'avoir toujours suivi l'évolution de mes travaux avec des recherches régulières sur Google et merci à ma grand-mère Pierrette d'avoir suivi mes présentations en ligne. Elvire, Pierre-Alexis, j'avais déjà trois ans quand vous êtes nés, mais j'ai adoré grandir avec vous et je chéris tous les moments passés ensembles maintenant que le terme « vieillir » commence à être plus approprié que celui de « grandir »... Maman, Papa, vous m'avez toujours encouragée à suivre une voie qui me plaisait, et à entretenir ma fibre artistique. Grâce à vous, je connais le plaisir procuré par le dessin, la danse ou l'écriture. Mais aussi celui d'évoluer au sein d'un foyer aimant et attentif. Pour tout ce que je suis aujourd'hui, du fond du cœur merci.

Martin, tu me fais rire et me sentir aimée quotidiennement. Ta présence à mes côtés, même (ou devrais-je dire surtout) dans les moments durs, me permet de continuer à avancer sereinement vers l'avenir. Merci d'avoir été un appui si solide pendant ces trois années, mon parcours aurait été à la fois bien plus stressant et bien plus morne sans toi.

À vous tous qui êtes venus assister à ma soutenance, alors même qu'il fallait venir sur le plateau, un immense merci.

Contents

Introduction **13**

Publications and Communications **19**

I Background 21

1 FHE Overview **23**
1.1 An Introduction to Fully Homomorphic Encryption 23
1.1.1 Homomorphic Encryption Families 23
1.1.2 A Beginner's Introduction to LWE 24
1.1.3 Bootstrapping: a Solution to the Noise Issue 25
1.1.4 A Generic Definition 26
1.2 A Brief History of (F)HE 27
1.2.1 Main Schemes 27
1.2.2 Other Schemes 31
1.2.3 FHE in Practice 32
1.2.4 Recent Works 32

2 The Hitchhiker's Guide to TFHE **35**
2.1 Learning With Error (LWE) 35
2.2 Ring Learning With Error (RLWE) 36
2.3 TFHE's Parameters Set 36
2.4 Ciphertexts 37
2.5 Basic Operations on Ciphertexts 39
2.5.1 Addition 39
2.5.2 Multiplication 40
2.5.3 CMux 42
2.6 Bootstrapping 43
2.6.1 Building Blocks 43

2.6.2 Gate Bootstrapping . . . 45
2.6.3 Circuit Bootstrapping . . . 45
2.7 Programmable Bootstrapping . . . 46
2.7.1 A Vanilla Approach . . . 46
2.7.2 On the Evaluation of Non-Negacyclic Functions . . . 47
2.8 Tree-Based Method . . . 48
2.9 Multi-Value Bootstrapping . . . 50
2.10 Other Recent Works . . . 51
2.10.1 WoP-PBS . . . 52
2.10.2 Faster Circuit Bootstrapping . . . 53
2.10.3 Chaining Method . . . 53

3 Background on Transciphering **55**
3.1 The Magic of Transciphering . . . 56
3.2 Stream Cipher Based Transciphering . . . 57
3.2.1 BGV/BFV Friendly Ciphers . . . 57
3.2.2 CKKS Friendly Ciphers . . . 59
3.2.3 GSW Friendly Ciphers . . . 59
3.2.4 TFHE Friendly Ciphers . . . 60
3.3 Block Cipher Based Transciphering . . . 61
3.3.1 LowMC . . . 61
3.3.2 Chaghri . . . 62
3.3.3 PRINCE . . . 62
3.4 AES Back in the Game? . . . 62
3.5 Beyond the Boolean Approach with TFHE . . . 63

II Contributions **67**

4 Homomorphic AES Executions with TFHE **69**
4.1 A Short Reminder on the AES . . . 70
4.2 AES Goes Homomorphic over TFHE . . . 72
4.2.1 Full-LUT Approach . . . 72
4.2.2 Optimizing the Original AES Code . . . 72
4.2.3 Finally Going Homomorphic for Real . . . 74
4.2.4 Some More Details about the Homomorphization of the XOR Operator 76
4.2.5 Remarks and Perspectives on LUT Evaluated Operations . . . 78
4.2.6 Experimental Results . . . 79
4.2.7 Concluding Remarks on the First Contribution . . . 81
4.3 The Hippogryph Framework . . . 82
4.3.1 Generalization of the "p-encodings" Approach . . . 82
4.3.2 Design of Hippogryph . . . 84
4.3.3 Experimental Results . . . 88

5 `Mad Panthera`: Designing a General-Purpose 8-bit TFHE Processor Abstraction 95
5.1 Related Work . . . 96
5.1.1 FHE Virtual Processors . . . 96
5.1.2 FHE Compiler Approaches . . . 97
5.2 Optimal Basis Selection for LUT Evaluation . . . 98
5.2.1 Decomposition Basis Choice . . . 98
5.3 An FHE-Optimized Instruction Set . . . 101
5.3.1 Instruction Set Overview . . . 101
5.3.2 Notations for Homomorphic Operator Specifications . . . 102
5.3.3 Implementing Univariate Instructions . . . 103
5.3.4 Implementing Bivariate Instructions . . . 104
5.4 Other Types of Ciphertexts . . . 108
5.4.1 Working with Signed Inputs . . . 108
5.4.2 Support for Fixed-point Arithmetic . . . 109
5.4.3 Input/Output . . . 109
5.4.4 Bit Decomposition and Recomposition . . . 111
5.5 Instructions Timings . . . 112
5.6 From Instructions to Algorithms . . . 114
5.6.1 Testing a Few Elementary Algorithms . . . 114
5.6.2 Evaluation of an Elementary Neuron . . . 117
5.7 Comparison with Other Approaches . . . 119
5.8 Conclusion and Perspectives . . . 123

6 Running Other Block Ciphers with TFHE 125
6.1 Method and Tools . . . 125
6.1.1 The full-LUT Approach . . . 126
6.1.2 The Hippogryph Framework . . . 126
6.2 Running Block Ciphers over TFHE . . . 127
6.2.1 CLEFIA . . . 127
6.2.2 PRESENT . . . 130
6.2.3 PRINCE . . . 131
6.2.4 SIMON . . . 134
6.2.5 SKINNY . . . 136
6.3 Results . . . 138
6.3.1 Several Categories of Instructions . . . 138
6.3.2 A Fantasy FHE-friendly Block Cipher . . . 139
6.4 Conclusion . . . 140

Conclusion 141

Bibliography 144

A Complete Listing and Timings of Instructions **157**

B Instruction Set Implementation Details **161**
B.1 Univariate Operations 161
B.1.1 Shifts ((U)SHLi, (U)SHRi) 163
B.1.2 Rotations (ROLi, RORi) 164
B.1.3 Univariate Bitwise Operations 165
B.1.4 Other Exceptions 165
B.2 Bivariate Operations 166
B.2.1 Bitwise Operators 166
B.2.2 Addition (ADD, SUB) 166
B.2.3 Multiplication (MUL(M)) 167
B.2.4 Minimum and Maximum (MIN, MAX) 169
B.2.5 Division by a 4-bit Ciphertext (DIV4) 170
B.2.6 Modulo 170
B.2.7 Comparisons 172
B.2.8 Rotations and Shifts 173
B.2.9 Conditional Assignment 174

Introduction

Why is homomorphic encryption so interesting? What advantages does it offer compared to the encryption commonly used today?

To answer these important questions, let us consider the example of a cloud service provider. Users can easily use cloud services to store large amounts of data or perform calculations on that data. The user sends their data to the cloud, which stores or processes it as requested and then returns the results.

However, this data is often sensitive (for instance medical records, government information, financial data...). Users do not want the cloud provider—who may be honest but curious (i.e., able to perform the necessary calculations while also trying to learn as much as possible about the data)—to access the information being sent.

This is where homomorphic encryption becomes particularly valuable. It allows data to remain protected throughout the entire process while enabling calculations to be performed. With this method, the cloud learns nothing about the data itself, eliminating any risk of leakage. Even if a malicious user manages to access data from other cloud users, that information will still be encrypted.

Examples of applications are numerous: run an AI on medical data to detect cancer, do a keyword search among our e-mails, apply a filter to a photo, use a GPS, participate in a group call... Imagine that all of this is possible while conserving the privacy of the medical data, the e-mails, the photos, the localizations, the conversations... Even on a government scale, homomorphic encryption can have unsuspected applications. This is the case, for example, of an application proposed by the German Aerospace Center[1]: using homomorphic encryption to protect satellite trajectories[2]. Indeed, the number of satellites has increased massively in recent years, and hundreds of launches are already planned for the next few years. This results in an orbital congestion of the Low Earth Orbit, which increases the risk of satellite collision. Various methods exist to calculate collision probability based on satellite trajectories. However, precise predictions require access to actual trajectory data, which satellite operators are unwilling to share. As a result, only observational

[1] `https://www.dlr.de/en`

[2] `https://fhe.org/conferences/conference-2025/resources`

data is used. This imposes fundamental precision restrictions on collision risk assessments. Today, engineers spend many hours studying the trajectories of this or that satellite in order to prevent a collision with another. Sometimes, however, the estimates are not precise enough, and a collision is inevitable. Homomorphic encryption would enable satellite-owning countries to broadcast the trajectories of their satellites securely and to calculate the risk of collision with satellites from other countries accurately without learning anything about the trajectories of the satellites. For instance, in a multi-key setting, each party would encrypt its data with its own key and would be able to perform computations on encrypted data on its own. Of course, this would require a minimum of collaboration between the different countries and the participation of a neutral authority for the distribution of keys, but this would make it possible to avoid collisions and, therefore, the loss of satellites.

In a nutshell, the motivations behind homomorphic encryption are driven by the need for enhanced data privacy, security, and confidentiality in an increasingly digital and interconnected world. Homomorphic Encryption, which is moreover supposed to be quantum-resistant, seems to be the future of our privacy.

Contributions Overview

One of the central challenges in homomorphic encryption is managing the ciphertexts' noise, a small amount of error introduced during the encryption process. While this noise ensures security, it grows with each homomorphic operation performed on the ciphertexts. If the noise becomes too large, it can prevent correct decryption of the message.
This thesis focuses on the TFHE cryptosystem, a fully homomorphic encryption scheme known for its efficient noise reduction operation (called *bootstrapping*), which typically takes only a few milliseconds. One notable feature of TFHE is that its bootstrapping is "programmable". This means that during the ciphertext's noise refreshing operation, a function can be applied to the ciphertext without any additional cost.

TFHE is particularly recognized for its effectiveness with binary ciphertexts. Indeed, its efficient programmable bootstrapping allows for the homomorphic evaluation of logic gates and, consequently, Boolean circuits. In this thesis, we aim to move beyond this traditional Boolean approach and explore applications of programmable bootstrapping using integer ciphertexts. Our contributions show that this approach is often more efficient than the state-of-the-art binary approach and can be divided into two main areas: applications related to transciphering and the development of a homomorphic processor for 8-bit messages.

The Transciphering Playground

We present three contributions on the topic of transciphering (the ability to homomorphically turn low-overhead symmetrically encrypted data into homomorphically encrypted data) with TFHE. Indeed, one significant challenge with homomorphic encryption is its large expansion factor, that is to say, the ratio of the size of the encrypted message to the size of the plaintext message. Consequently, optimizing the data uplink transmission of fully homomorphic encryption to a remote server is key to FHE's practicality and has become a major research area in the community. One potential solution to this issue is transciphering, which involves sending secure messages using symmetric encryption (with an expansion factor of 1). This approach delegates the calculation of homomorphic decryption of the symmetric scheme to a server with superior computing power. As a result, the server can obtain homomorphic ciphertexts and perform the calculations requested by the client with complete security. The transfer of symmetrically encrypted data occurring instantly, the researchers focus on optimizing the symmetric decryption process within the homomorphic domain.

With regard to this problematic, we made the following contributions:

1. We proposed the first transciphering of AES since 2013, which reduced the sequential execution timings from 20 minutes to only 4 minutes.
   Indeed, in this contribution, we proposed an AES implementation using TFHE programmable bootstrapping with integer messages, running in about 4 minutes on a standard laptop (and less than 30 seconds over 16 threads). We detailed the modifications carried out on the original AES code as well as the optimized FHE operators we developed to lead to an efficient homomorphic evaluation of the algorithm.

2. We contributed to the design of a new framework called Hippogryph that combined the byte-oriented approach of contribution 1 and a generalization of a new boolean-oriented method called "p-encodings", which allowed us to propose a second transciphering of AES. We showed that this new design pays off, as we improve the new state-of-the-art (which was reduced to 46 seconds) for a TFHE execution of the algorithm between 30 and 45% and almost break the 1-second latency barrier with a mild amount of parallelism. Of course, this framework could also be beneficial for other use cases.

3. We finally performed a study in which we execute several state-of-the-art standardized symmetric block ciphers (namely AES, PRINCE, PRESENT, SKINNY, CLEFIA, SIMON) over TFHE integers ciphertexts using Programmable Bootstrapping. To do so, leveraging the method introduced in contribution 1, we transform all the algorithms into a sequence of LookUp Table evaluations

for 8-bit inputs. Then, using home-made optimized TFHE operators, we implement these symmetric schemes with the TFHElib library.
While this approach may not be the fastest in the state-of-the-art, it is a useful metric for comparing different schemes. Building on our results, we then leverage the Hippogryph framework for a second look. This second study allows us to determine whether the conclusions drawn from the first approach are consistent or if new and interesting insights emerge when using that second framework. In addition, we propose a classification of these symmetric block ciphers according to their friendliness with FHE evaluation. Indeed, we identify several categories of operations that influence the efficiency of the different block ciphers when run in the homomorphic domain. Finally, we sketch an ideal (yet hypothetical) block cipher following the guidelines established in our study.

An FHE Processor Abstraction

The key idea behind homomorphic encryption is to be able to perform *any* calculation directly over ciphertexts. In the early years of FHE, the hope was to achieve this goal by executing boolean circuits over ciphertexts encoding binary messages with both XOR and AND (homomorphic) gates. Although this computing model is universal, it also leads to many efficiency bottlenecks: for example, to merely perform a simple multiplication over $\mathbb{Z}_t$ ($t >> 2$), one has to perform many boolean operations, and even more so for more complex operations such as divisions. Compared to the raw boolean-circuit approach, TFHE offers a toolbox to mitigate its efficiency bottlenecks by supporting a non-binary (albeit still small) plaintext domain $\mathbb{Z}_{2^k}$, thus allowing to factor the evaluation of k-bit to k-bit boolean circuits in single bootstrapping operations.
Natural questions to explore are then the following. Can we build on the TFHE functional bootstrapping toolbox to achieve universal encrypted domain computations beyond the Boolean circuit approach? And at which computational cost? Can this be achieved from a restricted set of patterns based only on functional bootstrapping, hence with a homogeneous algorithmic structure?
With this work, we give a first answer to these questions by designing and implementing a general-purpose 8-bit FHE processor abstraction working over encryptions of bytes represented by pairs of TFHE ciphertexts.
The main contributions of this work are the following:

- We define a set of functional bootstrapping tools and optimal parameters to manipulate encryptions of 8-bit data through LookUp Table (LUT) dereferencing with TFHE ciphertexts. We designed this toolbox such that blind rotations and keyswitches (the most costly operations within TFHE bootstrapping) can be factored as much as possible to improve efficiency. By analogy to a real microprocessor, this can be seen as the micro-code level of a processor

abstraction.

- We then define a complete set of over 50 instructions suitable for working with TFHE encryptions of 8-bit data, including FHE-specific instructions as well as advanced operators such as conditional assignment, division, or even fixed-point arithmetic operations among many others. We provide strategies to efficiently instantiate each of these instructions using our LUT-dereferencing building blocks. To the best of our knowledge, we present the first ever concrete implementation of the Euclidean and decimal division operators over FHE not relying on the boolean circuit approach.

- We test our approach over several higher-level simple algorithms (sorting, average computation, finding the minimum or maximum of an array, ...) and provide extensive timing experiments. To the best of our knowledge, we provide the first FHE instantiation of a fixed-precision sigmoid function over 16 bits, leading to the FHE instantiation of *standard* neurons which can be seamlessly chained to enable the evaluation of larger (possibly recurrent) neural networks over encrypted data.

- We compare our approach to the state-of-the-art FHE compilers and related approaches, and we demonstrate performance improvements between 60% and 99% on most algorithms (except those inducing very small Boolean circuits).

- As we essentially define the first FHE-oriented ISA, this work is a significant first step towards defining virtual FHE machine architectures to bridge the gap with standard existing compilers and benefiting from their powerful code optimizations engines to both express more complex programs and improve their execution performances over FHE.

- Lastly, as a matter of perspectives, we carefully analyze the computational hotspots in the approach, providing cleanly defined candidate kernels of increasing complexity for low-level optimizations or even hardware acceleration.

Outline

This manuscript is split into two parts: Part I regroups different background notions divided into three chapters. Chapter 1 introduces Homomorphic Encryption and is essentially composed of FHE history and basic mathematical concepts. Chapter 2 gives an exhaustive overview of the TFHE scheme, from the Learning With Errors Problem to recent results, including the choice of parameters, native operations, ciphertext types, state-of-the-art bootstrapping operations, and recent bootstrapping techniques. Special attention is given to the methods used in the contributions of this thesis. Finally, Chapter 3 provides details about transciphering and reviews the

various symmetric schemes created over the years for this purpose.
Part II describes the contributions of this thesis. As emphasized above, we propose four contributions spread over three different chapters:

- In Chapter 4, we address the contributions on AES homomorphic evaluation for transciphering purposes and thus regroup contributions 1 and 2 presented in the above overview in a unique chapter. We compare our results to state-of-the-art implementations and show that our approaches are competitive, highlighting the recent improvements in FHE's practicality.
- In Chapter 5, we detail the creation of an 8-bit processor abstraction over TFHE. We explain how, from a small set of optimized bootstrapping tools, we proceed to design a universal instruction set of over 50 instructions. In particular, we design some FHE-optimized instructions (for instance, to efficiently compute the addition of two ciphertexts when at least one of them encrypts 0).
- Finally, in Chapter 6, we leverage the methods proposed in Chapter 4 to study several state-of-the-art symmetric block ciphers. That is to say, we implement a representative set of symmetric block ciphers using TFHE. These implementations allow us to compare the efficiency of this set of symmetric schemes and to categorize them, which further enables us to sketch out what the ideal block cipher for transciphering homomorphic data in integer mode might look like.

I wish you a pleasant reading.

Publications and Communications

Publications

- **Building Blocks for LSTM Homomorphic Evaluation with TFHE**, D. Trama, P.-E. Clet, A. Boudguiga, R. Sirdey. In *The 7th International Symposium on Cyber Security, Cryptology and Machine Learning*, CSCML 2023.
- **A Homomorphic AES Evaluation in Less than 30 Seconds by Means of TFHE**, D. Trama, P.-E. Clet, A. Boudguiga, R. Sirdey. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC 2023.
- **Designing a General-Purpose 8-bit (T)FHE Processor Abstraction**, D. Trama, P.-E. Clet, A. Boudguiga, R. Sirdey, N. Ye. In *Transactions on Cryptographic Hardware and Embedded Systems*, TCHES 2025, Volume 2.
- **Further Improvements in AES Execution over TFHE**, S. Belaïd, N. Bon, A. Boudguiga, R. Sirdey, D. Trama, N. Ye. In *Communications in Cryptology*, CiC 2025, Volume 2.
- **Running Standard Block Ciphers Beyond AES with TFHE: Experiments and Lessons Learnt**, D. Trama, A. Boudguiga, R. Sirdey. In submission.

Communications

- Building Blocks for LSTM Homomorphic Evaluation with TFHE:
  - Talks: AICRYPT (2023), CSCML (2023), Journées Codes et Cryptographie (2023), PEPR SecureCompute (2023)
- A Homomorphic AES Evaluation in Less than 30 Seconds by Means of TFHE:

  - Talks: WAHC (2023), PEPR SecureCompute (2023), PEPR TrustInCloud (2024)
  - Poster: FHE.org (2024)

- Designing a General-Purpose 8-bit (T)FHE Processor Abstraction:
  - Invited talks: FHE.org MeetUp (2024), ANSSI crypto meeting (2025)
  - Talk: Journées Codes et Cryptographie (2025)
  - Poster: FHE.org (2025)

- Running Standard Block Ciphers Beyond AES with TFHE: Experiments and Lessons Learnt:
  - Poster: FHE.org (2025)

Other Activities

- I have had the opportunity to teach C++ for first year students at ENSTA in 2023 and 2024. It was a really nice experience.

- The French association *femmes et mathématiques*[3] aims to encourage young girls to pursue studies and careers in mathematics and raises awareness about gender inequalities in science. As part of this mission, the association organizes special events called "Filles et maths-info : une équation lumineuse", which are dedicated to encouraging high school girls to explore careers in mathematics and science. These events combine lectures and personal testimonies from women in mathematics. Since 2019, I take part in these events as often as possible to share my own academic journey in mathematics and computer science, hoping to inspire young girls and show them that they too have a place in these fields.

- I always joyfully accept any opportunity to promote mathematics among young girls (and boys), even outside of the *femmes et mathématiques* events. For instance, I recently participated to the "Maths C pour L"[4] program. I also like to participate in "forum des métiers" in high schools to speak about PhD in mathematics.

[3] `https://femmes-et-maths.fr/`.
[4] `https://smf.emath.fr/smf-dossiers-et-ressources/stages-math-c-pour-l`.

Part I

Background

Chapter 1

FHE Overview

1.1 An Introduction to Fully Homomorphic Encryption

1.1.1 Homomorphic Encryption Families

Homomorphic Encryption (HE) is a recent corpus of cryptographic techniques. Its particularity is to enable computations directly over the encrypted messages. As its name suggests, it refers to schemes having properties captured by the mathematical concept of *homomorphism*.

Homomorphism. A homomorphism is an application f defined from a set E with internal law $\top$ to a set F with internal law $\bot$ such that: $\forall\ (x, y) \in E \times E, f(x \top y) = f(x) \bot f(y)$.

This manuscript uses the following notations for ciphertexts: $[m]$ denotes an encryption of a message m. It is often referred to as $c = [m]$. We will also use the less common notation $[\![m]\!]$ to refer to the set of valid encryptions of a message m. Thus, we have $c = [m] \in [\![m]\!]$. With such notations, we can express the homomorphism relation between the plaintext space E and the ciphertext space F in the following way:

$$[x \top y] = [x] \bot [y].$$

Homomorphic Encryption schemes are not all part of the "Fully Homomorphic Encryption" family. Indeed, to be part of this category, a scheme must be homomorphic for both addition and multiplication (meaning that there exists a law in the ciphertext domain corresponding to the addition in the plaintext domain, and there exists a law in the ciphertext domain corresponding to multiplication in the plaintext domain). Thus, we have several categories of HE schemes that differ mainly in

the types and number of mathematical operations that can be performed on ciphertexts. Here, we detail the four types of existing HE schemes.

Partially Homomorphic Encryption (PHE) scheme. A cryptosystem is Partially Homomorphic if only one operation (addition or multiplication, but not both) can be performed (as often as required) on a ciphertext. PHE schemes are the first HE that were proposed. For instance, the RSA scheme [RSA78] is part of the PHE family as it is homomorphic for multiplication (although it is only true for its vanilla form). The ElGamal [ElG85] and Paillier [Pai99] cryptosystems are other examples of PHE schemes.

Somewhat Homomorphic Encryption (SHE) schemes. A cryptosystem is Somewhat Homomorphic if both addition and multiplication operations can be performed, but only a limited number of times, after which correct decryption is no longer assured.

Leveled Homomorphic Encryption (LHE) schemes. A cryptosystem is called "Leveled Homomorphic" if, when knowing the number and types of operations to be performed in the ciphertext domain, there exists a set of parameters that allows the complete evaluation of the said circuit. The major drawback of LHE is that the size of the parameters used for the scheme depends on the multiplicative depth (that is to say, the number of consecutive multiplications) involved in the calculations. And the more significant the parameters, the slower and more costly the operations.

Fully Homomorphic Encryption (FHE) schemes. A Fully Homomorphic Encryption (FHE) cryptosystem allows to chain a potentially unbounded number of additions and multiplications over ciphertexts.

To set the scene, this thesis specifically focuses on Fully Homomorphic Encryption and its applications for general-purpose uses. We primarily investigate how to homomorphically evaluate various functions using the TFHE scheme across different types of messages, primarily integers, but also including fixed-point arithmetic messages.

1.1.2 A Beginner's Introduction to LWE

Learning With Error (LWE) is a mathematical problem introduced in 2005 [Reg05] that allows hiding a secret value by introducing noise to it. The LWE problem is conjectured to be hard to solve, and thus, several cryptosystems' security relies on it. This section briefly introduces the problem and gives first hints on homomorphic ciphertext structure.
Given a binary secret key $s \in \{0,1\}^n$, a way to hide the value of the message m_1

is to uniformly pick a vector $a_1 \in \mathbb{Z}_q^n$ and to compute $b_1 = \langle a_1, s\rangle + m_1$. The key s being secret, a_1 can be public without leaking any information on b_1, and thus m_1 is indeed private.
But what happens if we hide n messages $m_1, \cdots, m_n$ with the same secret key s? Each message is well hidden, but a malicious adversary can create a linear system with all the b_i and retrieve each coefficient of s. Once s is known, the attacker can easily remove the $\langle a_i, s\rangle$ term associated with each b_i and retrieve all the messages. To avoid this security issue, we add a *noise*, or *error* e sampled from a Gaussian to each b_i. This way, the malicious attacker does not obtain n pairs of the form $(a_i, \langle a_i, s\rangle + m_i)$ but n pairs of the form $(a_i, \langle a_i, s\rangle + m_i + e_i)$. Such a pair is called an *LWE sample*. Each e_i being sampled from a Gaussian with an appropriate variance, it is conjectured that an attacker can no longer retrieve the secret key s with a polynomial attack (which is why the LWE problem is "hard"). Almost all homomorphic ciphertexts are structured as LWE samples or ring variants, ensuring that the messages will not be compromised if the key s is indeed secret.

1.1.3 Bootstrapping: a Solution to the Noise Issue

As most homomorphic ciphertexts have the structure of an LWE sample, when performing homomorphic operations, such as adding two encrypted messages, the noises in each ciphertext are summed. For instance, let us take two LWE ciphertexts $c_1 = (a_1, \langle a_1, s\rangle + m_1 + e_1)$ and $c_2 = (a_2, \langle a_2, s\rangle + m_2 + e_2)$. As they share the same secret key s, we can perform operations on them. The addition of these two entities gives us $c = (a_1 + a_2, \langle a_1 + a_2, s\rangle + m_1 + m_2 + e_1 + e_2)$, which is a well-formed LWE encryption of the message $m_1 + m_2$ with noise $e_1 + e_2$. The resulting ciphertext c thus has a new noise, which is more significant than in the two previous ciphertexts c_1 and c_2.
In the case of a multiplication of a ciphertext with a constant α, the noise is multiplied by this constant. For instance, we can homomorphically compute $\alpha \times c_1 = (\alpha \times a_1, \langle \alpha \times a_1, s\rangle + \alpha \times m_1 + \alpha \times e_1)$ which is a well-formed encryption of $\alpha \cdot m_1$.
So, at each operation performed, the resulting ciphertext is subject to a growth of its noise level. The challenge then arises when the noise becomes so large that it hinders the successful decryption of the message. Indeed, if $|e|$ associated to ciphertext $c = (a, b = \langle a, s\rangle + m + e)$ is two large, the decryption $\mathsf{Dec}(c) = \lfloor b - \langle a, s\rangle \rceil = \lfloor m + e \rceil$ can return $m + 1$ or $m - 1$ instead of m (with $\lfloor \cdot \rceil$ the operation returning the closest valid value in the plaintext space). To ensure that the decryption of the results from homomorphic computations is accurate, the noise associated with the ciphertexts must remain low enough to allow correct decryption. One way to avoid this is to reduce the noise level several times throughout the computation process (without lowering the data's security).

The easiest and most straightforward way to eliminate the noise of a ciphertext

is to decrypt it. However, such a solution poses a problem. Indeed, remember that the aim of FHE is to enable a client to encrypt its data and send it to a server that will securely process it, without learning anything about the data. But to decrypt ciphertexts, the server needs the secret key, which is, of course, a security flaw. And once decrypted, the protection of the messages no longer exists. In this case, FHE is thus no longer better than the Decrypt-then-Process-then-Encrypt approach that can be implemented using usual cryptography. A secure (super clever out-of-the-box) solution to these issues is to *homomorphically* evaluate the decryption circuit of the HE scheme. That is to say, the server is given an encryption of the secret key that allows it to compute the decryption of ciphertexts in the homomorphic domain. In other words, the server *recrypts* the message by homomorphically computing $\mathsf{Enc}(\mathsf{Dec}(c))$. This results in new ciphertexts with a new noise level only induced by the bootstrapping procedure. This noise level, although higher than that of a fresh ciphertext, is small enough to allow further homomorphic computations. This process of evaluating the decryption circuit of the homomorphic scheme in the ciphertext domain to reduce the noise level is called *bootstrapping*, and the encryption of the secret key used to perform it is referred to as the *bootstrapping key*.

1.1.4 A Generic Definition

Let $\mathcal{K}$, $\mathcal{M}$, and $\mathcal{C}$ denote a key space, a plaintext domain, and a ciphertext domain. Let $\mathsf{KeyGen} : 1^\lambda \longrightarrow \mathcal{K}$ be a function generating keys, $\mathsf{Enc} : \mathcal{M} \longrightarrow \mathcal{C}$ the encryption function, and $\mathsf{Dec} : \mathcal{C} \longrightarrow \mathcal{M}$ the corresponding decryption function. Then, an FHE scheme is a quadruple $\mathcal{E} = (\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Eval})$ such that:

- $(\mathsf{ek}, \mathsf{dk}) \leftarrow \mathsf{KeyGen}()$
- $c \leftarrow \mathsf{Enc}(\mathsf{ek}, m)$
- $m \leftarrow \mathsf{Dec}(\mathsf{dk}, c)$

Let us remind that for a message $m \in \mathcal{M}$, we denote $[\![m]\!] \subset \mathcal{C}$, the set of all its valid encryptions, which we sometimes refer to as the ciphertext class of m. Let $\mathcal{F}$ be the function domain of Eval i.e., $\mathsf{Eval} : \mathcal{F} \times \mathcal{C}^* \longrightarrow \mathcal{C}$ is such that for all $(\mathsf{ek}, \mathsf{dk}) \in \mathcal{K}$, all $f \in \mathcal{F}$ and all $m_1, \cdots, m_K \in \mathcal{M}^K$,

$$\mathsf{Eval}(f, \mathsf{Enc}(\mathsf{ek}, m_1), \cdots, \mathsf{Enc}(\mathsf{ek}, m_K))) \in [\![f(m_1, \cdots, m_K)]\!].$$

Also, an FHE scheme is said to be "correct" if, and only if for all $(\mathsf{ek}, \mathsf{dk}) \in \mathcal{K}$, all $f \in \mathcal{F}$ and all $m_1, \cdots, m_K \in \mathcal{M}^K$,

$$\mathsf{Dec}(\mathsf{dk}, \mathsf{Eval}(f, \mathsf{Enc}(\mathsf{ek}, m_1), \cdots, \mathsf{Enc}(\mathsf{ek}, m_K))) = f(m_1, \cdots, m_K) \tag{1.1}$$

holds with overwhelming probability over the randomness space of $\mathcal{E}$. This thesis focuses on TFHE, a scheme that satisfies this correctness assumption... under one

condition.
Indeed, the probability of correctly decrypting TFHE ciphertexts depends on the parameters selected for the cryptosystem. These parameters must ensure that the noise of fresh ciphertexts—meaning newly encrypted messages that have not undergone any calculations—is, with overwhelming probability, consistently low enough for successful decryption. Additionally, the chosen parameters should also guarantee that the noise from messages outputted by bootstrapping remains sufficiently low to allow for correct decryption in nearly all cases. In practice, the parameters are selected so that if bootstrapping is performed frequently enough, decryption errors do not occur.

1.2 A Brief History of (F)HE

Homomorphic encryption schemes have been developed using different approaches. Specifically, fully homomorphic encryption schemes are often grouped into generations corresponding to the underlying approach.

1.2.1 Main Schemes

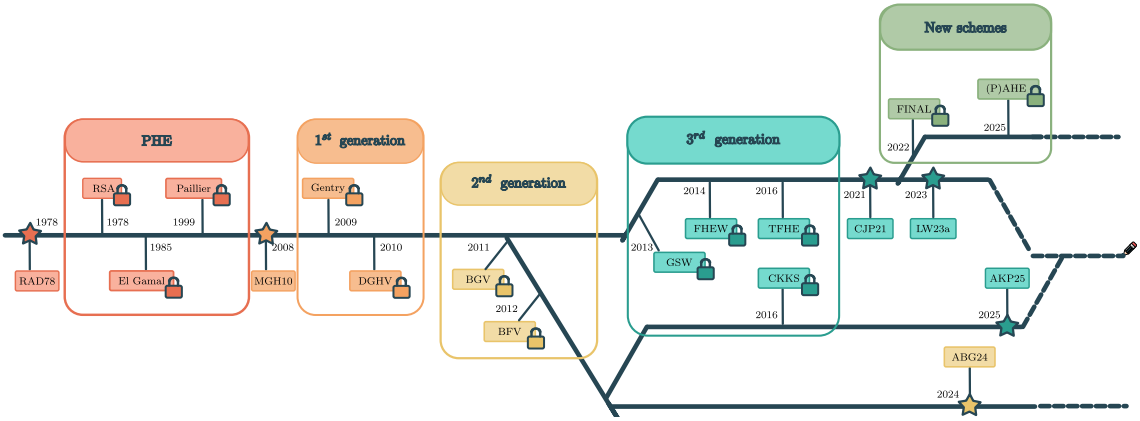


Figure 1.1: Order of appearance of the different HE schemes and groundbreaking findings.

Pre-FHE

In their work of 1978 on data banks and privacy homomorphisms, Rivest, Adleman, and Dertouzos [RAD78] first highlighted the need for encryption techniques allowing operations on encrypted data without requiring prior decryption. In this work, they take the example of companies' data banks that require privacy even during outsourced computations and expose the challenge of designing an FHE. For

almost 30 years, no solution is found, and data privacy during computations remains a cryptographic grail even though some partial results were achieved with public-key cryptosystems. For instance, the RSA scheme [RSA78], introduced in 1979 by Rivest, Shamir and Adleman allows an unbounded number of modular multiplications to be performed. Similarly, the ElGamal public-key cryptosystem proposed in 1985 [ElG85] also allows an unbounded number of modular multiplications on the ciphertexts. But neither of these two cryptosystems allows additions to be performed. On the contrary, the Paillier cryptosystem (1999) [Pai99], a public-key encryption scheme relying on the decisional composite residuosity assumption, is only homomorphically additive.
Some additional proposals towards fully homomorphic encryption emerge shortly before 2010. For instance, Melchor et al. proposed a generic method to construct LHE schemes allowing d multiplications and relying on the hardness of problems in integer lattices [MGH10]. In a nutshell, they present a cryptographic scheme that extends the capabilities of traditional additively homomorphic encryption systems. To do so, they define a theoretical construction called "chained encryption schemes", which enable efficient evaluation of polynomials of degree d over encrypted data.

First Generation

In 2009, Gentry gives the first design for an FHE based on ideal lattices [Gen09]. He proposes a construction for a SHE scheme with a special ability: the scheme can homomorphically evaluate its own decryption circuit. In other words, the scheme parameters allow the evaluation of the decryption algorithm in the homomorphic domain. This operation, used to "reset" ciphertexts resulting from homomorphic computations that have reached the maximum number of permissible operations, is referred to as *bootstrapping*. Once a ciphertext is close to reaching the limits of operations of the SHE, bootstrapping is applied, and new operations can further be performed. Note that bootstrapping transforms an SHE scheme into an FHE one, only if the SHE scheme allows the execution of its decryption algorithm and one additional homomorphic operation to be performed before reaching the limit beyond which a message will no longer decrypt correctly. Even if this theoretical work is groundbreaking, its first implementation in 2011 [GH11] shows that it takes about 30 minutes to perform a single-bit operation in the homomorphic domain, which makes it impractical for real-world uses.

In 2010, Dijk et al. [DGHV10] introduce a new homomorphic encryption scheme that operates over integers using elementary modular arithmetic. This approach simplifies the conceptual framework compared to previous schemes that relied on ideal lattices over polynomial rings. The authors propose a somewhat homomorphic encryption scheme that supports addition and multiplication over integers. By applying Gentry's bootstrapping technique, they convert this SHE scheme into an FHE one, enabling arbitrary computations on encrypted data. As a result, this new scheme is conceptually simpler than Gentry's ideal lattice scheme while maintaining

similar properties in terms of homomorphic operations and efficiency.

To summarize, the core ideas of the first generation schemes are the following:

- Start with an SHE scheme that supports a limited number of additions and multiplications.
- Use a process called bootstrapping to reduce noise in a ciphertext by homomorphically evaluating the decryption function.

This enables the scheme to evaluate arbitrary circuits, making it fully homomorphic. However, the schemes of the first generation are highly inefficient in practice and based on complex constructions, making them hard to implement.

Second Generation

The second generation of homomorphic cryptosystems builds upon techniques developed by Brakerski, Gentry, and Vaikuntanathan, which led to significantly more efficient HE based on the Ring Learning with Errors (R)LWE assumptions. In [MGH10], the authors also introduced a novel tensoring technique, which was later used by Brakerski et al. who proposed the BGV scheme [BGV12]. BGV is designed to perform a fixed number of chained operations (depth-limited circuits) without bootstrapping, making it more efficient than earlier schemes. As such, it is not an FHE scheme, but an LHE one, and each homomorphic operation still adds noise to the ciphertext. To control the noise growth, BGV includes techniques like modulus switching and key switching. One particularity of BGV is that it supports SIMD-style (Single Instruction, Multiple Data) operations, meaning one ciphertext can encrypt many values at once and operate on them in parallel.
The following year, Fan and Vercauteren introduced the FV scheme [FV12], which ports Brakerski's FHE scheme based on the LWE problem [Bra12] to the ring-LWE setting. This scheme was later referred to as BFV to acknowledge the influence of Brakerski's anterior work. The BFV scheme supports exact integer arithmetic on encrypted data. It is a leveled homomorphic encryption scheme, but bootstrapping can be added if full homomorphism is required, even if it comes with a significant computational cost. To ensure correctness as operations are performed, BFV includes mechanisms for managing noise, such as modulus switching and relinearization (which reduces the size of ciphertexts after multiplication). These techniques help maintain ciphertext integrity through several operations. BFV also supports batching, allowing multiple integers to be packed into a single ciphertext and processed simultaneously.

These new schemes demonstrate much slower noise growth during homomorphic computations than earlier models and propose techniques allowing the packing of multiple plaintext values into a single ciphertext, enabling operations on all these

plaintext values in a Single Instruction Multiple Data manner. This packing is referred to as "batching" and is one of the cornerstones of these systems.

Overall, the core idea of the second generation schemes involves converting an LHE scheme into an FHE scheme via bootstrapping. These constructions are more efficient than the first generation, and many remain widely used and implemented today.

Third Generation

In 2013, Gentry, Sahai, and Waters introduced a new scheme (GSW) [GSW13] designed to be asymptotically faster than second-generation schemes by using a different approach. The GSW scheme operates in $\mathbb{Z}_q$, the ring of integers modulo a prime number q, and its security relies on the LWE problem. This lattice structure allows for efficient computations and compact representations. Indeed, the addition of two ciphertexts is straightforward and involves component-wise addition of the ciphertext vectors. Multiplication of two ciphertexts is more complex and involves matrix multiplication and noise management techniques to ensure that the resulting ciphertext remains valid for further homomorphic operations. GSW simplified bootstrapping by making the ciphertext structure naturally suitable for it. The use of matrix operations and gadget matrices enables efficient bootstrapping, leading to better performance in practical homomorphic computations.

Brakerski and Vaikuntanathan noted that for certain types of circuits, the GSW cryptosystem exhibits a slower rate of noise growth, resulting in better efficiency and enhanced security [BV14]. Alperin-Sheriff and Peikert then described a very efficient bootstrapping technique based on this observation [ASP14]. Subsequent improvements led to the development of efficient ring variants of the GSW cryptosystem: FHEW [DM15] in 2014 and TFHE [CGGI19] in 2016. The key improvement in FHEW is that it uses modular arithmetic and binary operations (such as XOR and AND) that are particularly well-suited for efficient bootstrapping. Consequently, the FHEW scheme was groundbreaking as it demonstrated that by refreshing ciphertexts after every individual operation, the bootstrapping time could be reduced to a fraction of a second. The efficiency of FHEW was further enhanced by the TFHE scheme, which features a ring variant of the bootstrapping procedure utilizing a method similar to that of FHEW. Essentially, TFHE can be seen as adapting FHEW, which incorporates both standard Learning With Errors and ring-GSW over the Torus. While FHEW was an early attempt at improving FHE for binary data, TFHE goes further, providing even more efficient bootstrapping and operations.

In 2016, the Cheon-Kim-Kim-Song (CKKS) scheme was proposed [CKKS17], introducing a novel FHE method that supports approximate arithmetic. Unlike other schemes that work with integer plaintexts, CKKS is used to perform efficient computations on real or complex numbers. Since it is designed for approximate values, small errors introduced during encryption and computation are part of the results. Despite this, the errors can be controlled and kept small enough for many practi-

cal applications. The scheme supports addition and multiplication on ciphertexts, with each operation slightly increasing the noise. To manage this, CKKS uses a process called "rescaling" to reduce noise and keep the computation stable. Note that CKKS is considered leveled homomorphic, meaning it is meant to perform a limited number of operations without bootstrapping. However, a costly bootstrapping procedure can be applied to refresh ciphertexts and allow more operations.

These third-generation schemes, and especially TFHE and CKKS, are still the most widely studied today, and numerous works are constantly improving them, bringing them closer to the barrier of practicability.

1.2.2 Other Schemes

Some more recent schemes are also noteworthy, even if they have not been studied deep enough to ensure their security level. Moreover, these schemes are often theoretical novelties with no provided implementation to allow comparisons with the state-of-the-art. Still, these new approaches may lead to a new generation of HE schemes.

These include FINAL [BIP+22], a proposition of an FHE scheme which security relies both on the NTRU and LWE assumptions. By combining these two primitives, the FINAL scheme seems to outperform the efficiency of both key generation and encryption/decryption operations of TFHE. Indeed, by utilizing NTRU's structure for keys, the authors claim a 45% improvement in the size of the keys and a 28% faster bootstrapping. The FINAL scheme also reduces the size of the ciphertexts, which is a significant improvement for practicality as smaller ciphertexts reduce storage and communication costs, which are crucial for practical deployments. However, the security provided by the chosen parameters still remains to be studied, as some NTRU parameters are especially vulnerable against subfields lattice attacks [ABD16].

Another recent proposition is given in [AMDG25], where the authors describe the first code-based SHE scheme relying on random ideal codes. These new scheme supports homomorphic operations such as addition, multiplication, and plaintext multiplication. Moreover, a candidate bootstrapping for this code-based scheme is also depicted. However, the scheme suffers from two limitations that hamper its categorization as fully homomorphic. First, the number of multiplications is limited, and second, there is an upper bound to the number of independent ciphertexts that can be published without a polynomial key recovery attack, meaning that this proposition is not yet practicable, although it may pave the way to a new type of HE schemes.

1.2.3 FHE in Practice

Several FHE libraries have been developed to implement many of the above schemes, each offering varying levels of efficiency, ease of use, and support for different cryptographic functionalities. These libraries are actively developed and contribute to the growing practical use of FHE. In Table 1.1, we summarize the most commonly used libraries and those encountered during the course of this thesis.

Table 1.1: A list of common libraries with the cryptosystems they support, along with their year of creation and programming language.

library	creation	language	BGV	BVF	FHEW	TFHE	CKKS
SEAL	2016	C++	✓	✓			✓
HElib	2013	C++	✓				✓
OpenFHE	2021	C++		✓	✓	✓	✓
Lattigo	2020	Go	✓	✓			✓
HEANN	2020	C++					✓
TFHElib	2016	C++				✓	
tfhe-rs	2020	Rust				✓	
TFHEpp	2018	C++				✓	

1.2.4 Recent Works

FHE research is hectic, and a lot is going on! In the last few years, contributions to FHE have grown immensely and span a wide range of research areas. For example, there has been a great resurgence of interest in transciphering. Since the NIST's call[1] in 2022 for the standardization of threshold schemes with a dedicated FHE part, there have been numerous homomorphic implementations of AES. ZAMA has even launched a bounty competition for the fastest implementation of AES with tfhe-rs[2]! Moreover, since 2020, a dozen symmetric ciphers specially dedicated to FHE transciphering have been designed. And it seems that it is only the beginning, as the quest for the ideal symmetric cipher is far from being over.
On a more theoretical level, several works have developed the notions of security of HE schemes and have shown that the security offered by homomorphic schemes goes beyond $CCA1$ [MN24, BCF+25]. Similarly, several attacks carried out by different teams [CSBB24, CCP+24] have shown the weaknesses of all cryptosystems when faced with a CPA^D adversary: in all cases, keys were recovered! Luckily, the authors gave easy countermeasures, which, however, have an important impact on performance.
Numerous works have also proposed optimizations of cryptosystem parameters,

[1] `https://csrc.nist.gov/Projects/threshold-cryptography`.
[2] `https://github.com/zama-ai/bounty-program/issues/135`.

such as key size reduction or tricks to improve bootstrapping efficiency [BBB+23a, LMSS23, BCL+24, HKX+25]. Other active research areas include scheme switching, which refers to protocols for efficiently switching from one homomorphic scheme to another to benefit from the best of each cryptosystem. There is also work on proposing threshold versions of homomorphic cryptosystems. Finally, we also see FHE compilers emerging or being optimized over time, and even some ISAs starting to appear (as we shall later see in this manuscript).

Due to this unbelievable effervescence, and during this thesis alone, several of our crypto team's works came out within a few days of competing works. The contributions of this thesis are no exception in this fast-paced world. In particular, the first contribution on the AES really resembled a sprint race. And indeed, our work, the first on AES since 2013, was followed by many that have continued to improve the use case, allowing us to later establish a git repository serving as a benchmark for the community. Similarly, we were not much calmer regarding `Mad Panthera`, our homomorphic instruction set, as various teams began to emphasize their work on homomorphic compilers and other integer computing applications utilizing TFHE.

And when it wasn't the competition within the FHE community that was making the news, the danger came from outside. Indeed, in April 2024, our entire crypto team, and certainly all FHE teams worldwide, held their breath. A researcher of Shanghai had just published a paper in which he proposed a quantum algorithm that breaks LWE. The alarm was sounded within the community. Would the emergence of quantum computers finally put an end to this fabulous enthusiasm for FHE? Since no one in the team was an expert in quantum algorithms, we spent several weeks hanging on N. P. Smart's blog[3] as he dissected the article day after day for the community. Finally, an error was found in the new algorithm, putting an end to the angst of the team's PhD students. N. P. Smart summed up the general feeling very well: "So the panic in lattice quarters is now over.".

What has particularly struck me in recent works, apart from the amazing quantity of research being done everywhere, is the end of the status quo on the respective advantages and disadvantages of TFHE and CKKS. Indeed, for almost ten years, FHE researchers have generally chosen one of the two most recent cryptosystems and stuck to it. The two schools never cease to bicker amicably, each claiming its cryptosystem to be the most efficient. On the TFHE side, the emphasis is on bootstrapping speed and programmability. On the CKKS side, batching is emphasized, which delivers an advantageous throughput despite much more excessive latencies. For some time now, the respective advantages and disadvantages are less striking. Indeed, in 2023, two papers published at EUROCRYPT presented batching for TFHE [LW23a, LW23b]. Although this theoretical work has yet to be implemented, it marks a new era for the cryptosystem. Even more recently, a succession of papers

[3] `https://nigelsmart.github.io/LWE.html`.

([BCH+24, CHKS25, AKP25] to only cite a few) has improved CKKS bootstrapping, eventually proposing a programmable approach. These new powerful tools are reigniting the friendly battle between the two parties, which promise to give birth to some more beautiful advances in the next few years.

Leaving aside these technical advances, 2025 has already brought its share of celebrations as we witnessed the first real-world applications of FHE. I was lucky enough to participate in the Real World Crypto conference in March, and I was delighted to see that major players on the international scene, known to the general public, had finally taken a look at the FHE! For instance, Apple presented an implementation of BGV to efficiently secure queries for its search engine, allowing thousands of queries to be processed within seconds [ABG+24]. Similarly, Nokia proposed using FHE to secure audio streams during group calls [4]. They developed an initial version of this product, also based on BGV, which demonstrated imperceptible latency as I was able to see for myself during a demo at the Real World Crypto conference. I found it amazing to see that this exciting field of study is finally becoming practicable, fifteen years after Gentry's first work! I do not doubt that in the years to come, there will be more and more concrete applications, and I look forward to further contributing to these advances.

[4] `https://www.youtube.com/watch?v=PMmBfXC1HQE`.

Chapter 2

The Hitchhiker's Guide to TFHE

TFHE is a Fully Homomorphic Encryption scheme in which the ciphertext space is the torus (thus the name "Torus FHE"), which is the object of this thesis. Let us introduce some notations that we use in this manuscript.

Notations 1. Let $\mathbb{A}$ be a ring and $n, N, q \in \mathbb{N}$. $\mathbb{A}_q^n$ is the set of vectors of dimension n with coefficients in $\mathbb{A} \pmod q$ and $\mathbb{A}_N[X]^n$ is the set of vectors of dimension n whose coefficients are polynomials in $\mathbb{A} \pmod{(X^N + 1)}$.

Notations 2. We work on the torus $\mathbb{T} = \mathbb{R}/\mathbb{Z}$, the set of real numbers modulo 1.

Like all encryption systems, TFHE's security relies on a hard problem. For TFHE, it is the Learning With Error problem, which takes several forms depending on the ciphertext space.

2.1 Learning With Error (LWE)

We present here a revisited version of the LWE problem introduced by Chillotti et al. in [CGGI19], called the Torus LWE. Let $n \geq 1$ be an integer, $s \in \mathbb{Z}^n$ and ξ be a distribution on $\mathbb{R}$. We define $\mathbf{LWE}_{s,\xi}$ as a pair $(a, b) \in \mathbb{T}^n \times \mathbb{T}$ obtained by uniformly drawing a in $\mathbb{T}$ and taking $b = \langle a, s \rangle + e$ with $\langle \cdot, \cdot \rangle$ the dot product. The *error* e is sampled from ξ.
Finally, let $\mathcal{S}$ be a distribution on $\mathbb{Z}^n$. We can then define the following two problems:

- the LWE search problem: given a polynomially large set of $\mathbf{LWE}_{s,\xi}$ samples, recover the fixed secret s drawn from $\mathcal{S}$.
- the LWE decision problem: given a polynomially large set of $\mathbf{LWE}_{s,\xi}$ samples, distinguish elements following the distribution of $\mathbf{LWE}_{s,\xi}$ from elements chosen uniformly over $\mathbb{T}^n \times \mathbb{T}$, for a fixed s drawn from $\mathcal{S}$.

2.2 Ring Learning With Error (RLWE)

The Ring-LWE problem is a structured variant of the LWE problem, adapted to elements of a ring instead of general vectors. In the specific case of TFHE, the ring is $\mathbb{T}_N[X]$. We define $\mathbf{RLWE}_{s,\xi}$ as a pair $(a, b) \in \mathbb{T}_N[X]^n \times \mathbb{T}_N[X]$ obtained by uniformly drawing a in $\mathbb{T}_N[X]^n$ and taking $b = \sum_{i=1}^{n} a_i \times s + e$ with a_i the ith coefficient of a, that is to say a polynomial in $\mathbb{T}_N[X]$. The *error* e is a polynomial in $\mathbb{T}_N[X]$ which coefficients are sampled from ξ. The RLWE decision problem consists in distinguishing true TRLWEs from randomly chosen pairs in $\mathbb{T}_N[X]^n \times \mathbb{T}_N[X]$. The TRLWE search problem consists in finding the private polynomial s given an arbitrary set of TRLWE ciphers. More formally, the problems are the following:

- the RLWE search problem: given a polynomially large set of $\mathbf{RLWE}_{s,\xi}$ instances, find the fixed secret s drawn from $\mathcal{S}$.
- the RLWE decision problem: given a polynomially large set of $\mathbf{RLWE}_{s,\xi}$ samples, distinguish elements following the distribution of $\mathbf{RLWE}_{s,\xi}$ from elements chosen uniformly over $\mathbb{T}_N[X]^n \times \mathbb{T}_N[X]$, for a fixed s drown from $\mathcal{S}$.

When $N = 1$ and n is large, the TRLWE decision and search problems revert to the LWE decision and search problems (respectively).

With an appropriate choice of parameters, both problems are *in practice* unsolvable (LWE and its variants have hardness reduction to lattice problems such as the Shortest Vector Problem or the Closest Vector Problem, as resumed in [Pei16]), and their difficulty increases with the entropy of the key set $\mathcal{S}$. As presented in the following sections, TFHE's ciphertexts borrow (R)LWE samples structures, which directly connect the scheme's security with the hardness of finding the key s.

2.3 TFHE's Parameters Set

The parameters needed to define the instantiation of the TLWE and TRLWE problems, and which thus determine the scheme's security, are as follows:

- $n \in \mathbb{N}$, the size of the TLWE secret key.
- $N \in \mathbb{N}$ such that N is a power of two that is the degree of the polynomial $X^N + 1$ in the TRLWE problem.
- $k \in \mathbb{N}$, the size of the TRLWE secret key.
- σ_{TLWE}, the standard deviation for the Gaussian distribution of the TLWE noise.

- σ_{TRLWE}, the standard deviation for the Gaussian distribution of the TRLWE noise.
- q the plaintext moduli.

Some more parameters are needed for bootstrapping and key switching:

- $B_g \in \mathbb{N}$, a decomposition basis.
- $l \in \mathbb{N}$, the number of digits in the decomposition used during bootstrapping (with basis B_g).
- $B_{KS} \in \mathbb{N}$, a decomposition basis.
- $t \in \mathbb{N}$, the number of digits in the decomposition used during the key switch (with basis B_{KS}).

2.4 Ciphertexts

TFHE defines three structures to encrypt plaintexts, which we introduce here.

TLWE sample. This is the "usual" TFHE ciphertext on which we can perform bootstrapping. It has the simplest structure and allows to encrypt any message m of the discrete space of messages $\mathcal{M} \subset \mathbb{T}$, which can be seen as $\mathbb{Z}_q$. The corresponding secret key s is uniformly sampled from $\mathbb{B}^n = \{0,1\}^n$.
To encrypt m, one must create a pair $(a,b) \in \mathbb{T}^{n+1}$, with a uniformly sampled from $\mathbb{T}^n$ and $b = \langle a,s\rangle + e$. The error $e \in \mathbb{T}$ is sampled from a Gaussian distribution with mean 0 and standard deviation σ_{TLWE}.
In this case, (a,b) is an encryption of 0 with minimal noise, known in the literature as a *fresh sample*. We should also point out another special type of TLWE ciphertext, which is commonly called *noiseless trivial* and refers to a noise-free ciphertext whose a part is 0. For instance, the corresponding *noiseless trivial* encryption of a cleartext message m is $(0,m)$.
To encrypt a message $m \in \mathcal{M}$ non-trivially, we need to calculate $c = (a,b) + (0,m)$, where (a,b) is a fresh TLWE sample.
The decryption procedure is as follows. First, one must compute the *phase* ϕ of the ciphertext they want to decrypt: $\phi(c) = b - \langle a,s\rangle = m + e$. Then, the result must be rounded up to the nearest valid value in $\mathcal{M}$. Of course, if $|e|$ is too large, the rounding will not return the correct value, and the decryption will not be correct. That is why $|e|$ must be smaller than half the distance between two elements of the plaintext space, that is to say $|e| < \frac{1}{2q}$.

TRLWE sample. The second kind of ciphertext is rarely used outside the bootstrapping procedure, even if it allows to encrypt messages in the discrete space of

messages $\mathcal{M} \subset \mathbb{T}_N[X]$ which can be seen as $\mathbb{Z}_q[X]/(X^N+1)$. The corresponding secret key s is uniformly sampled from $\mathbb{B}_N[X]^k$.
To encrypt $m \in \mathcal{M}$, one must create a pair $(a,b) \in \mathbb{T}_N[X]^{k+1}$, where a is uniformly sampled from $\mathbb{T}_N[X]^k$ and $b = \sum_{i=0}^{N-1} a_i \cdot s_i + e$. The error $e \in \mathbb{T}_N[X]$ is a polynomial with random coefficients sampled from a Gaussian distribution with mean 0 and standard deviation σ_{TRLWE}. In this case, (a,b) is an encryption of 0 with minimal noise. One usually chooses $k = 1$; therefore, a and b are simple polynomials of $\mathbb{T}_N[X]$.

Let $\mathcal{M} \subset \mathbb{T}_N[X]$ be a discrete space of messages. To encrypt a message $m \in \mathcal{M}$, we add $(0,m) \in \mathbb{T}_N[X]^k \times \mathbb{T}_N[X]$ to a TRLWE sample encryption of 0.
The decryption process is similar to the TLWE one. First, the phase of the ciphertext must be computed: $\phi(c) = b - \sum_{i=0}^{N-1} a_i \cdot s_i = m + e$. Then, we must round the phase to the nearest element of the plaintext space. Again, this procedure outputs the right result only if the infinite norm of the noise polynomial e are bounded by $\frac{1}{2q}$.

TRGSW sample. Informally, TRGSW samples can be seen as vectors of $l \in \mathbb{N}$ fresh TRLWE samples (or a matrix $l \times k$ when $k > 1$). More formally, TRGSW encryptions rely on a "gadget matrix" $H \in \mathcal{M}_{(k+1)\cdot l, k+1}(\mathbb{T})$ and are constructed as follows:
Let us define

$$V = \begin{pmatrix} \frac{1}{B_g} \\ \frac{1}{B_g^2} \\ \vdots \\ \frac{1}{B_g^l} \end{pmatrix} \quad \text{and} \quad O = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

two vectors of l components. With such notations, the gadget matrix is

$$H = \begin{pmatrix} V & O & \cdots & O \\ O & V & \cdots & O \\ \vdots & \vdots & \vdots & \vdots \\ O & O & \cdots & V \end{pmatrix}.$$

Let $\mathcal{M} \subset \mathbb{Z}_q[X]/(X^N+1)$ be a discrete space of messages and let $\mathbb{B}_N[X]$ be the corresponding secret key space. To encrypt a plaintext value $m \in \mathcal{M}$, one has to sample a vector Z of $(k+1)\cdot l$ fresh TRLWE samples (that is to say encryptions of 0), and to compute $C = Z + m \cdot H$. When $k = 1$, which is often the case, this simplifies to:

$$C = \begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \\ \vdots & \vdots \\ a_l & b_l \\ a_{l+1} & b_{l+1} \\ \vdots & \vdots \\ a_{2l} & b_{2l} \end{pmatrix} + m \cdot \begin{pmatrix} \frac{1}{B_g} & 0 \\ \frac{1}{B_g^2} & 0 \\ \vdots & \vdots \\ \frac{1}{B_g^l} & 0 \\ 0 & \frac{1}{B_g} \\ \vdots & \vdots \\ 0 & \frac{1}{B_g^l} \end{pmatrix}$$

where the (a_i, b_i) are fresh TRLWE samples, for all $i \in \{1, \cdots, 2l\}$. Note that if $m = 0$, the corresponding TRGSW ciphertext is simply the vector $Z = \begin{pmatrix} a_1 & b_1 \\ a_2 & b_2 \\ \vdots & \vdots \\ a_l & b_l \\ a_{l+1} & b_{l+1} \\ \vdots & \vdots \\ a_{2l} & b_{2l} \end{pmatrix}$.

A TRGSW ciphertext can be decrypted as a TRLWE ciphertext by using the last line of the matrix and multiplying the result by B_g^l.

> Note
>
> For simplicity's sake and for all that follows, we consider the usual case where $k = 1$.

Notations 3. In this thesis, we note $c \in \mathsf{T(R)LWE}_s(m)$ a T(R)LWE ciphertext of m with key s. We will interchangeably use these notations with the one proposed in the first chapter, where $[\![m]\!]_s$ refers to the set of valid encryptions of a message m under key s. When there is no ambiguity about the key, we simply note $c \in \mathsf{T(R)LWE}(m)$. We will not often use TRGSW encryptions, but when needed, we will denote them with uppercase letters: $C \in \mathsf{TRGSW}_s(m)$.

2.5 Basic Operations on Ciphertexts

As a fully homomorphic scheme, TFHE natively supports several operations, which we describe in this section. Note that TFHE's most common use is in the private key setting (which we use in this manuscript), but there also exist variants with public key [CGGI19, Reg05, Joy24].

2.5.1 Addition

The addition is the easiest native operation of TFHE. Let us begin with the addition of a cleartext value γ with a T(R)LWE ciphertext $c = (a, \langle a, s\rangle + m + e) \in [\![m]\!]$. The result of the homomorphic addition is $c' = (0, \gamma) + (a, \langle a, s\rangle + m + e) = (a, \langle a, s\rangle + \gamma + m + e)$. Note that if c is a valid encryption of m, then c' is a valid encryption of $m + \gamma$ as the calculation does not change the noise e.
Of course, this operation also works with TRGSW encryptions. Indeed, $C' = Z + m \cdot H + \gamma \cdot H = Z + (m + \gamma) \cdot H$ is a valid TRGSW encryption of $m + \gamma$.

Now, let us consider the case of the addition of two ciphertexts.
Given two T(R)LWE ciphertexts $c_1 = (a_1, \langle a_1, s\rangle + m_1 + e_1) \in [\![m_1]\!]$ and $c_2 = (a_2, \langle a_2, s\rangle +$

$m_2 + e_2) \in [\![m_2]\!]$, the result of their homomorphic addition is $c' = (a_1 + a_2, \langle a_1 + a_2, s\rangle + m_1 + m_2 + e_1 + e_2)$. In this case, $c' \in [\![m_1 + m_2]\!]$ only if $|e_1 + e_2| < \frac{1}{2q}$. Similarly, the addition of two TRGSW ciphertexts $C_1 = Z_1 + m_1 \cdot H$ and $C_2 = Z_2 + m_2 \cdot H$ encrypting m_1 and m_2 gives $C = (Z_1 + Z_2) + (m_1 + m_2) \cdot H$.

Note that we can only perform addition on ciphertexts of the same type (and with the same key). Regarding noise growth, in the cleartext-ciphertext context, the addition has no effect. However, in the ciphertext-ciphertext case, the addition leads to an additive growth of the noise.

2.5.2 Multiplication

Cleartext-ciphertext multiplication is also a straightforward native operation of TFHE. Given a cleartext value γ and a T(R)LWE ciphertext $c = (a, \langle a, s\rangle + m + e) \in [\![m]\!]$, the result of the multiplication is $c' = (\gamma a, \langle \gamma a, s\rangle + \gamma m + \gamma e)$.
Given a TRGSW ciphertext $C = Z + m \cdot H$, the constant polynomial multiplication with γ gives $C' = \gamma Z + \gamma m \cdot H$. As Z is a vector of TRLWE encryptions of 0, γZ is the multiplication of each TRLWE of Z with γ, resulting in new TRLWE encryptions of 0. Thus, C' is a TRGSW encryption of γm.
Note that, as opposed to addition, the noise growth is not linear but depends on γ. Thus, such an operation should not be used with a large constant; otherwise, false decryption could occur.
Indeed, if γ is large, by multiplying every component of the ciphertext $c \in \mathsf{T(R)LWE}(m)$ by γ, the noise level is also multiplied by γ, which can compromise the decryption. To address this noise issue, Chillotti et al. [CGGI19] propose a simple method that involves both decomposition and inner products. The idea is to decompose the large constant into a smaller basis β:

$$\gamma = \gamma_1 \frac{q}{\beta} + \gamma_2 \frac{q}{\beta^2} + \cdots + \gamma_l \frac{q}{\beta^l}.$$

The elements of the decomposition $\gamma_1, \cdots, \gamma_l$ are in $\mathbb{Z}_\beta$, and are thus small. We note $\mathsf{Decomp}^{\beta,l}(\gamma) = (\gamma_1, \cdots, \gamma_l)$. As the elements of the decomposition are small, the impact of the multiplication on the noise is small. However, to obtain the product of γ with the encryption of m, we need to be able to invert the decomposition and recompose the constant. To do that, instead of multiplying the decomposed elements with the T(R)LWE encryption of m, we create l T(R)LWE encryptions of m times the different powers of the decomposition basis:

$$\bar{c} = (c_1, c_2, \cdots, c_l) \in \mathsf{T(R)LWE}(m \cdot \frac{q}{\beta}) \times \mathsf{T(R)LWE}(m \cdot \frac{q}{\beta^2}) \times \cdots \times \mathsf{T(R)LWE}(m \cdot \frac{q}{\beta^l}).$$

In practice, the multiplication is then performed by computing:

$$c' = \sum_{j=1}^{l} \gamma_j \cdot c_i$$

which has a slower noise growth than the naive approach.

Ciphertext-ciphertext multiplication between two T(R)LWE ciphertexts does not exist with TFHE. Indeed, the structure of these ciphertexts does not allow multiplication. However, the structure of TRGSW ciphertexts and the same idea of decomposition allow the multiplication of TRGSW ciphertexts with TRLWE ones (and thus with TLWE, as the conversion from a TLWE sample to a TRLWE one is trivial). Given a TRGSW ciphertext $C = Z + m_1 \cdot H$ and a TRLWE ciphertext $c = (a, b) \in [\![m_2]\!]$, let us compute $C \times c$. Let us remind that when $k = 1$, the a part of the TRLWE is a polynomial in $\mathbb{T}_N[X]$. Thus, the decomposition tool decomposes the polynomial a in the following way:

$$a = \sum_{j=1}^{l} a_j \cdot B_g^{-j} + \epsilon_a.$$

Similarly, the polynomial b is decomposed in the following way:

$$b = \sum_{j=1}^{l} b_j \cdot B_g^{-j} + \epsilon_b$$

where each a_j and b_j are polynomials with coefficients bounded by $\frac{B_g}{2}$ and ϵ_a and ϵ_b are polynomials with coefficients bounded by $\frac{1}{2B_g^l}$. Moreover, we note $\tilde{a} = a - \epsilon_a$ and $\tilde{b} = b - \epsilon_b$.
The decomposition of the ciphertext c is thus:

$$\mathsf{Decomp}^{B_g,l}(c) = (a_1, a_2, \cdots, a_l, b_1, b_2, \cdots, b_l).$$

We note Z_i the ith row of Z, that is to say the TRLWE encryption of 0 corresponding to the ith coefficient of Z.
Then, we can compute the multiplication of C and c:

$$\begin{aligned}
\mathsf{Decomp}^{B_g,l}(c) \cdot C &= (a_1, \cdots, b_l) \cdot (Z + m_1 \cdot H) \\
&= (a_1, \cdots, b_l) \cdot Z + (a_1, \cdots, b_l) \cdot m_1 \cdot H \\
&= a_1 \cdot Z_1 + a_2 \cdot Z_2 + \cdots + a_l \cdot Z_l + b_1 \cdot Z_{l+1} + b_2 \cdot Z_{l+2} + \cdots + b_l \cdot Z_{2l} \\
&\quad + m_1 \cdot (a_1, \cdots, b_l) \cdot H \\
&= \sum_{j=1}^{l} a_j \cdot Z_j + \sum_{j=1}^{l} b_j \cdot Z_{l+j} + m_1 \cdot (a_1 \cdot \frac{1}{B_g} + \cdots + a_l \cdot \frac{1}{B_g^l}, b_1 \cdot \frac{1}{B_g} + \cdots + b_l \cdot \frac{1}{B_g^l}) \\
&= S + m_1 \cdot (\sum_{j=1}^{l} a_j \cdot B_g^{-j}, \sum_{j=1}^{l} b_j \cdot B_g^{-j}) \\
&= S + m_1 \cdot (\tilde{a}, \tilde{b}) \\
&= S + m_1 \cdot (a, b) + m_1 \cdot (\epsilon_a, \epsilon_b)
\end{aligned}$$

Note that as a linear combination of TRLWE encryptions of 0, $S = \sum_{j=1}^{l} a_j \cdot Z_j + \sum_{j=1}^{l} b_j \cdot Z_{l+j}$ is a TRLWE encryption of 0. Also, $m_1 \cdot (\epsilon_a, \epsilon_b)$ can be seen as an error term, and $m_1 \cdot (a, b)$ is an encryption of $m_1 \cdot m_2$. As such, $\mathsf{Decomp}(c) \cdot C$ gives an encryption of $m_1 \cdot m_2$. This operation is called the external product and from now on it will be denoted $\boxdot$.

On the noise of the external product. For a given $\mathsf{T(R)LWE}$ ciphertext $c = (a, b)$, with $b = \langle a, s \rangle + m + e$, we note $\mathsf{noise}(c) = e$ its corresponding noise. The resulting noise of the above multiplication of a TRGSW ciphertext C with a TRLWE ciphertext c is given by:

$$\mathsf{noise}(S) + m_1 \cdot \mathsf{noise}(c) + m_1 \cdot \epsilon_b.$$

With $e_i = \mathsf{noise}(Z_i)$, we have $\mathsf{noise}(S) = \sum_{j=1}^{l} a_j \cdot e_j + \sum_{j=1}^{l} b_j \cdot e_{l+j}$. Each e_i follows a Gaussian distribution with variance at most V_{TRGSW}, and e follows a Gaussian distribution with variance at most V_{TRLWE}. Finally, ϵ_b follows a uniform distribution in $\left[-\frac{1}{2B_g^l}, \frac{1}{2B_g^l}\right]$ with standard deviation $\frac{1}{2\sqrt{3B_g^l}}$. Thus, the output of $C \boxdot c$ follows a Gaussian distribution with variance at most:

$$2 \cdot l \cdot N \cdot \left(\frac{B_g}{2}\right)^2 \cdot V_{\mathsf{TRGSW}} + \|m_1\|_2^2 \cdot \left(\frac{1+N}{12B_g^{2l}}\right) + \|m_1\|_2^2 \cdot V_{\mathsf{TRLWE}}.$$

As noted in [Cle24], this formula improves on the one proposed in the TFHE paper [CGGI19] where a variance of $\frac{1}{4B_g^{2l}}$ is considered, instead of $\frac{1}{12B_g^{2l}}$ for the ϵ_a and ϵ_b terms.

2.5.3 CMux

The CMux (Controlled MUltipleXer) gate is an important building block of TFHE. Given a TRGSW encryption of a bit B, it leverages the external product to perform a selection between two TRLWE ciphertexts (encrypted with the same key). Given a ciphertext $C \in \mathsf{TRGSW}(B)$ and two TRLWE ciphertexts $c_1 \in [\![m_1]\!]$ and $c_2 \in [\![m_2]\!]$, the CMux gate satisfies:

$$\mathrm{CMux}(C, c_1, c_2) = \begin{cases} \mathsf{TRLWE}(m_1) & \text{if } B = 1 \\ \mathsf{TRLWE}(m_2) & \text{otherwise.} \end{cases}$$

It is computed using the external product $\boxdot$:

$$\mathrm{CMux}(C, c_1, c_2) = C \boxdot (c_1 - c_2) + c_2.$$

It is easy to see that the variance of the noise of the result of a CMux gate will be very similar to that of a TRLWE resulting from an external product:

$$\mathrm{Var}(\mathrm{CMux}(C, c_1, c_2)) \leq 2 \cdot l \cdot N \cdot \left(\frac{B_g}{2}\right)^2 \cdot V_{\mathsf{TRGSW}} + \left(\frac{1+N}{12B_g^{2l}}\right) + V_{\mathsf{TRLWE}}.$$

2.6 Bootstrapping

2.6.1 Building Blocks

Bootstrapping, the operation that reduces the noise of a TLWE ciphertext, requires three building blocks that we present here. Figure 2.1 illustrates the bootstrapping procedure.

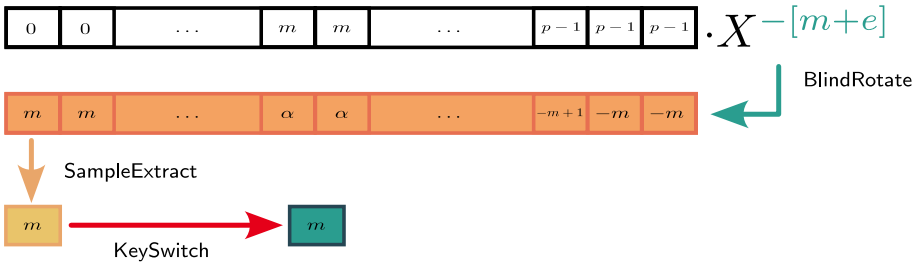


Figure 2.1: Illustration of TFHE's bootstrapping.

BlindRotate. The BlindRotate operation rotates a plaintext polynomial $testv$[1] by a TLWE encrypted index $c \in [\![m]\!]$. In fact, the cleartext polynomial $testv$ is a *noiseless trivial* TRLWE encryption $(0, testv)$. BlindRotate returns a TRLWE encryption of $testv \cdot X^{-\phi(c)} \mod (X^N + 1)$, where $\phi(c)$ is the phase of c rescaled in $\mathbb{Z}_{2N}$. The input parameters for this operation are thus as follows: a noiseless trivial polynomial ciphertext $testv$, an encrypted TLWE index $c = (a_1, \cdots, a_n, b)$ with coefficients rescaled in $[\![0, 2N-1]\!]$ and n TRGSW encryptions of $(s_1, \cdots, s_n)$ with $\forall i, s_i \in \mathbb{B}$ which is called the *bootstrapping key*.
The result is a ciphertext $c' \in \mathsf{TRLWE}(testv \cdot X^{\langle a,s\rangle - b})$. We often denote this ciphertext ACC as it acts as an accumulator.

Algorithm 1 BlindRotate

Require: a TLWE ciphertext $c = (a_1, \cdots, a_n, b) \in \mathsf{TLWE}_s(m)$, a TRLWE ciphertext $testv$, and a bootstrapping key BK such that $BK_i \in \mathsf{TRGSW}_k(s_i)$ for all $i \in \{1, \cdots, n\}$.
Ensure: a TRLWE sample $ACC \in \mathsf{TRLWE}(testv \cdot X^{\phi(c)})$.

```
ACC = X^{-b} · testv
for i ∈ {1, ···, n} do
    ACC = CMux(BK_i, X^{a_i} · ACC, ACC)
return ACC
```

Algorithm 1 describes the BlindRotate procedure, which relies heavily on CMux gates. At index u in the second **for** loop, ACC is set to $testv \cdot X^{-\sum_{i=1}^{u}(\bar{a_i} \cdot s_i) - b}$. Since

[1] We sometimes refer to this polynomial as the test polynomial or vector.

multiplying a TRLWE ciphertext by X^α does not increase the noise of the ciphertext, the noise of the output from the BlindRotate procedure depends primarily on the noise generated by the computation of the CMux gate. Consequently, the variance of the noise in the output is bounded by:

$$\mathrm{Var}(ACC) \leq n \cdot \left(2 \cdot l \cdot N \cdot \left(\frac{B_g}{2}\right)^2 \cdot V_{\mathsf{TRGSW}} + \left(\frac{2 \cdot N}{12 B_g^{2l}}\right)\right) + V_{\mathsf{TRLWE}}.$$

SampleExtract. The second operation is the TLWE Sample Extract, which, as its name suggests, extracts one of the coefficients of a TRLWE ciphertext and interprets it as a TLWE ciphertext. This operation takes as input a ciphertext $c \in \mathsf{TRLWE}_k(m)$ and a position $p \in \{0, \cdots, N-1\}$ and returns a ciphertext $c' \in \mathsf{TLWE}_k(m_p)$ with m_p the pth coefficient of the polynomial m. When no index is specified, it returns the ciphertext in position 0.
Note that this operation does not change the noise of the ciphertext.

Public Functional Keyswitch. Finally, the last operation is Public Functional Keyswitching which is the one that allows to convert a set of p ciphertexts $c_i \in \mathsf{TLWE}_k(m_i)$ into a ciphertext $c' \in \mathsf{T(R)LWE}_s(f(m_1, \cdots, m_p))$, where f is a public linear morphism from $\mathbb{T}^p$ into $\mathbb{T}_N[X]$. This operation not only allows the encryption keys to be changed, but also allows the type of ciphertext to be changed if desired. When f is the identity function, which is, for instance, the case for the bootstrapping procedure, we simply denote this operation KeySwitch. Note that if the morphism f is not public, then we perform a private version of the keyswitch called Private KeySwitch.
Finally, to perform the KeySwitch, a specific key, called the *keyswitch key*, is needed. Given the input TLWE encryption key s, this keyswitch key KS is composed of the T(R)LWE encryptions of $\frac{s_i}{B_{KS}^j}$ for $i \in \{1, \cdots, n\}$ and $j \in \{1, \cdots, t\}$ under key k.

The noise of the output $c \in \mathsf{T(R)LWE}_k(f(m_1, \cdots, m_p))$ is the sum of the noise of each component of the ciphertext:

$$f(e_1, \cdots, e_p) + \sum_{i=1}^{n} \tilde{e_i} \cdot s_i - \sum_{i=1}^{n} \sum_{j=1}^{t} \tilde{a_{i,j}} \cdot e_{i,j}.$$

This gives us the following bound:

$$\mathrm{Var}(c) \leq \mathrm{Var}(c^{(1)}, \cdots, c^{(p)}) + \frac{n}{12 B_{KS}^{2t}} + n \cdot N \cdot t \cdot \left(\frac{B_{KS}}{2}\right)^2 \cdot \mathrm{Var}_{KS}$$

with Var_{KS} the maximum variance of the ciphertexts composing the keyswitch key KS.

Algorithm 2 KeySwitch

Require: p TLWE ciphertext $c^{(i)} = (a_1^{(i)}, \cdots, a_n^{(i)}, b^{(i)}) \in \mathsf{TLWE}_s(m_i)$, a public morphism $f : \mathbb{T}^p \to \mathbb{T}_N[X]$, a keyswitch key KS such that $KS_i \in \mathsf{T(R)LWE}_k(\frac{s_i}{B_{KS}^j})$ for $i \in \{1, \cdots, n\}$ and $j \in \{1, \cdots, t\}$

Ensure: a T(R)LWE ciphertext $c \in [\![f(m_1, \cdots, m_p)]\!]_k$.

1: **for** $i \in \{1, \cdots, n\}$ **do**
2: $\quad a_i = f(a_i^{(1)}, \cdots, a_i^{(p)})$
3: $\quad$ Let $\tilde{a_i}$ be the closest multiple of B_{KS}^{-t} to a_i
4: $\quad$ Let $\sum_{j=1}^{t} \frac{\tilde{a_{i,j}}}{B_{KS}^j}$ be the closest decomposition of $\tilde{a_i}$ in basis B_{KS} with $\tilde{a_{i,j}} \in \{-\frac{B_{KS}}{2}, \frac{B_{KS}}{2} - 1\}$

return $(0, f(b^{(1)}, \cdots, b^{(p)}) - \sum_{i=1}^{n} \sum_{j=1}^{t} \tilde{a_{i,j}} \cdot KS_{i,j})$

TFHE offers two different bootstrapping methods. The first is called *gate bootstrapping*, and as just described, its purpose is to reduce the noise level of a TLWE ciphertext. The second type of bootstrapping is called *circuit bootstrapping* and is used to convert TLWE ciphertexts into TRGSW ciphertexts. It is mainly used for a leveled use of TFHE.

2.6.2 Gate Bootstrapping

TFHE specifies a gate bootstrapping to reduce the noise level of a TLWE sample that encrypts the result of a boolean gate evaluation on two ciphertexts, each of them encrypting a binary input. TFHE gate bootstrapping steps are summarized in Algorithm 3. Step 1 consists of selecting a value $\hat{m} \in \mathbb{T}$ which will serve later for setting the coefficients of the test polynomial *testv* (in step 3). Step 2 rescales the components of the input ciphertext c as elements of $\mathbb{Z}_{2N}$. Step 3 defines the test polynomial *testv*. Note that for all $p \in [\![0, 2N[\![$, the constant term of $testv \cdot X^p$ is $\hat{m}$ if $p \in]\!]\frac{N}{2}, \frac{3N}{2}]\!]$ and $-\hat{m}$ otherwise. Step 4 returns an accumulator $ACC \in [\![testv \cdot X^{\langle \bar{a}, s \rangle - \bar{b}}]\!]_{s'}$. Indeed, the constant term of ACC is $-\hat{m}$ if $c \in [\![0]\!]_s$, or $\hat{m}$ if $c \in [\![1]\!]_s$. Then, step 5 creates a new ciphertext $\bar{c}$ by extracting the constant term of ACC and adding to it $(0, \hat{m})$. That is, $\bar{c}$ either encrypts 0 if $c \in [\![0]\!]_s$, or m if $c \in [\![1]\!]_s$ (By choosing $m = \frac{1}{2}$, we get a fresh encryption of 1).

2.6.3 Circuit Bootstrapping

TFHE specifies a second type of bootstrapping known as circuit bootstrapping. This process converts TLWE samples into TRGSW samples and is primarily used for TFHE in a leveled manner. Since this additional type of bootstrapping falls outside the scope of this thesis, we will not provide extensive details about it but give an idea of the procedure.

Algorithm 3 TFHE gate bootstrapping

Require: a constant $m \in \mathbb{T}$, a TLWE sample $c = (a, b) \in [\![x \cdot \frac{1}{2}]\!]_s$ with $x \in \mathbb{B}$, a bootstrapping key $BK_{s \to s'} = (BK_i \in [\![s_i]\!]_{S'})_{i \in [\![1,n]\!]}$ where BK_i is a TRGSW sample of s_i with the key S'; the TRLWE interpretation of a secret key s',

Ensure: a TLWE sample $\bar{c} \in [\![x.m]\!]_s$

1: Let $\hat{m} = \frac{1}{2}m \in \mathbb{T}$ (pick one of the two possible values)
2: Let $\bar{b} = \lfloor 2Nb \rceil$ and $\bar{a}_i = \lfloor 2Na_i \rceil \in \mathbb{Z}, \forall i \in [\![1, n]\!]$
3: Let $testv := (1 + X + \cdots + X^{N-1}) \cdot X^{\frac{N}{2}} \cdot \hat{m} \in \mathbb{T}_N[X]$
4: $ACC \leftarrow \mathsf{BlindRotate}((0, testv), (\bar{a}_1, \cdots, \bar{a}_n, \bar{b}), (BK_1, \cdots, BK_n))$
5: $\bar{c} = (0, \hat{m}) + \mathsf{SampleExtract}(ACC)$
6: return $\mathsf{Keyswitch}_{s' \to s}(\bar{c})$

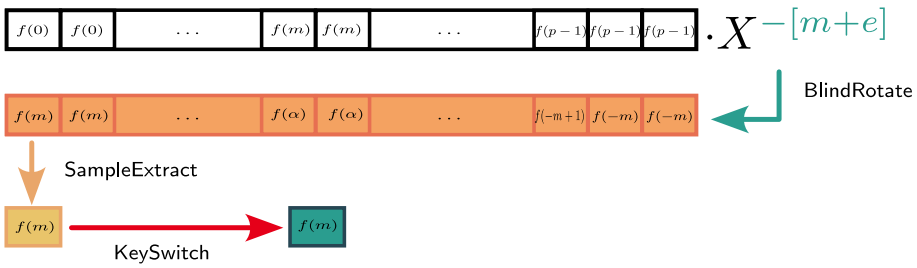


Figure 2.2: Programmable bootstrapping. From an encryption of m, we obtain an encryption of $f(m)$ with nominal noise level.

Given a TLWE encryption of the message m, the circuit bootstrapping is composed of a series of l gate bootstrapping steps to create l ciphertexts $c_i \in \mathsf{TLWE}(m \cdot \frac{q}{B_g^{i+1}})$ for $i \in \{0, \cdots, l-1\}$ under a TRLWE key s'. Then, we perform $k+1$ private key switches on each c_i to obtain TRLWE encryptions $c'_{i,j} \in \mathsf{TRLWE}_{s'}(-m \cdot \frac{q}{B_g^{i+1}} \cdot s'_j)$. The collection of those ciphertexts is a TRGSW ciphertext encrypting m.

2.7 Programmable Bootstrapping

Building on TFHE's Gate Bootstrapping, Bourse et al. [BMMP18] and Chilotti et al. [CJP21] showed that any function can be evaluated on encrypted inputs without any additional cost. In this section, we take a look at several state-of-the-art methods that allow us to homomorphically perform the evaluation of any function over encrypted inputs.

2.7.1 A Vanilla Approach

LookUp Table. A LookUp Table (LUT) is a list or table of values associated with a computation (for instance, tables for trigonometric value). It is a data structure

stored in memory, used to replace a calculation with a less expensive look-up operation. Indeed, the speed gain can be significant when searching for a value in memory instead of performing an expensive calculation.

LUTs are used in homomorphic encryption to simulate the calculation of complicated functions. The function to be applied to the ciphertexts is pre-computed in the cleartext space to create the function's LUT. This plaintext calculation, which then requires storing the results, is an excellent time-memory compromise for homomorphic calculation (for small plaintext domains). The stored cleartext values are negligible compared to the size of the encrypted messages, and the calculation of the function on cleartexts is almost instantaneous. During the instantiation phase of the bootstrapping, the values of the LUT to be evaluated on encrypted data are given to the coefficients of the *testv* cleartext polynomial to be rotated. If a function f is to be homomorphically applied, the server pre-computes its corresponding LUT and constructs *testv* accordingly. When applying bootstrapping on this polynomial with selector c encrypting m, the algorithm will return the m-th coefficient encrypting $f(m)$.
We refer to this bootstrapping as *programmable* or *functional*. We note that the original bootstrapping (in [CGGI19]) is a particular case of programmable bootstrapping with f set to the identity function.

TFHE bootstrapping is natively well-suited but limited to implementing LUTs of negacyclic functions[2] for two reasons. First, TFHE plaintext space is $\mathbb{T}$, where $[0, \frac{1}{2})$ corresponds to positive values and $[\frac{1}{2}, 1)$ to negative ones. So, if c is a TLWE encryption of a positive value, its phase $\phi(c)$ lies in $[0, \frac{1}{2})$, and it satisfies $\phi(c) \in [0, N)$ after rescaling to $\mathbb{Z}_{2N}$. Conversely, if c is a TLWE encryption of a negative value, its phase satisfies $\phi(c) \in [N, 2N)$ after rescaling to $\mathbb{Z}_{2N}$. Second, BlindRotate outputs an encryption of *testv* multiplied by $X^{-\phi(c)} \mod (X^N + 1)$[3]. So, if *testv* coefficients are set to the evaluation of a negacyclic function on the positive values of $\mathcal{M}$ (values in $\mathcal{M} \cap [0, \frac{1}{2})$), a bootstrapping with an input TLWE ciphertext c encrypting m returns either $f(m)$ if $m \in \mathcal{M} \cap [0, \frac{1}{2})$, or $-f(m - \frac{1}{2})$ if $m \in \mathcal{M} \cap [\frac{1}{2}, 1)$.

2.7.2 On the Evaluation of Non-Negacyclic Functions

Many works tackled the restriction of TFHE bootstrapping to the evaluation of LUTs of negacyclic functions. For instance, the half-torus method works around the negacyclic restriction by encoding all the plaintext space $\mathcal{M}$ on $[0, \frac{1}{2})$ (i.e., on the positive half of the torus). As no plaintext values are encoded on the negative half of the torus, any LUT can be encoded within the coefficients of the test polynomial. Then, it is evaluated with only one bootstrapping. But the cost of this technique is

[2]Negacyclic functions are antiperiodic functions over $\mathbb{T}$ with period $\frac{1}{2}$, satisfying $f(x) = -f(x + \frac{1}{2})$.

[3]We remind that $\forall \alpha \in [0, N), X^{\alpha+N} = -X^{\alpha} \mod (X^N + 1)$.

to work with larger parameters. Other methods, such as TOTA [YXS+21], FDFB [KS22], or ComBo [CBSZ23], specify several solutions to work around the restriction of working only with half of the torus. They provide different ways for implementing any LUT using the full torus as plaintext space at the cost of making at least two consecutive BlindRotate. All these methods allow smaller parameters compared to the half-torus and faster execution in certain cases. However, as stated in [CBSZ23], which proposes a comparison of all these methods, these gains come at the cost of several bootstrappings for a single evaluation.

Indeed, for TOTA and FDFB, the evaluation of one functional bootstrapping on a single input requires two bootstrappings. In the case of Combo, four bootstrappings are needed. Even if these bootstrappings can be parallelized, the method still requires two bootstrappings to be performed sequentially. These methods, therefore, enable precise bootstrapping over the entire torus but at the cost of being twice as time-consuming as the classical partial domain method.

As stated above, Clet et al. [CBSZ23] compared all of these methods for the same TFHE parameters and levels of security and showed that the half-torus method achieves the best speed-to-error-rate ratio. They concluded that the half-torus method is the fastest as it requires only one bootstrapping. This is why we only used this technique in the contribution of this thesis.

2.8 Tree-Based Method

Almost all of the functional bootstrapping methods from the state of the art ([CJP21, KS22, YXS+21, CLOT21, CBSZ23]) take as input a single ciphertext of a message in a relatively small set. In 2021, Guimarães *et al.* [GBA] specified the *Tree-Based* Method (TBM) for performing functional bootstrapping over several ciphertexts. A natural idea to perform computations on higher-precision messages is then to use this method over input messages decomposed into a smaller basis B.

Thus, the encryption of the initial plaintext value is a vector of encryptions of its decomposition digits in basis B. Let $B, B', d \in \mathbb{N}^*$ and m be a large integer message. B and B' are the bases on which to decompose the message. We then have $m = \sum_{i=0}^{d-1} m_i \cdot B^i$, with $m_i \in [\![0, B-1]\!]$. From this decomposition, we obtain d TLWE encryptions $(c_0, c_1, \cdots, c_{d-1})$ of $(m_0, m_1, \cdots, m_{d-1})$.We denote $f : [\![0, B-1]\!]^d \to [\![0, B'-1]\!]$ the target function and define g as the following bijection:

$$\begin{array}{rccc} g & : & [\![0, B-1]\!]^d & \to & [\![0, B^d-1]\!] \\ & & (a_0, a_1, \cdots, a_{d-1}) & \mapsto & \sum_{i=0}^{d-1} a_i \cdot B^i \end{array}$$

We then encode a LUT for f under B^{d-1} noiseless trivial TRLWE ciphertexts. Each

of these ciphertexts encodes a polynomial P_i such that:

$$P_i(X) = \sum_{j=0}^{B-1} \sum_{k=0}^{\frac{N}{B}-1} f \circ g^{-1}(j \cdot B^{d-1} + i) \cdot X^{j \cdot \frac{N}{B} + k}$$

Then, we apply the BlindRotateAndExtract (the BlindRotate directly followed by the SampleExtract in position 0) to each test polynomial $testv = \mathsf{TRLWE}(P_i)$ with c_{d-1} as a selector. We obtain B^{d-1} TLWE ciphertexts, each corresponding to the encryption of $f \circ g^{-1}(m_{d-1} \cdot B^{d-1} + i)$, for $i \in [\![0, B^{d-1} - 1]\!]$.

Finally, we use the TLWE-to-TRLWE KeySwitch operation to gather them into B^{d-2} encrypted TRLWE, corresponding to the LUT of h, with:

$$\begin{array}{rccc} h & : & [\![0, B-1]\!]^{d-1} & \to & [\![0, B'-1]\!] \\ & & (a_0, a_1, \cdots, a_{d-2}) & \mapsto & f(a_0, a_1, \cdots, a_{d-2}, m_{d-1}). \end{array}$$

Note that each new encrypted TRLWE has B different coefficients that are repeated $\frac{N}{B}$ times. We then perform BlindRotateAndExtract on each of these polynomials with selector c_{d-2}, which gives us B^{d-2} new TLWE samples that we pack together to create B^{d-3} new TRLWE encryptions, and so on. We repeat this operation, using the ciphertext c_{d-i} at step i, until we obtain a single TLWE ciphertext of $f(m_0, m_1, \cdots, m_{d-1})$. At the end of the procedure, we obtain one digit in basis B'. So the TBM must be run independently as many times as the number of digits in the output message's decomposition. More generally, the number N_{boot} of bootstrapping needed with the TBM increases with the number of input digits d and the number of output digits d' as well as the basis B chosen for the decomposition. In fact, for a complete evaluation of a LUT via the TBM, we have $N_{boot} = d' \times \sum_{i=0}^{d-1} B^i$. The number of required BlindRotate for the same operation is thus $\mathsf{N_{BR}} = d' \times \sum_{i=0}^{d-1} B^i$.

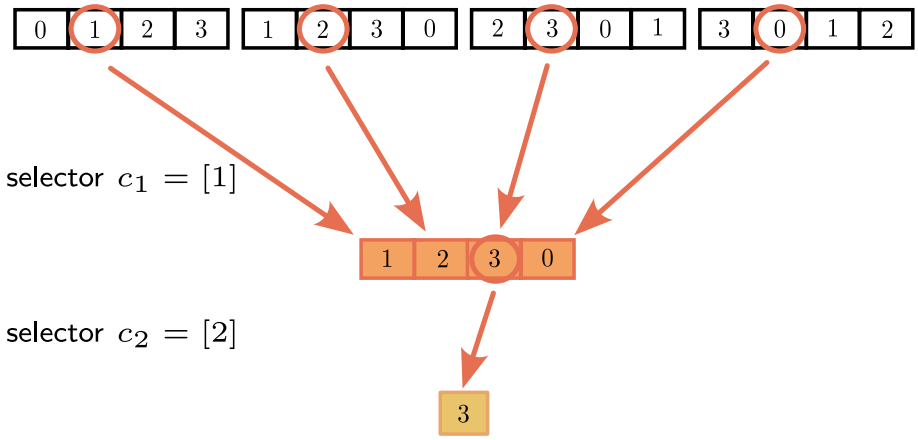


Figure 2.3: Example of the use of the Tree-Based Method to compute the homomorphic addition of ciphertexts in $\mathbb{Z}_4$. Arrows indicate bootstrapping.

Of course, this method is also used for the evaluation of multivariate functions over encrypted inputs. For example, Figure 2.3 illustrates the TBM for the addition in $\mathbb{Z}_4$. To use the TBM, we first compute the LUT that will be used to instantiate the test polynomials that will be rotated during the BlindRotate step. In this example, we want to compute the addition over $\mathbb{Z}_4$, so the LUT will be such that for $i, j \in \mathbb{Z}_4$, $\mathsf{lut}[4i + j] = i + j \pmod 4$. Then, we can instantiate the four polynomials P_i so that the coefficients of P_0 correspond to $i = 0$, that is to say, P_0 encodes the $j + 0 \pmod 4$ function, and so on for each test vector. Each coefficient is actually repeated consecutively $\frac{N}{4}$ times to fill the polynomials. Then, we perform four BlindRotate, one on each cleartext polynomial with the first input $c_1 \in [\![1]\!]$, followed by four SampleExtract. We get four ciphertexts that we combine together with a TLWE-to-TRLWE Keyswitch to create a TRLWE encryption of a new test polynomial. Then, we apply a BlindRotate to this encrypted test polynomial with the second encrypted input $c_2 \in [\![2]\!]$, and apply a SampleExtract followed by a Keyswitch to get the final result. From now on, we will refer by $\mathsf{N_{KS}}$ the number of calls to a TLWE-to-TRLWE Keyswitch required to evaluate a function on encrypted data. Note that for a complete evaluation of a LUT with the TBM, we have $\mathsf{N_{KS}} = d' \times \sum_{i=0}^{d-1} i$. For the considered example, the TBM requires five BlindRotate ($\mathsf{N_{BR}} = 5$) and one TLWE-to-TRLWE KeySwitch ($\mathsf{N_{KS}} = 1$).

2.9 Multi-Value Bootstrapping

Multi-Value Bootstrapping (MVB) [CIM18] refers to the method for evaluating k different LUTs on a single input with a single BlindRotate. MVB factors the test polynomial P_{f_i} associated with the function f_i into a product of two polynomials v_0 and v_i, where v_0 is a common factor to all P_{f_i}. In practice, we have:

$$(1 + X + \cdots + X^{N-1}) \cdot (1 - X) \equiv 2 \bmod (X^N + 1)$$

By writing P_{f_i} in the form $P_{f_i} = \sum_{j=0}^{N-1} \alpha_{i,j} X^j$ with $\alpha_{i,j} \in \mathbb{Z}$, we get from the previous equation:

$$\begin{aligned} P_{f_i} &= \frac{1}{2} \cdot (1 + X + \cdots + X^{N-1}) \cdot (1 - X) \cdot P_{f_i} \bmod (X^N + 1) \\ &= v_0 \cdot v_i \bmod (X^N + 1) \end{aligned}$$

where:

$$\begin{aligned} v_0 &= \frac{1}{2} \cdot (1 + X + \cdots + X^{N-1}) \\ v_i &= \alpha_{i,0} + \alpha_{i,N-1} + (\alpha_{i,1} - \alpha_{i,0}) \cdot X + \cdots + (\alpha_{i,N-1} - \alpha_{i,N-2}) \cdot X^{N-1} \end{aligned}$$

This factorization allows computing many LUTs using a unique bootstrapping. Indeed, it is enough to initialize the test polynomial *testv* with the value of v_0 during

bootstrapping. Then, after the BlindRotate operation, one has to multiply the obtained ACC by each v_i corresponding to the LUT of f_i to get ACC_i. Figure 2.4 illustrates the advantage of this method. This optimization reduces the number of bootstrappings required for an operation and, thus, the overall computation time. This method is summarized in Algorithm 4.

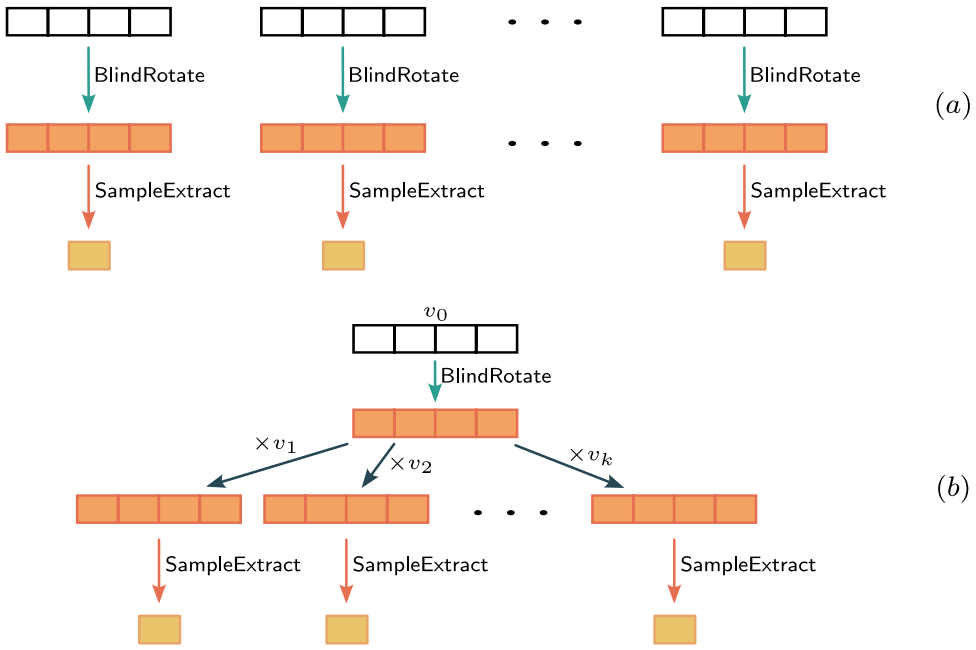


Figure 2.4: Illustration of the MVB optimization. (a) represents the classic method to process several bootstrapping, while (b) represents the MVB optimization. As seen here, it reduces the number of BlindRotate operations, which is the most expensive one of the bootstrapping.

As already noted in [GBA], the MVB can be applied to the first level of a TBM, as several BlindRotate are performed on different polynomials with the same encrypted input. For instance, regarding Figure 2.3, instead of requiring five BlindRotate and one TLWE-to-TRLWE Keyswitch (N_{BR}= 5 and N_{KS}= 1), the TBM evaluation of the addition in $\mathbb{Z}_4$ with MVB will only cost two BlindRotate, one TLWE-to-TRLWE Keyswitch, and four plaintext/ciphertext multiplications.

2.10 Other Recent Works

In this section, we discuss recent improvements to TFHE that have emerged throughout the thesis. While these works are often out of the scope of this thesis, it is important to mention them for the sake of this manuscript's completeness.

Algorithm 4 Multi-Value bootstrapping

Require: a TLWE ciphertext $c = (a, b) \in \mathsf{TLWE}_s(m)$ with $m \in \mathbb{T}$, a bootstrapping key $BK_{s\to s'} = (BK_i \in \mathsf{TRGSW}_{S'}(s_i))_{i\in[\![1,n]\!]}$ such that S' is the TRLWE interpretation of the secret key s' and k LUTs such that $\mathrm{LUT}(f_i) = v_0 \cdot v_i$, $\forall i \in [\![1, k]\!]$. $\overline{b} = \lfloor 2Nb \rceil$ and $\overline{a}_i = \lfloor 2Na_i \rceil \in \mathbb{Z} \quad \forall i \in [\![1, n]\!]$

Ensure: a list of k ciphertexts $\overline{c}_i \in \mathrm{TLWE}_s(f_i(\frac{\phi(\overline{a},\overline{b})}{2N}))$

$testv := v_0$
$ACC \longleftarrow \mathsf{BlindRotate}((0, testv), (\overline{a}_1, \cdots, \overline{a}_n, \overline{b}), (BK_1, \cdots, BK_n))$
$i \leftarrow 1$
while $i \leq k$ **do**
    $ACC_i := ACC \cdot v_i$
    $\overline{c}_i = \mathsf{SampleExtract}(ACC_i)$
    Return $\mathsf{KeySwitch}_{s'\to s}(\overline{c}_i)$
    $i \leftarrow i + 1$

2.10.1 WoP-PBS

The TFHE scheme natively encrypts only up to 11 bits[4] of message per ciphertext for a reasonable computational cost. It is possible to divide a message into several chunks and encrypt each into a ciphertext. However, the classical PBS cannot evaluate a lookup table with multiple ciphertexts as input. However, as seen in Section 2.8, we can use the TBM. Nevertheless, even with this method, the rapid cost growth (the greater the number of chunks, the greater the number of bootstrappings to be performed) does not allow for efficient computations over large precision messages. To overcome this issue, [BBB+23b] proposes a new Without Bit of Padding PBS, which inputs several ciphertexts and permits the evaluation of any multivariate lookup table. The idea behind this technique is the following: First, extract each bit of each chunk of the message into single TLWE ciphertexts by using bootstrapping. Then, apply circuit bootstrapping to turn the TLWE ciphertexts into TRGSW ciphertexts. Finally, use *vertical packing* [CGGI19] to evaluate the desired LUTs on the new ciphertexts. The authors of the WoP-PBS compared this technique with state-of-the-art, particularly with the TBM of [GBA]. They show that for 8-bit messages decomposed into two chunks of 4 bits, also called *nibbles*, the TBM is more efficient than the WoP-PBS. However, for any higher precision messages, the WoP-PBS gives better efficiency.
In the contributions of this thesis, we work with 8-bit messages decomposed into two nibbles and thus chose to work with the TBM.

[4] `https://www.zama.ai/post/parameter-optimization-and-larger-precision-for-tfhe.`

2.10.2 Faster Circuit Bootstrapping

Recall that the circuit bootstrapping can be decomposed into two steps:

1. Functional Bootstrapping: The functional bootstrapping takes as input TLWE encryptions of messages m_i and returns TLWE encryptions of $m_i \cdot \frac{1}{B_g^j}$ for $j \in \{1, \cdots, l\}$.
2. TLWE-to-TRLWE Ciphertext Conversion: Convert TLWE($m_i \cdot \frac{1}{B_g^j}$) to TRLWE($m_i \cdot \frac{1}{B_g^j}$) using Public Keyswitch and TRLWE($-m \cdot s \cdot \frac{1}{B_g^j}$) using Private KeySwitch, respectively.

In [WLW+24], the authors propose a "new" circuit bootstrapping with two improvements on the classical Circuit Bootstrapping:

- They first improve the first step of Circuit Bootstrapping by leveraging on the PBSManyLUT method of [CGGI19], which factorizes some bootstrapping.
- Then, they improve the second step of Circuit Bootstrapping by transforming each call to Private Keyswitch into a call to Public Keyswitch and a new function they call EvalSquareMult.

Using this new Circuit Bootstrapping, the authors proposed an AES evaluation with TFHE in leveled mode that was the fastest at the time (≈ 46 seconds in sequential execution), proving the importance of the improvements they made.

2.10.3 Chaining Method

The Chaining Method was introduced with the TBM in [GBA] as another technique to perform bootstrapping on several inputs. It is much more functionally restricted than the Tree-Based approach, as its main characteristic is that the output of a LUT is used to construct the selector of the next LUT, whereas, in the TBM, the output is used to construct the next LUT. Given a message space of size B, the chaining method requires using a plaintext space of size $2B^2$ (instead of only $2 \times B$ for the TBM). As such, the size of the parameters dramatically increases as the basis B increases. This growth of parameters jeopardizes the other speed improvements that could come with the chaining method. Indeed, we compared the parameter sets needed for the two approaches and decided to work with the Tree-Based one as parameters were much smaller (and thus allowed a better efficiency in computations). The different parameter sets are given in Table 2.1. We remind that B_g and l denote the basis and levels associated with the gadget decomposition, B_{KS} and t denote the decomposition basis and the precision of the decomposition of the KeySwitch. Finally, q denotes the size of the used plaintext size (meaning that the torus is discretized on q values), and ϵ is the error probability of one TBM evaluation or one evaluation of the chaining method. We also give the noises associated with the TRLWE and TLWE ciphertexts.

Table 2.1: Parameter sets depending on the LUT evaluation method and the chosen decomposition basis ($\lambda \approx 128$).

	basis	n	N	l	B_g	B_{KS}	t	q	ϵ	σ_{TRLWE}	σ_{TLWE}
Chaining Method	4	850	2048	2	2^{11}	1024	2	32	2^{-32}	9.6×10^{-11}	1.27×10^{-6}
	8	1024	8192	1	2^{28}	1024	2	128	2^{-27}	10^{-45}	5.6×10^{-8}
	16	1100	32768	1	2^{32}	8192	2	512	2^{-30}	1.4×10^{-8}	10^{-248}
Tree-Based Method	4	700	1024	5	2^{4}	1024	2	8	2^{-30}	5.6×10^{-8}	1.9×10^{-5}
	8	700	2048	2	2^{11}	1024	2	16	2^{-23}	9.6×10^{-11}	1.9×10^{-5}
	16	1024	2048	3	2^{8}	1024	2	32	2^{-23}	9.6×10^{-11}	6.5×10^{-8}

Chapter 3

Background on Transciphering

Unfortunately, FHE's fantastic ability to perform computations directly on encrypted data is not free. Indeed, with great power comes great efficiency bottlenecks.
First of all, the structure of the homomorphic ciphertexts induces a large expansion factor (that is to say, the ratio $\frac{\text{size of ciphertext}}{\text{size of cleartext}}$). Indeed, as most homomorphic encryption schemes' security relies on the (R)LWE assumptions, ciphertexts often take the form of (R)LWE samples, meaning that the expansion factor commonly reaches several hundred bits. For example, a standard set of parameters for 4-bit messages with TFHE requires $n = 700$ and $q = 32$. The TLWE encryption of a 4-bit message m would thus be $c = (a, b)$, with a a vector of n coefficients in $\mathbb{Z}_q$. That is to say, we need $700 \times 32 + 32 = 22432$ bits to encrypt m. In this case the expansion factor is $\frac{22432}{4} = 5608$...
This significant ciphertext expansion induces several performance bottlenecks: slow data transmission and slow computations.

Some ideas can be explored to address the issue of slow encryption and slow uplink data transmission. For instance, a standard approach [Reg09, Ajt05] consists of simply synchronizing the client and server using a PRF to avoid sending the a term of TLWE pairs (i.e., both the server and the client can compute on their own the a vector associated to a given $b = \langle a, s \rangle + m + e$) and thus to transmit only the unique coefficient b. The uplink expansion factor, therefore, becomes independent of the n parameter. For instance, using the same set of parameters for 4-bit messages ($n = 700$, $q = 32$) and using this optimization leads to an expansion factor of only $\frac{32}{4} = 8$, which is reasonable by "FHE standards". This synchronization between client and server allows for moderate communication overhead and negligible computation overhead. Another avenue of research with a negligible communication overhead (but moderate to large computation overhead) is transciphering, which consists of the transformation of low-overhead symmetrically encrypted data into homomorphically encrypted data. It is the solution that we investigated in this thesis and

that we present here.

3.1 The Magic of Transciphering

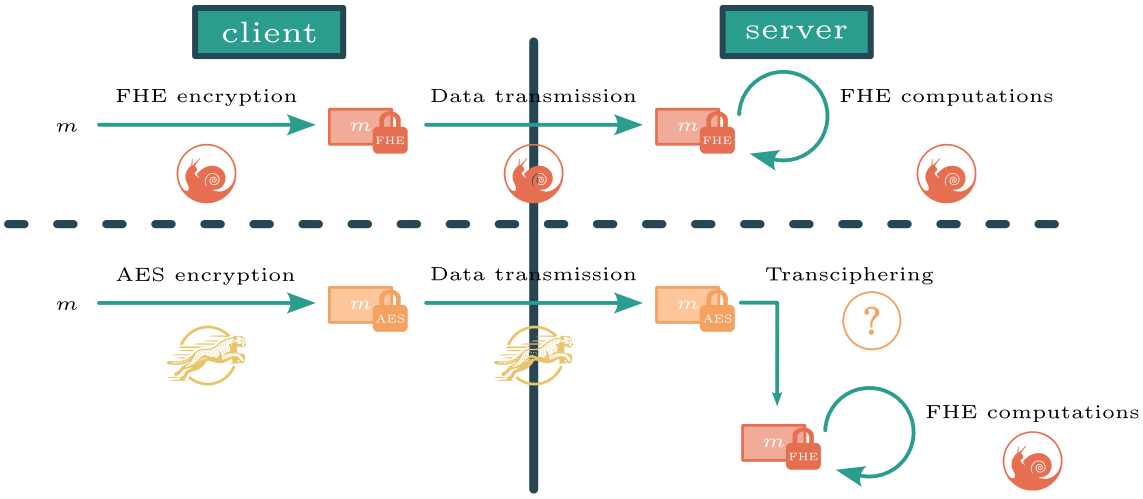


Figure 3.1: Principle of transciphering with AES for FHE.

As early as 2012, FHE pioneers were exploring transciphering to enhance the practicality of FHE. Indeed, the first attempt to transform an AES ciphertext into a homomorphic one was made shortly after the invention of the first homomorphic encryption scheme. Indeed, in [GHS12] Gentry, Halevi and Smart proposed the first transciphering of an AES ciphertext into a BGV one, which is resumed in Figure 3.1. With now obsolete parameters, they obtained an execution timing of 17.5 minutes. Two conclusions were then possible:

1. Transciphering is useless in practice.
2. AES is not the right candidate for that purpose.

Since transciphering is essential for FHE's practicality, conclusion 2 was preferred. Several teams thus decided to create new symmetric cryptosystems whose encryption operations were specifically chosen to be more rapidly executed in the homomorphic domain. As shown by a recent survey [TKLP25], the community's interest in transciphering has not diminished and is still a key area of research. In this chapter, we look at the various "Hybrid Homomorphic Encryption" (HHE) schemes designed over the years to propose an efficient transciphering.

As of today, there are many proposals, from block ciphers (LowMc [ARS+15], PRINCE [BCG+12], Chaghri [AMT22]) to stream ciphers (`Elisabeth` [CHMS22], PASTA [DGH+23], Kreyvium [CCF+18]), each coming with its pros and cons. We review them in the following sections.

Table 3.1: List of symmetric ciphers used for transciphering purposes.

Ciphers	Targeted HE	Construction	Year	Reference
Trivium	none	Stream	2006	[DCP06]
PRINCE	none	Block	2012	[BCG+12]
LowMc	BGV	Block	2015	[ARS+15]
Kreyvium	BGV,BFV	Stream	2016	[CCF+18]
FLIP	GSW	Stream	2016	[MJSC16]
Rasta	BGV,BFV	Stream	2018	[DEG+18]
Hera	CKKS	Stream	2021	[CHK+21]
Elisabeth	TFHE	Stream	2022	[CHMS22]
Rubato	CKKS	Stream	2022	[HKL+22]
Chaghri	BGV	Block	2022	[AMT22]
Transistor	TFHE	Stream	2025	[BBB+25a]

3.2 Stream Cipher Based Transciphering

Stream ciphers are usually preferred to block ciphers for their efficiency. Once the key stream is generated, the only operation to be performed is the XOR of the messages with the stream, which is one of the least costly operations in FHE. This method is significantly more efficient than applying the decryption function to each block of messages, as is done with block ciphers. Moreover, note that the cipher stream can even be pre-processed to reduce the delay.

3.2.1 BGV/BFV Friendly Ciphers

Kreyvium

Kreyvium [CCF+18] is a lightweight stream cipher designed for transciphering with BGV and (B)FV. It is an extension of Trivium, one of the ciphers selected in the eSTREAM portfolio, but with features especially designed for transciphering. Kreyvium incorporates elements of bitwise permutation and nonlinear feedback to generate a pseudorandom keystream that is XORed with plaintext to produce ciphertext. It operates with a 128-bit secret key and a 128-bit initial value. The internal state consists of three shift registers of varying lengths: two of 128 bits and one of 256 bits, similar to Trivium. Additionally, Kreyvium includes extra mixing steps in the update functions for these internal state registers. Note that in practice, Trivium is more commonly used with respect to transciphering, perhaps because it is already standardized (in the eSTREAM portfolio).

Rasta and Friends

The family of stream ciphers comprising Rasta, Masta, Dasta, Fasta, Pasta, and Agrasta represents a series of cryptographic primitives designed with an emphasis on minimizing multiplicative complexity. Indeed, these algorithms aim to reduce the number of AND operations required during their encryption and decryption processes, which is crucial for efficient homomorphic evaluations. Rasta, as proposed by Dobraunig et al. [DEG+18], is the first cipher of this family. Alongside Rasta, the authors also introduced Agrasta, an aggressively dimensioned variant with a block size slightly larger than the security level in bits. Over time, several other variants have been proposed, including Dasta by Hebborn and Leander [HL20], Masta by Ha et al. [HKC+20], Pasta by Dobraunig et al. [DGH+23], and Fasta by Cid et al. ([CIR22]) that developed the ciphers' family.

Rasta generates keystreams by applying the key to a cryptographic permutation that follows an ASASA (affine-substitution-affine-substitution-affine) structure. The core idea behind Rasta is to make large portions of the operations nonce-dependent while keeping them key-independent. This approach allows for a very low key-dependent multiplicative depth while rendering many standard attacks ineffective due to the nonce-dependent variations. Rasta minimizes both multiplicative depth and the number of multiplication per bit by randomly updating the affine layers with each round. Consequently, the security of this cipher relies on the randomness provided, although this can slow down the encryption and decryption processes.

Agrasta has a block size that matches the security level. As a consequence, Agrasta has a block size of 81 bits for 80-bit security, 129 bits for 128-bit security, and 257 bits for 256-bit security. Agrasta was developed to explore the limits of the design space by utilizing aggressive parameters, encouraging further cryptanalysis. Because generating the affine layers during each encryption in Rasta is time-consuming, the Dasta variant was introduced. Dasta replaces the linear layer with a constantly changing bit permutation and a deterministic linear mapping, making it hundreds of times faster than Rasta in offline settings.

Masta was proposed next, with two main differences from Rasta: it employs modular arithmetic to support homomorphic encryption schemes over a non-binary plaintext space and uses fewer random bits in the affine layers as they are defined using finite field multiplication. Pasta leverages the structure of the BGV and BFV schemes to reduce homomorphic evaluation latency. Finally, Fasta was designed with specific parameters and a linear layer chosen to facilitate efficient implementation over the BGV scheme, particularly as applied within the HElib library.

The designers carefully examined various attack vectors before selecting parameters for the instantiations, aiming to rule them out conservatively. However, an algebraic attack using low-degree equations, conducted by Liu et al. in 2021 [LSMI21], compromised a few instances of Agrasta. Regarding Dasta, the only recorded attack comes from Liu et al. [LSMI21], which also affected Rasta. Unfortunately, the other variants have not been extensively studied, resulting in a lack of significant

cryptanalysis findings to date.

3.2.2 CKKS Friendly Ciphers

Hera

Hera [CHK+21], proposed by Cho et al., is a stream cipher specially designed to be CKKS-friendly in the RtF (Real-to-Finite-field) transciphering framework. In the RtF transciphering framework, a modular arithmetic stream cipher is required as a building block. However, only a few existing ciphers employ modular arithmetic, and even those are often unsuitable for the transciphering framework due to their high multiplicative depths. Instead of generating random matrices, Hera aims to randomize the key schedule algorithm by combining the secret key with a (public) random value for every round. Using a simple randomized key schedule is the main feature of Hera. This simple yet effective approach allows Hera to use fewer random bits than FLIP and Rasta. In a study by Liu et al. [LKSM24], new algebraic attacks involving multiple collisions in the round keys were presented. Specifically, due to the unique way round keys are randomized in Hera, the authors demonstrated that it is possible to peel off the last nonlinear layer by exploiting collisions in the last round key. They constructed a system of equations with a significantly lower degree in relation to the key and solved the system efficiently using linearization techniques. However, this attack does not affect the primary instantiation of Hera—the 128-bit security version—as finding multiple collisions is computationally expensive.

Rubato

Rubato [HKL+22] proposed by Ha et al. is a stream cipher specially designed to be CKKS-friendly, following the RtF transciphering framework introduced for Hera. Rubato is said to significantly reduce the multiplicative complexity of the cipher compared to existing HHE frameworks. Moreover, Rubato utilizes building blocks from the Hera and Pasta ciphers: for instance, the linear layers and the key schedule are derived from Hera. Regarding security analysis, Grassi et al. [GMAH+23] demonstrated that at least 25% of the possible values for q fulfill certain criteria that could result in a successful key recovery attack. The complexity of these attacks is significantly lower than the claimed security level for five out of the six ciphers in the Rubato family.

3.2.3 GSW Friendly Ciphers

FLIP Family

FLIP [MJSC16] (family of filter permutators) and its variant FiLIP [MCJS19] are a group of stream ciphers based on a modified filter generator construction. The filter permutator paradigm, by limiting the multiplicative depth of the circuit, aims

to minimize the noise of the decryption algorithm to the evaluation of one Boolean function only. In FLIP, both encryption and decryption involve applying a different public wire-cross permutation to the secret key for each keystream bit, followed by filtering this permuted key using a unique filtering function. Since the permutation is derived from a Pseudorandom Generator and is publicly available, any error growth is primarily associated with the evaluation of the filtering function. The addition of the keystream bit to the plaintext or ciphertext has a negligible impact on this process. This structure works well with regard to HHE, as the efficiency of the framework only relies on the homomorphic evaluation of one function. Therefore, the main objective is to identify a function that minimizes noise growth in relation to an FHE scheme. Immediately after the initial design of FLIP, Duval et al. [DLR16] proposed cryptanalysis, revealing weaknesses in its filter function. Later, the design was updated to prevent the attack, and an updated design [MJSC16] was proposed.

3.2.4 TFHE Friendly Ciphers

`Elisabeth`

`Elisabeth` generalizes the filter permutator design approach used by the FLIP and FiLIP stream ciphers to arbitrary groups. The primary objective of this new design approach is to facilitate homomorphic computations in $\mathbb{Z}_q$ without the need for costly conversions. The theorization of TFHE Programmable Bootstrapping influences this initiative as PBS advocates for encryption schemes that integrate (negacyclic) Look-Up Tables and additions in $\mathbb{Z}_q$, where q is a power of 2. Overall, Elisabeth-4 necessitates only two levels of PBS and a few additions, thus allowing for a throughput of 4 bits/sec. However, Gilbert et al. presented several variants of a key-recovery attack on the full `Elisabeth-4` algorithm [GHBJR23] that breaks the 128-bit security claim of the scheme.

`Transistor`

Transistor [BBB+25a] is a recent stream cipher designed for efficient homomorphic evaluation within the TFHE framework. Transistor operates over the finite field $\mathbb{F}_{17}$, which has been selected to optimize TFHE performance. `Transistor` is composed of an AES-like "round function" (comprising a 4-bit Sbox, a MixColumn operation, and a ShiftRows) that is performed and which result is XORed with the output of a "silent LFSR". This stream is then XORed with the messages. The authors also demonstrate that an attacker cannot gain information about the secret key from three or fewer consecutive keystream outputs. Finally, the authors achieve a throughput of 60 bits per second on a standard laptop. However, this brand-new cipher has to stand the test of time.

Trivium

Trivium, proposed by De Cannière and Preneel [DCP06], is a stream cipher that was one of the finalists in the eSTREAM project and is included in the ISO/IEC 29192-3 standard for lightweight stream ciphers. It is a stream cipher that uses an 80-bit secret key (K) and an 80-bit initial value (IV). The internal structure of Trivium consists of three shift registers totaling 288 bits, along with ten XOR gates and three AND gates for feedback. Each of the three shift registers contains 93, 84, and 111 bits, respectively. The key (K) and initial value (IV) are loaded into the internal registers along with some predefined constants. During each round, a bit is shifted into the three shift registers using a non-linear combination of taps from that register and one other register, resulting in one bit of output. After the first 1,152 clock cycles, the cipher generates a valid pseudorandom bit sequence. The final ciphertext is produced by XORing the generated bit sequence with the plaintext. As an eSTREAM finalist, Trivium has been thoroughly studied [Vie07, MB07, DS09, FV14] and continues to be regarded as secure.
Even if Trivium was not designed for transciphering, numerous works have linked this stream cipher and TFHE, looking for an efficient HHE. These include [BBS22], which proposes an evaluation based on the chaining method [GBA], as well as [BOS23] and [BPR24], which use Trivium as a benchmark for the p-encodings method.

3.3 Block Cipher Based Transciphering

There are a few block ciphers specifically designed for transciphering, as the stream cipher approach is much more efficient. But even if transciphering with symmetric block ciphers is usually less efficient than with stream ciphers, it remains interesting to work on them. Indeed, block ciphers are more versatile cryptographic primitives, for example, allowing building FHE-friendly hash functions, which can be useful in some cases ([BSS+23]).

3.3.1 LowMC

LowMc is a family of block ciphers proposed by Albrecht et al. [ARS+15]. It aims to achieve a low number of multiplications and is the first cipher dedicated to transciphering, and especially with BGV. The LowMc block cipher utilizes a Substitution-Permutation Network (SPN) structure, which only applies S-boxes to specific bits of the cipher's state. In the specific case of LowMc, a 3-bit S-box with low multiplicative complexity is used. Additionally, the design minimizes the number of S-boxes used in parallel to reduce multiplicative complexity further, leaving some parts of the substitution layer as an identity mapping. The linear layer incorporates pseudorandomly generated binary matrices to maintain security despite the low multiplicative complexity. Regarding security, the first version of the cipher, LowMcv0, was in-

troduced at the end of 2014, followed by an updated version, LowMcv1 [ARS+15], early in 2015. Subsequently, a revised edition, LowMCv2, was released due to the optimized interpolation attack by Dinur et al. [DLMW15] and new higher-order differential cryptanalysis from Dobraunig et al. [DEM15]. Eventually, LowMCv3 was introduced to address the difference enumeration attack by Rechberger et al [RS18]. Some attacks on LowMcv3 have been proposed [BBVY21, LIM21, RLW25], making some parameters insecure.

3.3.2 Chaghri

In 2022 Ashur et al. presented Chaghri [AMT22], an FHE-friendly block cipher enabling efficient transciphering in BGV-like schemes. Chaghri relies on a SPN structure with elements from $\mathbb{Z}_{2^{63}}$ and a linear layer composed of 3×3 MDS matrices. A complete Chaghri circuit can be implemented using 16 multiplications, 48 Frobenius automorphisms, and 32 rotations. The authors implemented it with HElib in order to compare it with Gentry et al. work on AES. Although their implementation is claimed to be 63% faster than [GHS12], an attack on Chaghri has recently been proposed in [LAW+23]. Indeed, Liu et al. showed that a higher-order differential attack on eight rounds could be achieved using a new technique called "coefficient grouping". Hence, it indicates that the full eight rounds are far from being secure. Moreover, Liu et al. have also proposed a modification in the design to avert the attack, which has a non-negligible impact on performance.

3.3.3 PRINCE

The PRINCE block cipher [BCG+12] was not designed for transciphering, but is optimized with respect to latency when implemented in hardware. As such, PRINCE is part of the lightweight cipher family and is thus a good option for transciphering. To the best of our knowledge, very few works propose attacks against PRINCE, proving its resistance to known cryptanalysis methods. For these reasons, PRINCE has already been studied for homomorphic purposes. For instance, Doröz et al. [DSES14] proposed an implementation relying on a generalization of NTRU that took 57 minutes to execute. More recently, Bendoukha et al. [BSS+23] also studied the PRINCE block cipher with TFHE.

3.4 AES Back in the Game?

Since the work of Gentry et al. on the AES [GHS12], there have been few studies of the AES regarding transciphering. However, in 2022, the NIST announced a call for proposals on threshold encryption[1] with an FHE part, which benchmark would be

[1] `https://csrc.nist.gov/Projects/threshold-cryptography`.

the AES. This announcement has rekindled the FHE community's interest in AES, and since then, works on AES have been numerous.
The first work to launch what is now a friendly, unofficial international competition to obtain the fastest AES execution was presented in WAHC'23 [TCBS23b] and is the first contribution of this thesis. Using only TFHE's programmable bootstrapping, we obtained a sequential execution of about 4 minutes. A few months later, Bon et al. proposed a new implementation running in 135 seconds and leveraging on the *p-encodings* framework [BPR24] using TFHE. Then, twice in a row, two papers snapped the "record" by using TFHE's circuit bootstrapping in two new frameworks: Fregata [WWL+23] and Thunderbird [WLW+24]. These works brought the execution timings down to respectively 86 seconds and 46 seconds in sequential mode. Finally, in 2025, with a new framework using both the Full-LUT methodology presented at WAHC and the *p-encodings*, Belaïd et al. managed to execute the AES in only 32 seconds in sequential mode, and in 1.1 seconds with some mere parallelism [BBB+25b]. This last result is also a contribution of this thesis.

3.5 Beyond the Boolean Approach with TFHE

Homomorphic Encryption is supposed to allow any operation to be performed on ciphertexts. For a long time, the idea was to achieve this universality goal by evaluating logical gates on binary ciphertexts. Indeed, as any algorithm can be transformed into a succession of logical gates, this method allows any operation to be performed. While this computing model is universal, it also presents several efficiency bottlenecks. For instance, to perform a simple multiplication over $\mathbb{Z}_t$ (where $t >> 2$), one must execute many Boolean operations—and even more for more complex operations such as division or full algorithms. As a result, a part of the FHE research community has gradually shifted away from this model, focusing instead on running arithmetic circuits over polynomial rings with a plaintext modulus significantly larger than 2. This shift has dramatically improved the efficiency of FHE, enabling it to tackle practical applications, such as machine learning, with reasonable latencies and overheads.

However, this new approach poses challenges for applications requiring zero testing or non-linear functions. These challenges remain despite various attempts to implement comparisons and zero-testing with schemes like BFV/BGV or CKKS using bivariate polynomial optimizations or polynomial approximations. On the other hand, we have TFHE. The downside of TFHE is that it is intrinsically an LWE scheme. This means it does not support batching (except for additions) and only allows for small plaintext moduli (e.g., less than 32). On the positive side, TFHE features the most efficient bootstrapping procedure, which is also programmable. TFHE bootstrapping refreshes ciphertext noise by interpreting the input ciphertext as an encrypted index for accessing a cleartext table that encodes the identity function with some redundancies. When the identity function is substituted with

another function $f : \mathbb{Z}_t \longrightarrow \mathbb{Z}_t$, the bootstrapping operation can evaluate f "for free." Therefore, compared to the original Boolean circuit approach, TFHE offers tools to alleviate its efficiency issues by supporting a non-binary (though still small) plaintext domain $\mathbb{Z}_{2^k}$. However, this capability is rarely used, and most literature works operate on binary ciphertexts with TFHE.
In this thesis, we departed from the binary paradigm and used integer ciphertexts to answer several natural questions: Can we enhance the TFHE functional bootstrapping toolbox to achieve universal encrypted domain computations beyond the Boolean circuit approach? What would be the computational cost of such an enhancement? Is it possible to accomplish this using a restricted set of patterns based solely on functional bootstrapping, thereby maintaining a homogeneous algorithmic structure? Using the integer paradigm with TFHE, we proposed several solutions to improve transciphering with AES. Still using this tool, we were able to benchmark a zoo of standardized block ciphers and thus emphasize which specific operations should be used to create an efficient Hybrid Homomorphic Encryption scheme. To facilitate the universal application of TFHE with integer ciphertexts, we have further proposed an 8-bit FHE instruction set that advances beyond the current state-of-the-art. Details on these works are given in the following sections.

Reminder of Our Running Tools

This 2-page section has been designed as a memory refresher: it summarizes the main tools we use throughout the contributions of this manuscript. Feel free to refer back to it as needed between readings!

Tree-Based Method (TBM)

This method enables bootstrapping on multiple encrypted inputs, making it particularly suitable for evaluating multivariate functions. Here, we present how to use the TBM to evaluate bivariate functions. As for classical bootstrapping, the first step of the TBM is to construct the polynomials that will be rotated during the BlindRotate. If we want to homomorphically evaluate a function

$$\begin{aligned} f{:}\mathcal{M} \times \mathcal{M} &\longrightarrow \mathcal{M} \\ (x, y) &\mapsto f(x, y) \end{aligned}$$

we need $|\mathcal{M}|$ polynomials that we denote P_i, for $i \in \mathcal{M}$ and which coefficients correspond to the LUT of the univariate function $f_i(x) = f(x, i)$. Then, to homomorphically evaluate the function f on two ciphertexts $c_0 \in [\![m_0]\!]$ and $c_1 \in [\![m_1]\!]$, we first apply bootstrapping on each polynomial with selector c_0. We obtain $|\mathcal{M}|$ ciphertexts encrypting $f_i(m_0) = f(m_0, i)$ that we pack together with a TLWE-to-TRLWE Keyswitch into an encrypted polynomial. Note that the new polynomial corresponds to the encryption of f_{m_0}. Then we perform bootstrapping on this new encrypted polynomial, this time with selector c_1, which gives an encryption of $f(m_0, m_1)$. Figure 3.2 illustrates the XOR example applied to encryptions of 4-bit messages.

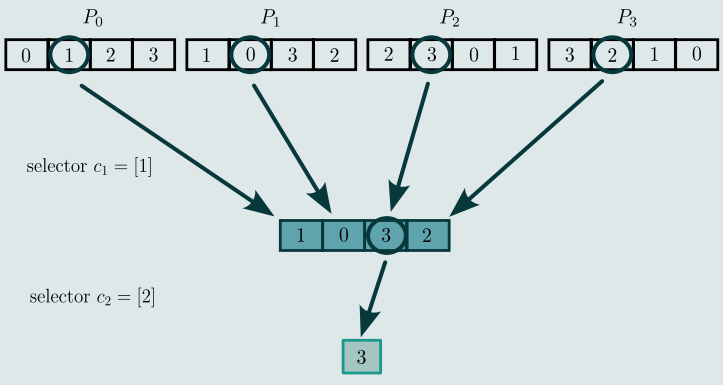


Figure 3.2: Here we use the TBM on encrypted messages in $\{0, 1, 2, 3\}$. Polynomials P_i correspond to univariate functions "XOR by i".

Multi-Value Bootstrapping (MVB)

Multi-Value Bootstrapping (MVB) [CIM18] is a method for evaluating k different LUTs on a *single* input using *only one* BlindRotate operation. Since blind rotation is the most time-consuming part of the bootstrapping process, this approach can lead to significant improvements in execution time. So, when we want to evaluate several functions on the same input, MVB is a real advantage. It is, for instance, very useful when using the TBM. Indeed, in the first layer of the tree, several LUTs are evaluated on the same encrypted input.

The key idea behind MVB is to factor the test polynomial P_{f_i} associated with the function f_i into a product of two polynomials: v_0 and v_i, where v_0 is a common factor for all P_{f_i}. With such a factorization, we can evaluate all f_i with input c the following way: we execute the BlindRotate on test vector v_0. This operation returns an encrypted polynomial that we multiply separately with each v_i. Thus, we obtain k encrypted polynomials that have been rotated by an encrypted index c. All that remains to do is proceed to the SampleExtract on these polynomials to obtain the k encrypted results of the evaluations of the k different LUTs. Figure 3.3 illustrates the benefits of this method. This optimization significantly reduces the number of BlindRotate operations required, resulting in a decrease in the overall computation time. Specifically, evaluating k LUTs using MVB only requires one BlindRotate instead of k separate ones.

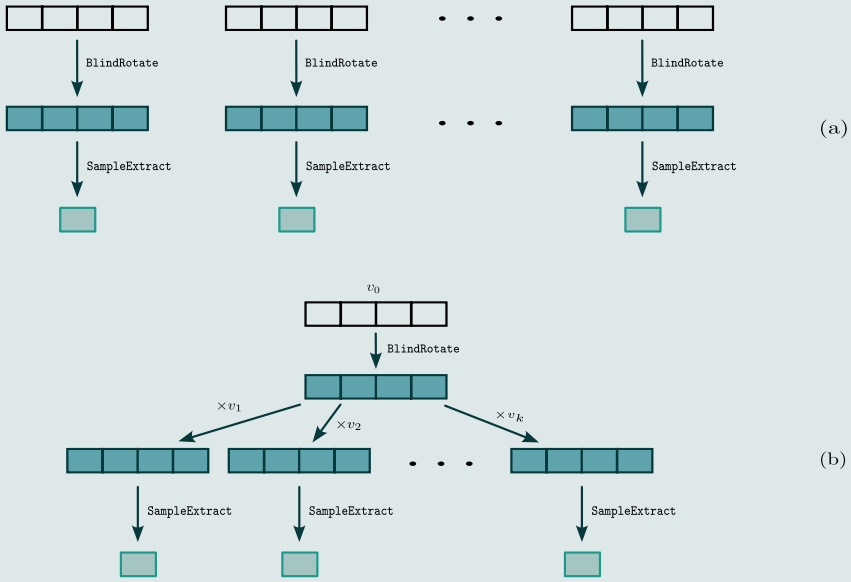


Figure 3.3: Illustration of the MVB optimization. (a) represents the classic method to process several bootstrapping, while (b) represents MVB optimization. Note that once the `BlindRotate` is performed, we then work with an encrypted polynomial, hence the coloration.

Part II

Contributions

Chapter 4

Homomorphic AES Executions with TFHE

Introduction

This chapter regroups two contributions (respectively in Section 4.2 and Section 4.3) that are presented after a brief section on the AES algorithm.

In the first contribution, we propose an AES implementation using TFHE programmable bootstrapping, which runs in less than a minute on an average laptop. We detail the transformations carried out on the original AES code as well as the optimized FHE operators we developed to lead to a more efficient homomorphic evaluation. We also duly give several execution times on different machines, depending on the type of execution (sequential or parallelized). These times vary from 4.5 minutes (resp. 54 secs) for sequential (resp. parallel) execution on a standard laptop down to 28 seconds for a parallelized execution over 16 threads on a multi-core workstation.

This first work has been the object of a paper which has been accepted to WAHC 2023: **A Homomorphic AES Evaluation in Less than 30 Seconds by Means of TFHE**, D. Trama, P.-E. Clet, A. Boudguiga, R. Sirdey.

In the second contribution, we resolve the tension between Boolean- and byte-oriented operations within the AES algorithm by proposing a new approach, coined "Hippogryph", which consistently combines the (byte-oriented) "full-LUT" approach of the first contribution with a generalization of the (boolean-oriented) p-encodings one to get the best of both worlds. Indeed, the Sbox is evaluated with the "full-LUT" approach, and the rest of the algorithm is performed with the p-encoding approach that executes Boolean circuits over binary TFHE ciphertexts.

This second work has been the object of a paper which has been accepted to

the new IACR journal *Communications in Cryptology*, CiC 2025, Volume 2: **Further Improvements in AES Execution over TFHE**, S. Belaïd, N. Bon, A. Boudguiga, R. Sirdey, D. Trama, N. Ye.

4.1 A Short Reminder on the AES

Advanced Encryption Standard (AES) is the name given to the Rijndael algorithm, the winner of the NIST standardization competition in 2000 [DR02]. It is a symmetric block cipher, defined to work with different key sizes. Several rounds are applied to the original message to obtain an encrypted message. Each round consists of the same operations performed in the same order. We chose to work on the 128-bits AES, which uses 10 rounds. A ciphertext c is composed of 16 bytes denoted $c = c_0 c_1 \cdots c_{15} \in (\mathbb{F}_{2^8})^{16}$ and is encoded in what we call a *state matrix* in the following way:

$$S = \begin{pmatrix} c_0 & c_4 & c_8 & c_{12} \\ c_1 & c_5 & c_9 & c_{13} \\ c_2 & c_6 & c_{10} & c_{14} \\ c_3 & c_7 & c_{11} & c_{15} \end{pmatrix}$$

The round operations affect this matrix as follows:

- `SubBytes`: The `SubBytes` operation is the only non-linear transformation of the cipher. It is a permutation consisting of an S-box applied to the bytes of the state matrix. As it acts on the individual bytes of the state, it can be parallelized for efficient execution.

- `AddRoundKey`: Before the encryption, the secret key is "expanded" into several round keys. Note that the encryption process relies only on these round keys and not on the initial secret key. As the name of the `AddRoundKey` transformation suggests, it performs a bitwise XOR of the state with the corresponding round key. Of course, to do so, the size of each round key is equal to the size of the ciphertext (in our case, 128 bits encoded in a round key matrix to match the state matrix). That is to say, we have at round $i \in \{0, \cdots, 9\}$:

$$\begin{pmatrix} c_0 & c_4 & c_8 & c_{12} \\ c_1 & c_5 & c_9 & c_{13} \\ c_2 & c_6 & c_{10} & c_{14} \\ c_3 & c_7 & c_{11} & c_{15} \end{pmatrix} \oplus \begin{pmatrix} k_{i,0} & k_{i,4} & k_{i,8} & k_{i,12} \\ k_{i,1} & k_{i,5} & k_{i,9} & k_{i,13} \\ k_{i,2} & k_{i,6} & k_{i,10} & k_{i,14} \\ k_{i,3} & k_{i,7} & k_{i,11} & k_{i,15} \end{pmatrix} =$$

$$\begin{pmatrix} c_0 \oplus k_{i,0} & c_4 \oplus k_{i,4} & c_8 \oplus k_{i,8} & c_{12} \oplus k_{i,12} \\ c_1 \oplus k_{i,1} & c_5 \oplus k_{i,5} & c_9 \oplus k_{i,9} & c_{13} \oplus k_{i,13} \\ c_2 \oplus k_{i,2} & c_6 \oplus k_{i,6} & c_{10} \oplus k_{i,10} & c_{14} \oplus k_{i,14} \\ c_3 \oplus k_{i,3} & c_7 \oplus k_{i,7} & c_{11} \oplus k_{i,11} & c_{15} \oplus k_{i,15} \end{pmatrix}$$

- `ShiftRows`: The `ShiftRows` step is a byte transposition that cyclically shifts the rows of the state over different offsets. For AES-128, row 0 is shifted over 0 bytes, row 1 over 1 byte, row 2 over 2 bytes and row 3 over 3 bytes. As this operation only alters the position of the bytes in the state matrix, it does not require a homomorphic equivalent (instead of shifting regular bytes, we shift homomorphic bytes).

- `MixColumns`: The `MixColumns` step is operating on the state column by column via matrix multiplication: each column of the state is multiplied by the matrix

$$M = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}.$$

  In practice, each byte can be computed individually with XORs and $GF(256)$ multiplications. For instance, to obtain the result of the `MixColumns` step on the first column of the state, we have to compute:

$$\begin{pmatrix} \bar{c_0} \\ \bar{c_1} \\ \bar{c_2} \\ \bar{c_3} \end{pmatrix} = \begin{pmatrix} 02 \times c_0 + 03 \times c_1 + c_2 + c_3 \\ c_0 + 02 \times c_1 + 03 \times c_2 + c_3 \\ c_0 + c_1 + 02 \times c_2 + 03 \times c_3 \\ 03 \times c_0 + c_1 + c_2 + 02 \times c_3 \end{pmatrix}.$$

A typical execution of the 128-bits AES begins with the first `AddRoundKey` directly followed by the first iteration of the rounds. Each round proceeds as follows:

1. `SubBytes`
2. `ShiftRows`
3. `MixColumns`
4. `AddRoundKey`

Except for the last round, which does not require the `MixColumns` step.

About the Key Expansion– `KeyExpansion` is an operation that is performed once and for all, from the secret key. Indeed, from the 128-bit key are derived eleven 128-bit round keys, which are used in the `AddRoundKey` operation. As a result, to evaluate the AES encryption or decryption algorithm, a server only needs to know the round keys. This operation, consisting of XOR and $GF(256)$ multiplication, is expensive in the homomorphic domain. It is therefore more efficient for a client to generate its own key, derive the round keys, and then homomorphically encrypt them. Sending these eleven homomorphically encrypted round keys is faster than

encrypting the initial key with FHE, and sending it to a server to perform the `KeyExpansion` in the homomorphic domain. Especially, if lots of AES blocks are sent, the cost of sending the round keys to the server is easily amortized. For these reasons, we remove the key schedule from the encrypted-domain computations.

4.2 AES Goes Homomorphic over TFHE

In this work, we propose an AES implementation relying only on TFHE's functional bootstrapping, and which runs in less than a minute on an average laptop. We provide a detailed overview of the transformations applied to the original AES code, as well as the optimized bootstrapping operators we developed. Additionally, we present various execution times recorded on different machines, taking into account the type of execution (whether sequential or parallel). These times range from 4.5 minutes for sequential execution on a standard laptop to 28 seconds for parallel execution.

4.2.1 Full-LUT Approach

To use the full potential of programmable bootstrapping, we aim to transform the AES algorithm into a succession of LUT evaluations. The permutation given by the Sbox is already in LUT form, so all that remains is to modify the multiplication in $GF(256)$ and XOR operations. These are bivariate operations, which are not easily evaluated with the classical bootstrapping procedure. Indeed, evaluating a LUT is like dereferencing an array from an encrypted index. The LUT evaluation operator, therefore, takes only one input: the encrypted index. If possible, these operations should thus be transformed into univariate table indirection operations. Since AES works on a byte-by-byte basis, LUTs must encode the 8-bit results of the evaluation of a function on 8-bit encrypted inputs. In this section, we explain the various steps involved in transforming the original code into the sequence of 8-bit-to-8-bit LUTs required for efficient homomorphic execution.

4.2.2 Optimizing the Original AES Code

The first step towards the homomorphization of a symmetric cryptosystem is to look at the original code for small changes that would ease the transition. In doing so, we quickly observed that the proposed C++ implementation (given by the creators of the AES in [DR02]) is only "pseudo" 8-bit, as a carry that requires an additional ninth bit is necessary for specific byte operations.

Indeed, this occurs during the $GF(256)$ multiplication of two integers (see Fig. 4.1). To compute this operation efficiently, the authors use a generator $\alpha = X + 1$ of the message space $GF(256) \cong {}^{\mathbb{F}_2[X]}\!/\!_{\langle X^8 + X^4 + X^3 + X + 1 \rangle}$. From this, they construct two tables of 256 elements. The first one is `Logtable`, defined such that

`Logtable`$[\alpha^i]$ = i. The second is `Alogtable` where `Alogtable`$[i]$ = α^i. Then, instead of computing the naive $GF(256)$ multiplication of the two desired bytes, the result is obtained with a simple sum of logs and two table indirections. Consequently, and as seen in Figure 4.1, when the `mul`(word8 a, word8 b) function is called, two 8-bit integers representing the logs are summed together. And this sum may exceed 255, which is why the authors apply a %255 to the result. We must thus change the whole function structure to avoid this overhead and have an actual 8-bit implementation.

```
word8 mul(word8 a, word8 b) {
    /* multiply two elements of GF(256)
    * required for MixColumns and InvMixColumns
    */
    if (a && b) return Alogtable[(Logtable[a] + Logtable[b])%255];
       else return 0;
}
```

Figure 4.1: The original $GF(256)$ multiplication implementation given in [DR02]. The sum of `Logtable[a] + Logtable[b]` may exceed 255. Therefore, this operation requires more than 8 bits in practice, which is unsuitable for a homomorphic evaluation with an 8-bit processor.

We observe that there are only a few calls to the `mul` function throughout the algorithm. During these calls, the parameter a only takes six different values. Indeed, a careful reading of the code shows that $a \in \{2, 3, 9, \mathtt{b}, \mathtt{d}, \mathtt{e}\}$. This means that we can divide the bivariate `mul`(a, b) function into six univariate `mul_a`(b) functions, with $a \in \{2, 3, 9, \mathtt{b}, \mathtt{d}, \mathtt{e}\}$ by removing the variable a of the parameters and just encoding the correct values in the corresponding new functions. Then, to work around the problematic addition of two 8-bit integers, we compute and hardcode six tables `Tlog_a` such that `Tlog_a`$[b]$ = `(Logtable[a]+Logtable[`b`])%255`. This precalculation of the tables enables us to eliminate the problem of the "pseudo" 8-bit implementation. At the same time, this allows us to avoid an explicit modulo operation, which is difficult to efficiently perform in the homomorphic domain. To optimize a step further and reduce the number of tables indirections per encrypted index, we finally consider the six new `T_a` tables such that `T_a`$[b]$ =`Alogtable[Tlog_a[b]]` and use them directly into our implementation.
We now have six univariate functions of the form:

```
word8 mul_a(word8 b) {
    if (b) return T_a[b];
    else return 0;
}
```

To avoid an explicit evaluation of the `if` condition on b (which should be performed via a conditional assignment as FHE disallows branching), we modified the

`T_a` tables so that they consider the case where $b = 0$. That is to say, for every $a \in \{2, 3, 9, b, d, e\}$, `T_a`$[0] = 0$. So our final implementations of the $GF(256)$ multiplications are straightforward and only require one indirection, as seen in Figure 4.2.

```
word8 mul_a(word8 b) {
  return T_a[b];
}
```

Figure 4.2: Our final $GF(256)$ multiplication operator. The new multiplication functions are very simple and easy to execute in the homomorphic domain, thanks to pre-calculation and table hardcoding.

Therefore, this work on the multiplication functions allows us to operate with only one table indirection instead of

- three indirections,
- an addition,
- a modulo operation,
- and an if condition

as proposed in the original code version. These changes allow us to convert the initial bivariate $GF(256)$ multiplication operator into several univariate operators, which is ideal for a "full-LUT" transformation.

When investigating for other possible optimizations, we concluded we achieved an optimal or nearly optimal form regarding LUT factorization and, thus, LUT-based homomorphic evaluation (at least when starting from the standard AES implementation). Finally, as an additional optimization, we have merged the `AddRoundKey` function with the `SubBytes` one, as the first always precedes the second one, for increased parallelism efficiency.

4.2.3 Finally Going Homomorphic for Real

As we now have univariate $GF(256)$ multiplication operators, we just need to transform the XOR into a succession of LUT evaluations. As XOR is a bivariate operation, we must use a method allowing us to bootstrap on several encrypted inputs. As discussed in Sections 2.8 and 2.10.3, we choose the Tree-Based Method (TBM) of [GBA].

On top of this, using the MVB optimization (presented in Section 2.9) in association with the TBM allows us to evaluate a LUT on two encrypted inputs with only two BlindRotate. Still, as the parameters needed for calculation directly on 8-bit inputs are gigantic, it will not be efficient. Indeed, we would need to use

the cyclotomic polynomial $X^{32768}+1$, which results in a very slow bootstrapping and a non-implementable small variance (still for 135 bits of security according to the lattice-estimator). To also avoid my computer crashing when trying to generate keys for such parameters, we had to break the messages down into a smaller decomposition basis B.

Table 4.1: Number of BlindRotate for one XOR or one Sbox evaluation (using MVB) depending on the decomposition basis. XOR requires less bootstrapping as we only perform it digit-wise.

basis	# XOR	# LUT
256	2	1
16	4	4
8	6	30
4	8	88

Choosing the optimal decomposition basis is not easy. Indeed, a small decomposition basis induces a greater number of bootstrapping for the Sbox or multiplication evaluation. But on the other hand, the smaller the decomposition basis, the smaller the parameters to be used. Small ciphertext sizes allow small parameters and, therefore, faster operations. A tradeoff must thus be achieved between the number of bootstrappings to perform and the size of the parameters. For this reason, we produce a benchmark of the execution time of LUT evaluation depending on the ciphertext decomposition.
We consider the following different decompositions:

- basis 16: The message m is decomposed into two digits in basis 16 (4 bits per digit): $m = 16m_0 + m_1$.
- basis 8: The message m is decomposed into three digits in basis 8 (3 bits per digit, but only 2 bits for the most significant digit): $m = 8^2m_0 + 8m_1 + m_2$.
- basis 4: The message m is decomposed into four digits in basis 4 (2 bits per digit): $m = 4^3m_0 + 4^2m_1 + 4m_2 + m_3$.

The results of this experiment, given in Table 4.2, prove that basis 16 is the optimal choice regarding the execution time per LUT evaluation as well as for a full AES execution (as the overall number of 8-bit entries LUT evaluations is independent of the basis choice). In fact, our work shows that *basis 16 is the optimal decomposition basis when working only with TFHE's functional bootstrapping.* As a bonus, Table 4.3 provides the number of bootstrappings needed with the MVB optimization for a full AES evaluation, depending on the decomposition basis.

Now that we have identified the optimal decomposition basis, we have everything needed to obtain homomorphic versions of the `mul_a` functions. To do so, as already

Table 4.2: Unitary timings for a classical bootstrapping and full LUT evaluation depending on basis and parameter choices ($\lambda \approx 128$). MVB is applied for complete LUT evaluations.

basis	single boot.	complete LUT eval.	n	N
256	1.5s	1.5 s	1024	32768
16	0.029s	**0.3 s**	1024	2048
8	0.015s	1.4 s	700	2048
4	0.007s	2.0 s	700	1024

emphasized, we decompose every table `T_a` into two tables of 256 basis 16 digits, i.e., one per digit of the decomposition. That is to say, to each `T_a` table will correspond two tables `T_a_msn` and `T_a_lsn` where "msn" stands for "most significant nibble" and "lsn" stands for "least significant nibble". These tables are defined as follows: for $i \in \{0, \cdots, 255\}$, $\texttt{T_a_msn}[i] = \lfloor \frac{\texttt{T_a}[i]}{16} \rfloor$ and $\texttt{T_a_lsn}[i] = \texttt{T_a}[i] \pmod{16}$.
We apply the same method to create two new basis-16 lookup tables corresponding to the Sbox. Finally, we use a similar approach to convert the XOR operator into a LUT evaluation: we create a single table `xor` such that for $i, j \in \{0, \cdots, 15\}$, $\texttt{xor}[16i + j] = i \oplus j$. This being done, we now have duly "translated" our optimized-for-FHE version of the AES code into a real homomorphic one.

4.2.4 Some More Details about the Homomorphization of the XOR Operator

Using decomposition basis $B = 16$ means that we decompose our 8-bit messages into two 4-bit digits the following way: $m = 16m_0 + m_1$, with $m_0, m_1 \in \{0, \cdots, 15\}$. The corresponding ciphertext is a vector of the form $c = (c_0, c_1) \in [\![m_0]\!] \times [\![m_1]\!]$. So, to compute the XOR of two ciphertexts c and c', we have to compute:

$$c \oplus c' = (c_0, c_1) \oplus (c'_0, c'_1) = (c_0 \oplus c'_0, c_1 \oplus c'_1)$$

Using the table `xor` defined above, the tricky part is to choose the correct way to construct the test polynomials for the first level of the TBM. Indeed, on the first step of the tree, we have 16 test polynomials. Each one must encode the coefficient of a univariate XOR. That is to say, the first polynomial P_0 encodes the values of the univariate operation `xor_by_0` such that for $i \in \{0, 1, \cdots, 15\}$ $\texttt{xor_by_0}(i) = i \oplus 0$, the second polynomial P_1 encodes `xor_by_1`, etc. So when applying the bootstrapping on these polynomials with selector $c_0 = [m_0]$, we obtain 16 TLWE ciphertexts. using a TLWE-to-TRLWE KeySwitch, we "pack" them together to create the TRLWE P_{final} encoding `xor_by_m0`. We then apply the bootstrapping on P_{final} with selector $c'_0 = [m'_0]$, and we obtain the final result, which is an encryption of $c_0 \oplus c'_0$.

Table 4.3: Number of operations and the corresponding number of bootstrapping depending on decomposition basis. Readers are reminded that AES-128 consists of a `RoundKey`, followed by 9 classic rounds and a final round that does not include the `MixColumns` operation.

basis	function	# XOR	# LUT	# Boot
256	`AddRoundKey`	16	0	32
	`SubBytes`	0	16	16
	`MixColumns`	48	32	128
	Round	64	48	176
	Full AES	608	448	1664
16	`AddRoundKey`	16	0	64
	`SubBytes`	0	16	64
	`MixColumns`	48	32	320
	Round	64	48	448
	Full AES	608	448	4224
8	`AddRoundKey`	16	0	96
	`SubBytes`	0	16	480
	`MixColumns`	48	32	1248
	Round	64	48	1824
	Full AES	608	448	17088
4	`AddRoundKey`	16	0	128
	`SubBytes`	0	16	1408
	`MixColumns`	48	32	3200
	Round	64	48	4736
	Full AES	608	448	44288

By using the same method, we can compute $c_1 \oplus c'_1$ and obtain the vector $c \oplus c' = (c_0 \oplus c'_0, c_1 \oplus c'_1)$ encoding $m \oplus m'$.

4.2.5 Remarks and Perspectives on LUT Evaluated Operations

Note

In this section, which was part of the 2023 paper, we discuss our thoughts of the time on the bootstrapping tools we developed. The concept of the 8-bit processor abstraction (a subsequent contribution of this thesis) was only a vague idea we wanted to investigate. We believed it was worth exploring and felt confident we could implement it using a method similar to the one we proposed for evaluating the AES. With notations still in the early stages (for instance, the LUT operator later became LUTeval), we aimed to establish a solid foundation for the universality of our operators. These initial concepts were further examined and developed, ultimately leading to the creation of our 8-bit processor abstraction developed in section 5.

In this section, we briefly present our various operations in a unified way and illustrate that *it allows us to express more general operations than needed for our AES execution attempt*. A distinction must be made between ciphertext-ciphertext operators and cleartext-ciphertext operators. The latter is the case of the $GF(256)$ multiplication operator in the AES algorithm, which we turned into several univariate operators (the `mul_a` functions, which could easily have been as many as 256 if it had been required). The Sbox, as a univariate instruction, is implemented following the same structure. Regarding the case of ciphertext-ciphertext operators, we can divide them into two types: the bitwise ones and the others.

Bitwise ciphertext-ciphertext operator. The AES implementation requires an XOR operation, which can finally be implemented using the same tool as for the univariate functions (`Sbox`, `mul_a`). Slightly loosely speaking, let us say that our 8-bit-to-4-bit table dereferencing tool takes the following form $\mathsf{LUT}(c_0, c_1, table)$ and in basis 16 $\mathsf{LUT}(c_0, c_1, table) = table\,[c_0 + c_1 \cdot 16]$. To evaluate the univariate operator of the `Sbox` (or `mul_a`), one must call $\mathsf{LUT}(c_0, c_1,$ `Sbox_0`$)$ and then $\mathsf{LUT}(c_0, c_1,$ `Sbox_1`$)$, where `Sbox_0` and `Sbox_1` respectively denote the least significant bits and most significant bits of each coefficient of the LUT `Sbox`, to obtain the two result digits and reform `Sbox`$[c_0 + c_1 \cdot 16]$ (respectively $\mathsf{LUT}(c_0, c_1,$ `mul_a_0`$)$ and $\mathsf{LUT}(c_0, c_1,$ `mul_a_1`$)$). But for XOR, we use $\mathsf{LUT}(c_0, c'_0,$ XOR$)$ and then $\mathsf{LUT}(c_1, c'_1,$ XOR$)$ as explained above. In the first case, we use the same ciphertexts on different tables; in the second case, we use different ciphertexts on the same table. This means a bivariate bitwise homomorphic operator is as easy (and costly) to implement as a univariate non-bitwise operator. Furthermore, such an instruction can easily be

adapted to any bitwise operation (AND, NOR, etc.) or univariate operation by modifying the initial polynomials accordingly.

Non-bitwise ciphertext-ciphertext operator. Although not needed for AES, the case of non-bitwise ciphertext-ciphertext operators is a bit more involved. Let us take the example of the addition of two encrypted messages in basis 256, c and c', decomposed in basis 16. Two different tables with 256 4-bit entries then come into play: `ADD`, which encodes the addition of two digits, and `CAR`, which handles the carry. So, using the same dereferencing tool as above, a simple addition over $\mathbb{Z}_{256}$ can be broken down into 4 invocations of the operator:
$\mathsf{LUT}(c_0, c'_0, \mathsf{ADD}) = res_0$
$\mathsf{LUT}(c_0, c'_0, \mathsf{CAR}) = r$
$\mathsf{LUT}(c_1, c'_1, \mathsf{ADD}) = v$
$\mathsf{LUT}(v, r, \mathsf{ADD}) = res_1$

where $\bar{c} = (res_0, res_1)$ encrypts the result of $c + c'$. The dereferencing tool is the same for all of these cases and is, in essence, universal, although on a case-by-case basis, there might be more optimized methods for certain operations (e.g., by combining the tree-based and chain-based methods, the latter being, for example, more efficient for carry computations). This gives interesting perspectives for building a more general optimized "instruction set".

4.2.6 Experimental Results

TFHE Library Implementation

We worked with TFHElib[1] using the parameters given in Table 4.4. These parameters were especially chosen to enable MVB evaluation with error probability $\epsilon = 2^{-23}$. According to the lattice estimator[2], these give us at least 128 bits of security.
Once the parameters set was defined, the first step was to implement the TBM and its MVB optimization. Indeed, none of these methods are part of TFHElib. Furthermore, we had to adapt the code to be able to use vectors of TLWE and TRLWE samples (as our ciphertexts are decomposed into 2 digits in basis 16, we put them in vectors of 2 elements). We thus created a generic and optimized LUT evaluation tool that takes as input a vector of ciphertexts, the number d of input digits, the number d' of output digits, the decomposition basis B and the LUT to be evaluated. Depending on how it is called, this operator can perform our homomorphic XOR as well as our $GF(256)$ multiplications and Sbox.

[1] `https://tfhe.github.io/tfhe/`
[2] `https://github.com/malb/lattice-estimator`

Table 4.4: Parameter set to work with decomposition basis $B = 16$ ($\lambda \approx 128$).

n	N	l	B_g	B_{KS}	t	q	ϵ	σ_{TRLWE}	σ_{TLWE}
1024	2048	3	2^8	1024	2	32	2^{-23}	9.6×10^{-11}	6.5×10^{-8}

A Word about Parallelization

The purpose of transciphering is to avoid transferring large ciphertexts. Since a server has more computing power than a client, it can efficiently exploit several cores to parallelize computations and optimize execution times. With that respect, AES is naturally parallelizable up to a certain degree: within each round step, operations can be performed simultaneously on each byte of the state matrix (except for the `ShiftRows` step). This is why, in continuing our work, we use the OpenMP library to parallelize our code and optimize the execution times of our homomorphic AES. We run our tests on two machines. The first one is a 12th Gen Intel(R) Core(TM) i7-12700H CPU (using six cores) laptop with 64 Gib total system memory with an Ubuntu 22.04.2 LTS server. The second is an AMD EPYC 7702P 64-cores Processor server with an Ubuntu 20.04.6 LTS server. In the following sections, we will refer to them respectively as `i7-laptop` and `AMD-server`.

Table 4.5: Our execution times compared to the state of the art (the `"i7-server"` time corresponds to an extrapolation of the speedup observed with 16 threads on the `AMD-server` from the `i7-laptop` sequential time as we did not have an i7-based server to perform experiments).

	execution time	time ratio
`i7-laptop` (1 thread)	4.5 mins = 270 secs	9. 4
`i7-laptop` (6 threads)	54.31 secs	1.9
`AMD-server` (1 thread)	5.7 mins = 342 secs	11.9
`AMD-server` (16 threads)	36.39 secs	1.3
`"i7-server"` (16 threads)	28.73 secs	1
Gentry et al. [GHS12] (1 thread)	18 mins	37.6
Mella and Susella [MS13] (1 thread)	22 mins	45.9
Stracovsky et al. [SMK22] (16 threads)	4.2 mins = 252 secs	8.8

Using the OpenMP library, we first parallelize with the six cores of the `i7-laptop`. As it is a standard laptop, it is interesting to see how fast a partially parallelized homomorphic AES can go. Indeed, it gives a reasonable hint of the potential of a larger scale parallelization. That is why we parallelize every round function, except for the `ShiftRows` one, as it only involves ciphertexts reorganization within the state matrix. Still, thanks to OpenMP, we could use 16 cores for a larger-scale parallelization on the `AMD-server` (which hosts 64 cores in total). As the server has less powerful cores, it is slower than the `i7-laptop` in sequential time. But,

by running a sequential execution and a full parallelized one, we can measure the speedup factor induced by parallelization. For instance, the execution parallelized on 16 cores is 9.5 times faster than the sequential one on the `AMD-server`.
Results can be found in Table 4.5. The last column of this table indicates the ratio between the (fastest) `"i7-server"` timing and the other (slower) ones.

Even if our measured speedups are not linear in the number of cores, we improve the state of the art as the so far best-known implementation runs in 4.2 minutes with 16-thread parallelization [SMK22] using the standard programmable bootstrapping in base 16 as implemented in Zama's Concrete [3]. However, note that no implementation details were provided (to the best of our knowledge) beyond a poster presented by Stracovsky et al. at FHE.org 2022 [SMK22]. In [GHS12], Gentry et al. use a packed implementation with BGV so that each slot can hold a state byte. The round operations are computed as single linear transformations over $GF(2^8)^{16}$. These linear transformations combine several permutations that are computed via automorphism evaluation and additions. In [MS13], Mella and Susella improve these latter results by using an optimized data representation to further factor some of these operations in ciphertext slots. As they relied heavily on batching, these works focused primarily on optimizing the amortized time metric, which is relevant only when one has to perform many independent AES executions in parallel. As Table 4.5 shows, we obtain a *sequential* time comparable to [SMK22] on `i7-laptop` and a very improved one with parallel execution. Indeed, our parallelized version on `i7-laptop` is 4.6 times more efficient than Stracovsky et al.'s [SMK22] and almost 20 times more efficient than Gentry et al.'s. [GHS12] in terms of latency. Furthermore, our parallelized version on `AMD-server` is almost 7 times faster than [SMK22] and 30 times faster than [GHS12] as it only takes 3% of its execution time.

4.2.7 Concluding Remarks on the First Contribution

This first contribution was the starting point of a friendly international competition to obtain the "record" of the fastest homomorphic AES evaluation. This is, thus, with no surprise, that we discovered a few months after our publication a CryptoExperts' team work that had just been published that proposed better execution timings [BPR24]. This work was followed by another from another team that brought the execution timings under 90 seconds [WWL+23], and so on. In late 2024, the fastest sequential AES evaluation ran in 46 seconds [WLW+24]. And in 2025, we came back stronger in association with the CryptoExperts team behind [BPR24] and proposed a new framework providing better results: this is the story of Hippogryph.

[3] https://github.com/zama-ai/concrete

4.3 The Hippogryph Framework

In the second contribution, we address the tension between Boolean- and byte-oriented operations within the AES algorithm by proposing a new approach, named "Hippogryph". This method consistently combines the byte-oriented "full-LUT" approach approach presented above with a generalized version of the Boolean-oriented p-encodings. We show that this new framework is competitive with other state-of-the-art AES homomorphic evaluations as it improves the last "record" by a factor 1.4.
Note that most of the text of this section, and especially the p-encoding parts, is reused from [BBB+25b].

4.3.1 Generalization of the "p-encodings" Approach

The work of [BPR24] takes an orthogonal approach compared to the one presented in the first contribution of this manuscript. In this method, data is encrypted bit by bit, and only Boolean operations are performed. It leverages the fact that, in the plaintext space $\mathbb{Z}_2$, the native TFHE addition actually performs an XOR. Thus, the linear operations `MixColumns` and `AddRoundKey` can be efficiently performed with minimal cost, using only the native TFHE addition. Specifically, the `MixColumns` step is evaluated following the circuit representation proposed in [Max19], which is ideally only composed of shifts and XORs. Furthermore, because operations are performed on individual bits, the `ShiftRows` transformation can be evaluated for free, as it merely involves rearranging the ciphertexts.

Evaluating `SubBytes` is trickier. Using $p = 2$ (i.e., plaintext space $\mathbb{Z}_2$), it is impossible to use native TFHE operations to perform a bivariate Boolean gate other than XOR. Thus, evaluating the Boolean circuit of the Sbox (which comprises AND gates) cannot be done. To deal with this problem, the authors introduced the notion of p-encoding, which embeds the bits into a larger space $\mathbb{Z}_p$ with $p > 2$.

In this section, we generalize this notion beyond the Boolean case by defining the (o, p)- encoding construction. Informally, instead of embedding the Boolean space in $\mathbb{Z}_p$, we embed any space $\mathbb{Z}_o$ in $\mathbb{Z}_p$ (with $o < p$). So, what was called p-encoding in [BPR24] corresponds to a $(2, p)$-encoding in this work. Definition 1 formalizes this generalization[4]:

Definition 1 ((o,p)-encoding)**.** Let $\mathbb{Z}_o$ be the message space. A (o,p)-*encoding* is a function $\mathcal{E} : \mathbb{Z}_o \mapsto 2^{\mathbb{Z}_p}$ that maps each element of $\mathbb{Z}_o$ to a subset of the discretized torus $\mathbb{Z}_p$. A (o,p)-encoding is *valid* if and only if:

$$\begin{cases} \forall (i,j) \in \mathbb{Z}_o^2, i \neq j, \mathcal{E}(i) \cap \mathcal{E}(j) = \varnothing \quad \text{and} \\ \text{if } p \text{ is even: } \forall\, x \in \mathbb{Z}_p, \forall i \in \mathbb{Z}_o : x \in \mathcal{E}(i) \iff \left[x + \frac{p}{2}\right]_p \in \mathcal{E}([-i]_o) \end{cases} \tag{4.1}$$

[4] (o,p)-encodings being simply a change of encoding in the plaintext space, they have no impact on the ciphertext structure nor on the bootstrapping procedure.

The latter property is a direct consequence of the negacyclicity of the torus.

In this work, we focus exclusively on cases where $p = 2$ or p is an odd prime. As a result, a lot of the subleties of negacyclicity can be overlooked. Furthermore, among the various types of (o, p)-encodings, one particular class proves especially useful for our purposes: the *canonical* (o, p)-encoding.

Definition 2 (canonical (o, p)-encoding)**.** A (o, p)-encoding $\mathcal{E}$ is said *canonical* if and only if it satisfies:

$$\begin{aligned} \mathcal{E} : \mathbb{Z}_o &\to \mathbb{Z}_p \\ x &\mapsto x \end{aligned}$$

(with $o < p$). Informally, we simply embed a smaller space into a larger one, without altering the order of the elements.

In [BPR24], the `SubBytes` circuit is evaluated using $(2, 11)$-encoding, while the other steps are evaluated with a $(2, 2)$-encoding (i.e., the trivial encoding of TFHE). Consequently, an *encoding switching* operation is required. This operation can be straightforwardly performed using a PBS.

Definition 3 (encoding switching)**.** Let c be a ciphertext encrypting a message $m \in \mathbb{Z}_o$ under the (o, p)-encoding $\mathcal{E}$. Its encoding can be switched to the (o, p')-encoding $\mathcal{E}'$ by applying a PBS on c evaluating the function:

$$\begin{aligned} \texttt{Cast}_{\mathcal{E} \mapsto \mathcal{E}'} : \mathbb{Z}_p &\to \mathbb{Z}_{p'} \\ x &\mapsto x' \end{aligned}$$

where x' is defined as $\forall i \in \mathbb{Z}_o, x \in \mathcal{E}(i) \implies x' \in \mathcal{E}'(i)$.

It remains to explain how the `SubBytes` circuit is evaluated in [BPR24]. The authors use a circuit representation for the Sbox ([BP10]), and decompose it into so-called *gadgets*, which are smaller subcircuits evaluable in one single bootstrapping if the inputs are provided under the right p-encodings. This makes the evaluation of the circuit of the Sbox much faster than with the naive approach of "gate bootstrapping" where every logic gate is evaluated with a bootstrapping. As an order of magnitude, the authors of [WWL+23] have also implemented the AES with the "gate bootstrapping" approach and report a sequential evaluation timing of more than an hour on a standard laptop.

Manipulating the data bit-per-bit makes the evaluation of the linear part blazingly fast, however the circuit decomposition into gadgets is less efficient (even if better than gate bootstrapping).

4.3.2 Design of Hippogryph

Building on the foundation of the two previous works, we develop a hybrid approach, Hippogryph, that not only combines their respective strengths but also introduces new contributions to enable their effective integration. The guiding principles of this design are as follows:

- The `SubBytes` step, which was the weak point of [BPR24], is evaluated using the strategy presented in Section 4.2.
- Conversely, the linear steps (namely `ShiftRows` , `MixColumns` and `AddRoundKey`) are computed using a trivial $(2, 2)$-encoding, which makes them extremely fast.
- Since the two aforementioned points rely on different data representations (arithmetic for `SubBytes` and Boolean for the other steps), a decomposition layer and a recomposition layer are necessary to transition from one to another. The decomposition and recomposition steps are denoted by Decomposer and Recomposer, respectively.

Our design for one round of AES is summed up in Figure 4.3. In the following we explain each of its components.

`SubBytes`. The `SubBytes` step is implemented following the design of [TCBS23b] (Section 4.2). Each 8-bit input is represented by two ciphertexts, each encrypting a 4-bit limb. Two instances of the TBM are then used to compute the limbs of the output. The only modification from the design of [TCBS23b] is the adoption of the canonical $(16, 17)$-encoding:

$$\begin{aligned} \mathcal{E}_{17} : \mathbb{Z}_{16} &\to \mathbb{Z}_{17} \\ i &\mapsto i. \end{aligned}$$

In Figure 4.3, ciphertexts encrypted under this $(16, 17)$-encoding are represented by blue rectangles. This process is repeated 16 times, once for each byte of the AES state. An additional improvement comes from the fact that the two TBM are using an MVB to evaluate the first step. So, the same common factor can be used for both evaluations, requiring only one BlindRotate per byte for this first step.

Linear Circuit. For this part, we follow the design of [BPR24]. The ciphertexts manipulated in this block are encoded under the trivial $(2, 2)$-encoding $\mathcal{E}_2$, and encrypt a single bit each. They are represented by yellow squares on Figure 4.3. Consequently, this circuit takes 256 inputs (one for each of the 128 bits in an AES block, and one for each of the 128 bits in the current round key), and outputs a new state of 128 bits, by combining the following three steps:

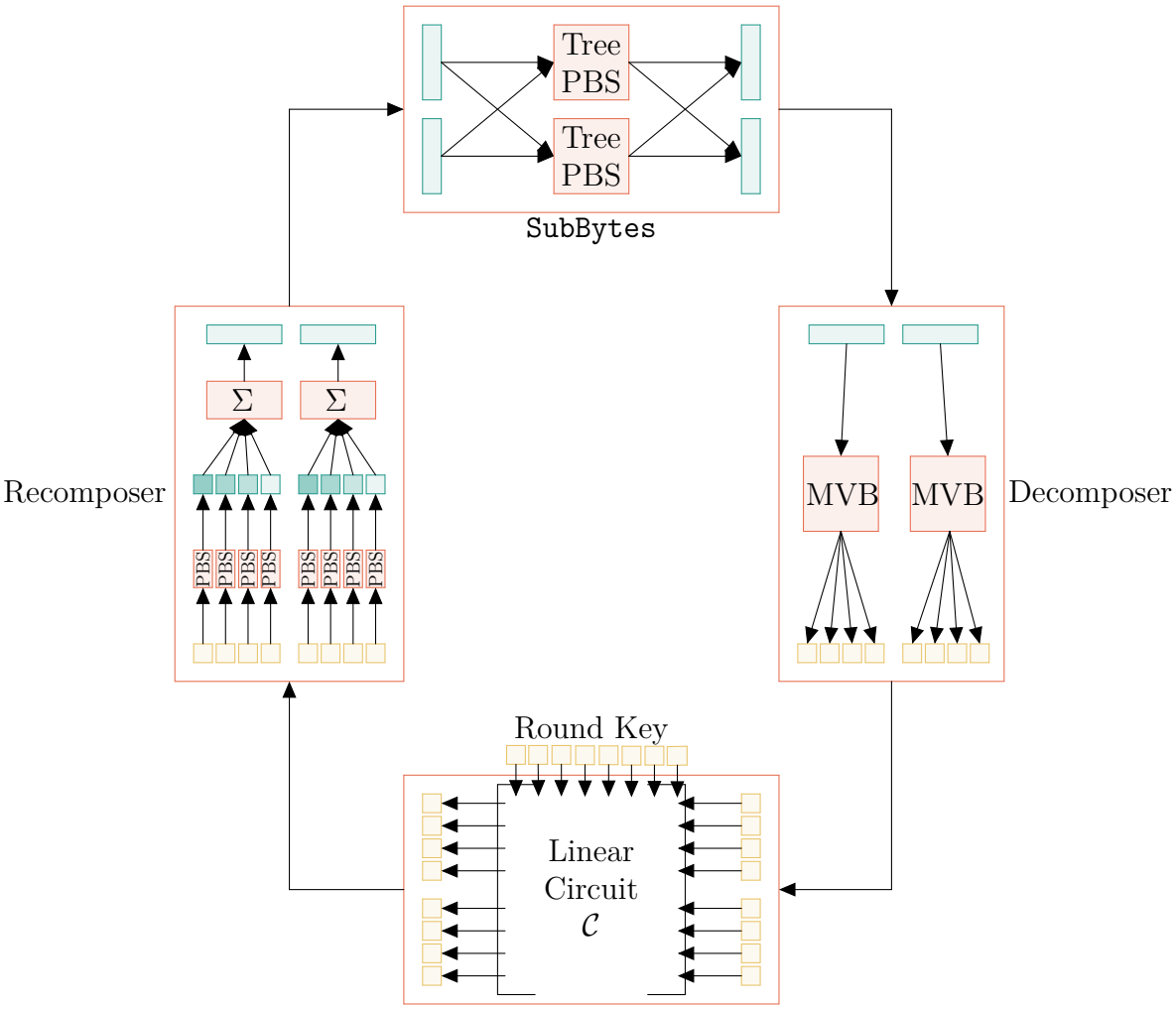


Figure 4.3: Structure of one round of AES with Hippogryph. Ciphertexts in blue live in $\mathbb{Z}_{17}$ while the ones in yellow are in $\mathbb{Z}_2$. Squares represent encryptions of one single bit while rectangles represent nibbles.

- `ShiftRows` : This step is implemented by permuting the input ciphertexts according to the AES specifications.
- `MixColumns` : Here, we use the XOR-and-shifts-only circuit representation of [Max19]. XOR gates are evaluated on ciphertexts under $\mathcal{E}_2$ by using the native addition of TLWE ciphertexts.
- `AddRoundKey` : This step is a simple XOR between the state and the round key.

Evaluating the additions corresponding to XOR increases the noise in the ciphertexts. However, this problem can be overlooked: using $p = 2$, there is plenty of room for the noise to grow, so the bottleneck of the construction in terms of noise is actually the TBM in $\mathbb{Z}_{17}$. In our experimentations, we made sure to select parameters ensuring correctness up to the target probability of success.

Decomposer. At the end of the `SubBytes` step, a switch of representation is needed to pursue with the linear circuit evaluation. We need to decompose each nibble encryption into 4 ciphertexts, each encrypting a single bit. Simultaneously, we must switch from encoding $\mathcal{E}_{17}$ to encoding $\mathcal{E}_2$. Fortunately, by combining the MVB primitive and the encoding switching primitive (from Definition 3), it is possible to perform both changes with a single PBS. Formally, we use an MVB to evaluate four LUTs on each nibble. These LUTs are defined as follows:

$$\text{for } x \in \{0, \dots, 15\}, \mathsf{decomp_i}[x] = (x \texttt{ >> i) \& 1}$$

where $\mathsf{decomp_i}[x]$ gives the i-th bit of a nibble x.

Recomposer. Conversely, a transformation from the Boolean domain to $\mathbb{Z}_{17}$ is required after the `AddRoundKey` step. As for the Decomposer operation, this involves two key steps:

- Casting the ciphertexts from a plaintext modulus of 2 to 17.
- Recombining each group of 4 bits into a single ciphertext, encrypting the whole nibble.

To achieve this efficiently, we introduce four intermediary $(2, 17)$-encodings, namely:

$$\forall i \in \{0, \dots, 3\}, \mathcal{E}_{17}^{(i)} : \mathbb{Z}_2 \to \mathbb{Z}_{17}$$
$$x \mapsto \begin{cases} 0 \text{ if } x = 0 \\ 2^i \text{ if } x = 1. \end{cases}.$$

We perform this encoding switching on the four bits ($i \in \{0, \cdots, 3\}$) of each nibble, transitioning from $\mathcal{E}_2$ to $\mathcal{E}_{17}^{(i)}$. In Figure 4.3, the resulting ciphertexts are represented

by squares filled with different shades of blue. Once the bits are expressed in this intermediary representation, we use the native TFHE TLWE addition to reconstruct the nibble in $\mathcal{E}_{17}$.

The inputs of the Recomposer operation are encrypted modulo 2. Since no padding bits are used, the negacyclicity of the torus induces evaluating a negacyclic function with the PBS of the Recomposer operation. As stated in Property 1, the existence of a Boolean recomposition algorithm relying solely on PBS and linear operations depends on the parity of the output plaintext modulus.

Property 1. A Recomposer using only linear operations and one PBS per bit exists only if the output modulo is **odd**.

Proof. Let p be an integer. Let $(b_0, \dots, b_{d-1})$ be the bits to encrypt, and let $(c_0, \dots, c_{d-1})$ denote their corresponding ciphertexts, encoded with the trivial $(2,2)$-encoding $\mathcal{E}_2$. We aim to construct a Recomposer that uses only one programmable bootstrapping per bit and linear operations to homomorphically compute an encryption of the message $m = \sum_{i=0}^{d-1} b_i 2^i$ under the canonical $(2^d, p)$-encoding $\mathcal{E}_p$. The purpose of this proof is to demonstrate how the parity of p influences the existence of such an algorithm.

To do so, following the blueprint introduced earlier in the section, we want to bootstrap the ciphertext c_i into $\mathbb{Z}_p$ with the p-encoding $\mathcal{E}_p^{(i)} = \begin{cases} \mathbb{Z}_2 \mapsto \mathbb{Z}_p \\ 0 \mapsto \{0\} \\ 1 \mapsto \{2^i\} \end{cases}$. Once we have those, a simple sum will reconstruct the message under the canonical $(2^d, p)$-encoding.

As the ciphertexts are encrypted modulo 2, there is no bit of padding. So, if we send them modulo p with a PBS, the result will necessarily be encoded under a negacyclic $(2,p)$-encoding, that is to say of the form: $\mathcal{E}^{(\text{neg})} = \begin{cases} \mathbb{Z}_2 \mapsto \mathbb{Z}_p \\ 0 \mapsto \{\gamma\} \\ 1 \mapsto \{-\gamma\} \end{cases}$ with $\gamma \in \mathbb{Z}_p$.

Now, we need a linear transformation that casts a ciphertext from $\mathcal{E}^{(\text{neg})}$ to $\mathcal{E}_p^{(i)}$. Let us denote this hypothetical linear transformation by $\mathcal{L}$, and define it as:

$$\begin{aligned} \mathcal{L} : \mathbb{Z}_p &\mapsto \mathbb{Z}_p \\ x &\mapsto \alpha \cdot x + \beta \end{aligned}$$

By simply considering the encoding switching from $\mathcal{E}^{(\text{neg})}$ to $\mathcal{E}_p^{(0)}$, it is clear that the constants α and β need to verify the property:

$$\begin{cases} \alpha \cdot \gamma + \beta = 0 \mod p \\ \alpha \cdot (-\gamma) + \beta = 1 \mod p \end{cases}$$

which can be rewritten as:

$$\begin{cases} \beta = 2^{-1} \mod p \\ \gamma = (b-1) \cdot \alpha^{-1} \mod p \end{cases}$$

It is clear that such a β only exists if and only if 2 has an inverse modulo p. This latter argument forces p to be odd. In that case, fixing α to 1, the $(2, p)$-encoding

$$\mathcal{E}^{(neg)} = \begin{cases} \mathbb{Z}_2 \mapsto \mathbb{Z}_p \\ 0 \mapsto \{[2^{-1} - 1]_p\} \\ 1 \mapsto \{[1 - 2^{-1}]_p\} \end{cases}$$

is supposed to be what we are looking for.

As this encoding is negacyclic, we can perform a classic PBS. Then, the linear transformation $x \mapsto x + 2^{-1} \mod p$ produces a ciphertext under the right p-encoding. Trivially, adding a constant to a TFHE ciphertext does not increase its noise. The same reasoning can be followed for the other bits.

Finally, summing the produced ciphertexts gives an encryption of m under $\mathcal{E}_p$. The whole procedure is only possible if p is odd. □

As seen in the first contribution presented in Section 4.2, the most efficient way to slice 8-bit inputs for a TBM evaluation of the Sbox is to decompose the inputs into two 4-bit chunks. Given that our Recomposer block requires an odd plaintext modulus, as established in Property 1, we selected the smallest odd modulus capable of representing 4-bit values: $p = 17$.

4.3.3 Experimental Results

In this section, we compare our new framework to several state-of-the-art homomorphic AES executions, including the ones performed with the two building blocks of our new design. We particularly emphasize that *all implementations have been tested on the same machine*, a 12th Gen Intel(R) Core(TM) i7-12700H CPU laptop with 64 Gib total system memory with an Ubuntu 22.04.2 LTS operating system. All execution timings can be found in Table 4.7. Parameter sets used to obtain these results are presented in Table 4.6. Depending on the framework, we had to use different implementations of TFHE as available in the `TFHElib`[5], `tfhe-rs`[6] or `TFHEpp`[7] libraries.

[5] `https://tfhe.github.io/tfhe/`
[6] `https://github.com/zama-ai/tfhe-rs`
[7] `https://github.com/virtualsecureplatform/TFHEpp`

Table 4.6: Parameters sets used for our homomorphic AES evaluation, with $\lambda \approx 128$ bits as the security parameter. P_{err} denotes the probability of bootstrapping failure.

P_{err}	n	N	k	l	B_g	B_{KS}	t	σ_{TLWE}	σ_{TRLWE}
2^{-40}	754	1024	1	2	2^{23}	2^4	3	$2^{46.4}$	$2^{16.7}$
2^{-128}	900	4096	1	2	2^{15}	2^3	6	$2^{44.5}$	2^2

Table 4.7: Comparison of our method with different state-of-the art approaches *on a single core*. The only execution timing that was not obtained on our machine is marked with a *, i.e. for Thunderbird, making the comparison more in favor of that method. See the discussion at the end of Section 4.3.3.

Year	**Reference**	**Framework**	**Library**	**Timings (s)**
2023	Section 4.2	Tree-Based Method (TBM)	`TFHElib`	270
	[BPR24]	p-encoding method	`tfhe-rs`	90
	[WWL+23]	Fregata	`TFHEpp`	87
2024	[WLW+24]	Thunderbird	`TFHEpp`	46*
2025	this work	Hippogryph	`tfhe-rs`	32

The implementation of Hippogryph and the unified software test bench for AES execution over TFHE, which we used to obtain the consistent same-machine experimental results presented in this paper, are available as open-source Git repositories[8].

Note

The proposed repository aims to be an open-source unified software test bench for AES execution over TFHE, designed to obtain consistent same-machine experimental results, as given in our contribution. It is a valuable resource for enabling fair comparisons in further works on AES in the community.
Since the repository opened, I have received requests from a few other teams to add their new implementation, and I regularly receive emails showing the community's interest in the subject.

State-Of-The-Art Homomorphic AES Executions

The approaches introduced in Section 4.2 and [BPR24], which form the foundation of this contribution, are discussed in the previous sections. Additionally, we briefly describe here the two other main state-of-the-art methods for homomorphic AES executions: Fregata [WWL+23] and Thunderbird [WLW+24].

[8] `https://github.com/CryptoExperts/Hippogryph` and `https://github.com/daphnetrm/Benchmark-of-AES-Evaluation-with-TFHE`.

Fregata [WWL+23]

In this work, the authors present a novel evaluation framework especially designed for faster AES homomorphic evaluation. Instead of relying on functional bootstrapping, they decided to use CMUX gate as the building block of their framework. They also propose a new technique for an efficient Sbox evaluation relying on mixed packing (which combines different ways of organizing encrypted data within polynomials to balance parallelism and flexibility). But one of the major contributions of this work is the optimization of TFHE's circuit bootstrapping. Indeed, they propose to use PBSManyLUT [CGGI19] in the first step of circuit bootstrapping. As their framework relies on the use of TFHE in LHE mode, this optimization of circuit bootstrapping is the key to an efficient homomorphic AES evaluation. Fregata is designed to perform one round of AES without any bootstrapping and to use circuit bootstrapping on each bit of the state matrix after a full round evaluation. Running these circuit bootstrappings then becomes the most time-consuming part. Finally, they also leverage on encoding messages in $\{0, 1\}$ as $\{0, \frac{1}{2}\}$ over the torus to transform XOR operations into simple TLWE additions (which is the same idea as using our $(2, 2)$-encoding in the linear parts).

Their results, obtained with the `TFHEpp` library [Mat20], reached an AES homomorphic evaluation latency of 86 seconds on a 12th Gen Intel(R) Core(TM) i5-12500× 12 with 15.3 GB RAM machine. When running the Fregata implementation[9] on our machine, we also obtained a latency of about 87 seconds.

Thunderbird [WLW+24]

The work presented in the Thunderbird paper leverages on the Fregata framework to produce an even faster AES homomorphic evaluation, still using the `TFHEpp` library. Specifically, Thunderbird combines the gate bootstrapping and leveled evaluation modes of TFHE to cater to various function types within symmetric encryption algorithms. More specifically, their approach builds upon the Fregata framework with additional optimizations:

- The circuit bootstrapping proposed in Fregata is optimized by replacing the second step (namely a private keyswitch) by a public keyswitch followed by a new operation called `EvalSquareMult`.
- Instead of following a standard AES implementation, the authors introduce a LUT-based AES implementation that merges `SubBytes`, `ShiftRows`, and `MixColumns` operations into 8-to-32-bit tables (which results in a smaller number of XOR operations when running the overall AES).

Moreover, as in [WWL+23], they rely on encoding the messages in $\{0, 1\}$ as $\{0, \frac{1}{2}\}$ over the Torus. With such encoding, XOR operation can be performed for free. They

[9] https://github.com/WeiBenqiang/Fregata

call this optimization `FreeXOR`. They also propose another technique to evaluate XOR, namely `HomoXOR` relying on gate bootstrapping with messages encoded in $\{\frac{-1}{8}, \frac{1}{8}\}$ over the Torus. The evaluation of AES with this technique is less efficient than with `FreeXOR`. The tests were run on an Intel(R) Core(TM) i5-11500 CPU @ 2.70GHz machine with 32 GB of RAM, and they obtained an average execution latency of 46 seconds.

It is important to note that the implementation of the Thunderbird framework is not publicly available. To obtain a fair comparison with our work, we tried to reproduce their results by implementing the framework ourselves, starting from Fregata on which Thunderbird is based. We successfully implemented all of Thunderbird building blocks but one–the improved circuit-bootstrapping–which was producing decryption errors despite our best efforts to faithfully follow [WLW+24]. As a consequence, for this specific building block, we relied on the theoretical speedup reported in the Thunderbird paper, which results in a slightly unfair comparison to our approach. In summary, *we measured an AES execution time of 60 secs with our implementation, but used instead the 46 secs reported in [WLW+24] for comparison.*

Results

To measure the performances of our method, we implemented it using a fork of `tfhe-rs`[10] that supports odd moduli. The results were then compared against the current state-of-the-art frameworks.

For a fair comparison, all implementations were tested on the same machine, *using a single core.* As shown in Table 4.7, our novel framework achieves the lowest latency when evaluating the AES as the evaluation of the algorithm only takes about 30 seconds. Hence, Hippogryph is between 1.44 and 1.87 times faster than the best-in-class Thunderbird approach (depending, as discussed above, whether we respectively consider the 46 secs timing given in the Thunderbird paper or a timing of 60 secs as measured with our implementation). Moreover, when enabling several cores on our 12th Gen Intel(R) Core(TM) i7-12700H CPU laptop, we can reach an execution time that is smaller than 5 seconds, using only 6 cores, and further reduce this timing to 1.1 seconds by using 32 cores on a more powerful machine as discussed below.

A Few Words About Parallelisation. The purpose of transciphering is to minimize the bandwidth overhead when transferring large amounts of data. Given that servers typically have more computational resources than clients, they can effectively leverage multiple cores to parallelize computations and enhance execution times. In this context, AES offers inherent parallelizability, as most operations within each encryption round can be executed simultaneously on each byte of the state matrix—except for the ShiftRows step.

[10] `https://github.com/zama-ai/tfhe-rs.`

It is worth noting that in practical transciphering applications, one could process multiple AES blocks in parallel to achieve better amortized performance. However, the parallelization we apply here focuses on accelerating computations within a single AES block, rather than processing multiple blocks independently.

To implement this approach, we used Rust's `rayon` crate. Our tests were conducted on two distinct machines to assess performance across different setups:

- Laptop (12th Gen Intel(R) Core(TM) i7-12700H CPU, 6 cores): We parallelized all round functions except `ShiftRows`, which mainly involves reordering ciphertexts within the state matrix. This setup already provided a significant speedup compared to single-core execution: 4.6 seconds for a failure probability of 2^{-40}.

- Server (AMD Ryzen Threadripper PRO 7995WX, 96 cores): We leveraged 32 cores to process the 16 bytes of the AES internal state in parallel, each using one thread for each of the 2 independents TBM in the `SubBytes` step. This setup brought us remarkably close to breaking the 1-second barrier, with an execution time of just 1.1 seconds.

Detailed execution timings illustrating these improvements can be found in Table 4.8.

Memory-wise, our implementation manipulates at most 128 TFHE ciphertexts which account for the internal state and 128×11 other ciphertexts for the round keys. Each TFHE ciphertext accounts for around 48 kbits (755*64 bits). Hence, the total memory consumption of our implementation is approximately 74 Mbits when encrypting a single block. When encrypting multiple blocks in parallel, the ciphertexts for the round keys are shared across all blocks, reducing the per-block overhead.

What About Recent CPA$^{\mathbf{D}}$ Attacks? To obtain a fair comparison, we use parameters equivalent to those used in the state-of-the-art, which typically achieve an error probability of about 2^{-40}. But to take into account recent attacks in the CPA^D model [LM21] on several FHEs (including TFHE) [CSBB24, CCP+24], we also give execution times of our approach with an example parameters set achieving an error probability of 2^{-128} (Table 4.6). When running with such parameters, an AES evaluation takes about 463 seconds on our machine, still using a single core (see also Table 4.8). Although more optimal parameters may be found, this timing also illustrates that achieving CPA^D security may have a significant cost on FHE performances. At this point, we leave that cost mitigation as a future work.

Table 4.8: Different evaluation timings of Hippogryph for different setups.

Machine	# cores	P_{err}	Timings (s)
laptop	1	2^{-40}	32
laptop	1	2^{-128}	463
laptop	6	2^{-40}	4.6
server	32	2^{-40}	1.1

Chapter 5

`Mad Panthera`: Designing a General-Purpose 8-bit TFHE Processor Abstraction



Introduction

We aim to implement a universal set of instructions for an 8-bit software processor working on encrypted data. Such a set of instructions will allow any non-expert developer to implement a homomorphic version of any algorithm with 8-bit entries and outputs. To do so, we propose to rely on LUT dereferencing to implement each instruction. Indeed, as already seen, each 8-bit-to-8-bit or 2×8-bit-to-8-bit instruction can be implemented using several LUT dereferences. In this contribution, we show that using only the TBM and the MVB optimization (which are both summarized in the blue box above), we can evaluate any multivariate function. We thus propose a set of some fifty instructions that form a virtual ISA that is competitive with recent state-of-the-art.

This work has been the object of a paper which has been accepted to CHES 2025: **Designing a General-Purpose 8-bit (T)FHE Processor Abstraction**, D. Trama, P.-E. Clet, A. Boudguiga, R. Sirdey, N. Ye.

5.1 Related Work

5.1.1 FHE Virtual Processors

To the best of our knowledge, previous attempts at building FHE-based virtual processors are pretty scarce and, for the most part, date back to the first few years after Gentry's breakthrough. By *virtual processor* or *processor abstraction*, we mean a software entity that works over FHE-encrypted data and presents itself to the programmer by means of a conventional-looking assembly instruction set. Perhaps the first attempt is that of Brenner et al. [BPS12][1], which was based on the Smart-Vercauteren scheme [SV10]. This work proposes an abstract processor that executes an *encrypted program* (over encrypted data). The processor has a minimal instruction set containing only bitwise logical operations as well as load/store (with encrypted addresses, hence with an access complexity linear in memory size) and three branching instructions. Each (encrypted) instruction is fetched from the encrypted memory and then homomorphically interpreted (at an extra cost equivalent to explicitly running all instructions in the set). Being more than twelve years old, from an experimental point of view, this latter work is obsolete. Still, our approach departs significantly from it because we run *public* programs over encrypted data, i.e., the stream of instructions is *not* encrypted. The main consequence is that we restore constant-time memory access (because all the addresses are public), but cannot perform any branching (conditioned on encrypted values). Branching then has to be emulated using explicit conditional assignments at an extra cost equivalent to that of explicitly running all branches. In that sense, our programming model

[1] `github.com/hcrypt-project`.

somewhat resembles the "constant time programming" model often used in embedded computing [ABB+16]. Also, we can then afford to have a much more complete instruction set, which is tailored to the capabilities of our modern functional bootstrapping toolbox. Another attempt is that of [FSF+13], which considers a richer set of operators (rather than explicit instructions) and is boolean-circuit oriented. From an experimental viewpoint, this latter work is also too old not to be obsolete. Other works include experiments at building a one-instruction set processor abstraction working over FHE-encrypted data [TM14, TM13, CS19], an approach which also achieves Turing completeness but leads to even worse blow-ups in the number of instructions than the boolean circuit one. A more recent attempt at supporting a subset of the ARM (v8) instruction set over TFHE is given in [GN]. This approach has two main drawbacks. First, it uses TFHE only in gate-bootstrapping mode and, as such, does not work over a larger plaintext space as we do with functional bootstrapping techniques. Second, it handles conditional branching in a client-aided fashion with the consequence of granting the FHE processor access to a decryption oracle. This is likely to induce vulnerabilities in realistic deployment scenarios since TFHE is trivially insecure against a CCA(1) adversary. By opposition, we "handle" branching in a non-interactive way via conditional assignment instructions (but at the extra cost of running all branches). Lastly, a few works [IMP18, CGRS14] propose to extend the instruction set of existing processors with a small set of additional instructions for driving FPGA-implementations of FHE operations (with [IMP18] also handling branching in a client-aided fashion). On top of the above, there presently are many works on hardware implementation of FHE building blocks without any focus on instruction sets.

5.1.2 FHE Compiler Approaches

Another relevant line of work to compare to is the FHE compiler approach. Indeed, although they do not explicitly attempt to build FHE-based virtual processors with an explicit instruction set, they all come with a runtime environment capable of executing the FHE calculations derived from their input programs. Such approaches, include Cingulata [CDS15a], E3 [CMG+18], Concrete [Zam22] and the so-called Google Transpiler [GSPH+21]. Additionally, a more instruction-oriented approach, Juliet [GMT24], has been proposed recently. All these approaches rely on TFHE. However, contrary to ours, they do not leverage on multivariate functional bootstrapping techniques over multi-bit inputs and essentially represent and execute an FHE calculation as a Boolean circuit to be run over FHE binary-payload ciphertexts.

Let us emphasize that the ISA-based approach we develop in this paper is qualitatively different from the (Boolean) circuit generation approach used by the aforementioned FHE compilers. Indeed, while the compilers directly turn high-level programs into circuits (with some optimizations), our ISA-oriented approach provides an optimized instruction set and relies on the programmer to use it as optimally

as possible. In some sense, the two kinds of approaches thus rely on different assumptions depending on whether the human assembly programmer can do better than the compiler's circuit optimization heuristics. Of course, this does not mean that the two kinds of approaches cannot be meaningfully compared and we do so extensively in Section 5.7. This latter section also provides additional details on the functioning and philosophy of each of the aforementioned FHE compiler approaches.

5.2 Optimal Basis Selection for LUT Evaluation

5.2.1 Decomposition Basis Choice

As seen in the first contribution of this thesis (see Section 4.2), working directly over 8-bit messages requires very large TFHE parameters, leading to a very slow bootstrapping (≈1.5 secs for a single bootstrapping). Consequently, we need to decompose our 8-bit data into a smaller basis. For 8-bit plaintexts, several decompositions are available: we can decompose a message into four 2-bit digits, into three 3-bit digits (with the most significant one only taking values in $\{0, \cdots, 3\}$), or into two *nibbles* (the cute name for 4-bit digits). We remind the reader that the smaller the decomposition basis, the smaller the parameters, and thus the faster the bootstrapping evaluation. However, the smaller the decomposition basis, the greater the number of digits, and so the greater is the number of bootstrapping to be performed. A tradeoff must, again, be achieved between the number of bootstrapping needed and the parameters set corresponding to the decomposition basis. To find this ideal decomposition basis, we evaluated several bootstrapping operators that we detail here.

We refer to the evaluation of a LUT using the TBM (combined with MVB) on an 8-bit payload message decomposed into d digits in basis B as LUTeval, as opposed to SimpleBoot, which is the usual bootstrapping operation taking only one encrypted input. LUTeval takes as inputs d encrypted chunks and returns *only one* encrypted digit. To obtain the d digits forming the 8-bit payload result of the evaluation of a LUT on an 8-bit payload message, LUTeval must thus be performed d times on the *same inputs*. That is why we further introduce MVLUTeval. With LUTeval, the MVB optimization allows factorization of the BlindRotate operations of the first layer of the tree. With MVLUTeval, we evaluate several trees on the same inputs. Especially, each first layer of the trees is evaluated on the same encrypted input, but on different polynomials P_{f_i} corresponding to the different LUTs. As the polynomial $v_0 = \frac{1}{2} \cdot (1 + X + \cdots + X^{N-1})$ is a common factor of all the P_{f_i}, we can only execute the BlindRotate on v_0 with the first encrypted chunk as selector, and then pursue independently each tree with the other selectors.

Table 5.1 presents the timings obtained for these three bootstrapping operators (SimpleBoot, LUTeval, and MVLUTeval) depending on the decomposition basis of the plaintexts. We can see that, when run under TFHElib [CGGI16] with the parame-

ters from Table 2.1, the SimpleBoot operator is the most efficient with decomposition basis $B = 4$. However, in the sequel, the most used operators are LUTeval and different flavors of MVLUTeval. The best timings for these operations are obtained with decomposition basis $B = 16$ (which is no surprise as we had the same conclusion in the first contribution).

Table 5.1: Execution times of SimpleBoot, LUTeval and MVLUTeval depending on the plaintext decomposition basis. MVLUTeval* stands for an evaluation of two different LUTs, and MVLUTeval$^\diamond$ for four different LUTs. As the number of output digits is equal to the number of LUT evaluations, MVLUTeval* outputs two new basis B ciphertexts and MVLUTeval$^\diamond$ outputs four such ciphertexts. For instance, for basis 16, MVLUTeval* (respectively MVLUTeval$^\diamond$) thus output 8 bits (respectively 16 bits), or equivalently a "digit" in basis 256 (respectively 256×256).

Decomposition basis		LUT size	Number of output digits	Corresponding output basis	N_{BR}	N_{KS}	Timings (secs)
16	SimpleBoot	16	1	16	1	0	0.029
	LUTeval	256	1	16	2	1	**0.13**
	MVLUTeval*	256	2	256	3	2	**0.23**
	MVLUTeval$^\diamond$	256	4	256×256	5	4	**0.43**
8	SimpleBoot	8	1	8	1	0	0.015
	LUTeval	256	1	8	10	9	0.47
	MVLUTeval*	256	2	64	19	18	0.93
	MVLUTeval$^\diamond$	256	4	256×8	37	36	1.83
4	SimpleBoot	4	1	4	1	0	**0.007**
	LUTeval	256	1	4	22	21	0.5
	MVLUTeval*	256	2	16	43	42	0.993
	MVLUTeval$^\diamond$	256	4	256	85	84	1.98

Remark 1. Note that performing two LUTeval in basis 16 (and thus obtaining two 4-bit payload ciphertexts) costs 0.26 seconds. We see that MVLUTeval*, which does the same thing with one less BlindRotate (as we factorized the two first-layer BlindRotate operations), takes only 0.23 seconds, showing the impact of further MVB optimizations. However, MVLUTeval is less interesting in other decomposition bases, where the initial number of BlindRotate is larger: MVLUTeval* saves one BlindRotate, i.e., $\frac{1}{4}$ with decomposition basis $B = 16$, but only $\frac{1}{20}$ with decomposition basis $B = 8$ and $\frac{1}{44}$ with decomposition basis $B = 4$. †

Remark 2. Another important detail to take into account is the amount of information that is produced depending on the operator that is being performed and the

decomposition basis. Indeed, MVLUTeval* performs the evaluation of two LUTs on the same inputs, and thus returns two encrypted digits in the corresponding decomposition basis B. For $B = 16$, we obtain two nibble encryptions, that is to say, the equivalent of an 8-bit payload encrypted message. When performing MVLUTeval* with decomposition basis $B = 4$, we obtain two 2-bit payload encrypted digits, which is only the equivalent of an encrypted nibble. So MVLUTeval* is not only faster with basis $B = 16$, but it also outputs more data! The decomposition basis $B = 16$ is thus twice as advantageous as any other decomposition basis. †

LUT Dereferencing Operators

Now that we have settled on the optimal decomposition basis for our 8-bit plaintext inputs, we can instantiate our three LUT dereferencing tools SimpleBoot, LUTeval, and MVLUTeval.

- SimpleBoot is the basic TFHE bootstrapping with a 4-bit ciphertext as an encrypted index. Let tab_16 be a cleartext LUT with 16 entries in $\mathbb{Z}_{16}$, given a ciphertext $c \in [\![m]\!]$, SimpleBoot$(c;$ tab_16$)$ returns $c' \in [\![\mathsf{tab_16}[m]]\!]$.

- LUTeval allows us to evaluate a 16×16 LUT on two nibble encryptions $c_0 \in [\![m_0]\!]$ and $c_1 \in [\![m_1]\!]$, with $m_0, m_1 \in \mathcal{M} = \{0, 1, \cdots, 15\}$. We denote it LUTeval$(c_0, c_1;$ tab$)$, with tab the 16×16 table that will be used to instantiate the 16 test-vectors polynomials required for the first layer of the TBM. LUTeval$(c_0, c_1;$ tab$)$ returns a 4-bit payload ciphertext $c' \in [\![\mathsf{tab}\,[16m_0 + m_1]]\!]$.

- Lastly, let us assume that we want to evaluate k LUTeval on the following pairs of ciphertexts $((c_\alpha, c_1),\cdots,(c_\alpha, c_k))$ using the tables $(\mathsf{tab}_1, \cdots, \mathsf{tab}_k)$. Each pair (c_α, c_j) is an encryption of $T_j = 16m_\alpha + m_j$, where $m_\alpha, m_j \in \mathbb{Z}_{16}$. As c_α is a common input for the k LUTeval, we can rely on only one MVB to compute the first level of the k trees simultaneously instead of running k separate MVBs for each LUTeval$(c_\alpha, c_j; \mathsf{tab}_j)$, where $j \in \{1, \cdots, k\}$. The second level of each tree is then computed separately on $(c_1, \cdots, c_k)$. As such, we end up running $k + 1$ BlindRotate ($\mathsf{N_{BR}} = k + 1$) instead of $2k$ ones for computing k LUTeval, with k Keyswitch ($\mathsf{N_{KS}} = k$) and $16k$ plaintext/ciphertext multiplications.
  From now on, we define MVLUTeval$(c_\alpha;\ c_1, \cdots, c_k; \mathsf{tab}_1, \cdots, \mathsf{tab}_k)$ as the operation that computes with a unique MVB the first level of the trees associated to LUTeval$(c_\alpha, c_j;\ \mathsf{tab}_j)$, and outputs k encrypted 4-bit digits $c'_j \in [\![\mathsf{tab}_j\,[16m_\alpha + m_j]]\!]$ $\forall j \in \{1, \cdots, k\}$. MVLUTeval can be further optimized when provided with the same table tab_j twice (or more) by computing less Keyswitch.

In summary, our toolbox mainly consists of LUTeval $: \mathcal{C}^2 \times \mathcal{L} \longrightarrow \mathcal{C}$ ($\mathcal{L}$ being the set of all 256 4-bit entries tables) which, given $(c_0, c_1) \in [\![m_0]\!] \times [\![m_1]\!]$, is such that

$$\mathsf{LUTeval}(c_0, c_1; \mathsf{tab}) \in [\![\mathsf{tab}[16m_0 + m_1]]\!].$$

and $\mathsf{MVLUTeval} : \mathcal{C}^{(k+1)} \times \mathcal{L}^k \longrightarrow \mathcal{C}^k$, its optimization for running several $\mathsf{LUTeval}$ with one common input $c_\alpha \in [\![m_\alpha]\!]$ and k other inputs $c_j \in [\![m_j]\!], \forall j \in \{1, \cdots, k\}$, which satisfies

$$\mathsf{MVLUTeval}(c_\alpha; c_1, \cdots, c_k; \mathsf{tab}_1, \cdots, \mathsf{tab}_k) \in [\![\mathsf{tab}_1[16m_\alpha + m_1]]\!] \times \cdots \times [\![\mathsf{tab}_k[16m_\alpha + m_k]]\!].$$

5.3 An FHE-Optimized Instruction Set

5.3.1 Instruction Set Overview

In this contribution, we propose an exhaustive set of some fifty 8-bit instructions that manipulate (T)FHE-encrypted data. Some provided instructions are relatively standard, but others are more specific and included because a smaller number of homomorphic operations are required to implement them. As an example of this, for additions, we provide three instructions: ADD, ADDi, and ADDZ. The ADD instruction takes *two* input ciphertexts (with an 8-bit cleartext payload) and produces a third one whose decryption is expected to be the sum of the two input ciphertexts' plaintexts. The ADDi instruction takes *one* input ciphertext and an immediate (public) value V. This instruction can then be seen as a family of *univariate* instructions ADDi_V (for $V \in \{0, \cdots, 255\}$) and, as we shall later see, can be much faster than the previous general purpose ADD. Lastly, the ADDZ instruction also takes *two* input ciphertexts and performs an addition *under the assumption that at least one of the two input ciphertexts is an encryption of 0*. This case occurs recurrently in several algorithmic patterns, particularly when values must be selected based on the results of conditions over encrypted data. Some examples are the computation of conditional assignment instruction CSEL (Section 5.3.4), bubble sorting (Section 5.6), array dereferencing and assignment (Section 5.6). For more details about ADDZ, see Section B.2.2. As a result, this instruction also executes much faster than the general-purpose ADD instruction. This first example illustrates our design mindset, according to which we have proposed standard general-purpose instructions for all usual operations found in a typical ISA, as well as additional variants providing better FHE evaluation when some (frequently occurring) assumptions are met.

In summary, we provide the following categories of instructions:

- **Bitwise/arithmetic** instructions (addition, multiplication, division, modulo, shift, rotation, etc.), each coming in different flavors as discussed just above. These instructions' names are relatively conventional.
- **Test** instructions for testing equality and performing comparisons over encrypted data. These instructions also come with different flavors and are expected to return encryptions of either 0 or 1 .
- **Conditional assignment** instructions (CDUP, NCDUP and CSEL, the latter being the only trivariate instruction in the set). These instructions provide the

building blocks to emulate if-then-else or do-while statements with encrypted data-dependent conditions.

- **Advanced** instructions: support for multiplication with 16-bit results (i.e., computation of the most significant *byte* of the product of two bytes), support for fixed-point arithmetic (including decimal division), min/max operators, absolute value, to name a few.
- **User defined** *univariate* instructions: we further provide an XOP instruction which the programmer may arbitrarily configure.

For readability's sake, the following sections are intended only to discuss the key difficulties we had to overcome and the optimization techniques we had to consider to implement the complete set of instructions. Full details on each instruction are provided in Appendix B.

5.3.2 Notations for Homomorphic Operator Specifications

In this work, we manipulate 8-bit plaintexts broken down into two 4-bit digits. Thus, to encrypt an 8-bit plaintext M decomposed into two 4-bit digits m_0 and m_1 such that $M = 16m_0 + m_1$, we encrypt m_0 and m_1 separately under the same scheme $\mathcal{E}$ to obtain $C = (c_0, c_1) \in [\![m_0]\!] \times [\![m_1]\!]$. We consistently denote 8-bit plaintexts $M \in \mathcal{M}^2$ and their corresponding ciphertexts $C \in \mathcal{C}^2$ with uppercase letters. Conversely, 4-bit plaintexts and their encryptions are denoted with lowercase letters. For instance, for $h, l \in \mathcal{M}$, $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!] \subset \mathcal{C}^2$ denotes an encryption of the 8-bit cleartext value $(h, l) \in \mathcal{M}^2$ which encodes the 8-bit message $M = 16h + l$. We call h the *most significant nibble* of M. Similarly, l is the *least significant nibble* of M. We denote these parts MSN and LSN. With a slight abuse of notation, as already done above, we will use $T = (u, v)$ and $T = 16u + v$ interchangeably. Sometimes, a ciphertext C may have no MSN and is denoted by $(\perp, c_1)$. This, for example, occurs for outputs of test instructions, which are encrypted booleans (in that case, it can further be assumed that $c_1 \in [\![0]\!] \cup [\![1]\!]$). Some instructions also result in a cleartext 0 value in the MSN or LSN of a given ciphertext, e.g., an unsigned right (respectively left) shift of $C = (c_0, c_1)$ gives ciphertext $(0, c_0)$ (respectively $(c_1, 0)$). We can use this to perform cleartext/ciphertext operations on the fly.

As a "Hello world!" example of how we later use these notations to specify our operators and instructions, let us consider the AND instruction which, given $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$, is defined as

$$\mathsf{Eval}(\mathsf{AND}; C, C') = \bar{C} = (\bar{c}_0, \bar{c}_1) \in [\![h \& h']\!] \times [\![l \& l']\!]$$

To actually *implement* the above, we then proceed by evaluating

$$\bar{c}_0 = \mathsf{LUTeval}(c_0, c'_0; \mathsf{tab_and}) \text{ and } \bar{c}_1 = \mathsf{LUTeval}(c_1, c'_1; \mathsf{tab_and})$$

where $\mathsf{tab_and}$ is a table with 256 4-bit entries such that $\mathsf{tab_and}[16i+j] = i\&j$ and where $\mathsf{LUTeval} : \mathcal{L} \times \mathcal{C}^2 \longrightarrow \mathcal{C}$ is the functional bootstrapping operator instantiated in Section 5.2.

5.3.3 Implementing Univariate Instructions

Univariate instructions only take *one* input ciphertext (with an 8-bit cleartext payload). These can correspond to univariate operators, such as the absolute value (ABS) or the negation (NEG) of a signed 8-bit value, the bitwise inversion operator (INV), etc. They can also correspond to cleartext-ciphertext operations such as the addition of a (public) immediate value (ADDi), left shift, or rotation by a (public) number of positions (SHLi or ROLi), etc. *With respect to our 8-bit plaintext domain*, all these operations can be implemented following a generic method. From the initial instruction inst operating on 8-bit inputs, we create the corresponding LUT $\mathsf{tab_inst}$ such that for $i \in \{0, \cdots, 255\}$, $\mathsf{tab_inst}[i] = \mathsf{inst}(i)$. Note that for instructions implementing cleartext-ciphertext operations, there is one such table $\mathsf{tab_inst}_V$ for each of the 256 possible plaintext inputs, V, with the proper table selected at runtime (and, even possibly generated on the fly). Once the 8-bit-to-8-bit $\mathsf{tab_inst}$ LUT is generated, we compute two new tables that separately handle the results of the instruction on the MSN and on the LSN parts of the message. More formally, we create $\mathsf{tab_inst_msn}$ and $\mathsf{tab_inst_lsn}$ such that for $i \in \{0, \cdots, 255\}$, $\mathsf{tab_inst_msn}[i] = \lfloor \frac{\mathsf{tab_inst}[i]}{16} \rfloor$ and $\mathsf{tab_inst_lsn}[i] = \mathsf{tab_inst}[i] \pmod{16}$. Then, to perform the instruction inst over $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ we simply have to evaluate

$$(\bar{c}_0, \bar{c}_1) = \mathsf{MVLUTeval}(c_0; c_1, c_1; \mathsf{tab_inst_msn}, \mathsf{tab_inst_lsn}) \tag{5.1}$$

To illustrate that this pattern allows implementing arbitrary complex univariate instructions, we can consider the case of the divide-by-V ($V \in \mathbb{Z}_{256}$) operation[2] which induces the instructions: DIVi (quotient of the euclidean division by V with $\mathsf{tab_divi}_V[i] = \lfloor i/V \rfloor$), MODi (remainder of the euclidean division by V with $\mathsf{tab_modi}_V[i] = i \mod V$).

Univariate test instructions are handled slightly differently in the sense that, with respect to the plaintext domain, they output only encryptions of boolean 1-bit values (still contained in a single 4-bit digit). As such, only an evaluation of $\mathsf{LUTeval}$ is needed to perform them. For example, the $\mathsf{LT}(C, V)$ instruction, which outputs ciphertext $\bar{C} = (\perp, c_1) \in \{\perp\} \times [\![b]\!]$ from ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ with $b = 1$ if $16h + l < V$ and $b = 0$ otherwise, is performed by evaluating only

$$\bar{c}_1 = \mathsf{LUTeval}(c_0, c_1; \mathsf{tab_lt}_V). \tag{5.2}$$

[2]Division, even by a cleartext value, is a good example of an operation which is notoriously difficult to perform efficiently over FHE (even when one of the two operands is cleartext). Here, with our techniques, division by a cleartext value does not cost much more than a mere addition...

Remark 3. Note that some univariate instructions can be implemented more efficiently than by (5.1). For example, for an addition by V (ADDi) we can proceed as follow:

$$\begin{aligned}\bar{c}_1 &= \mathsf{SimpleBoot}(c_1, \mathsf{tab_add4}_{V\&15})\\ \bar{c}_0 &= \mathsf{LUTeval}(\bar{c}_0, c_1, \mathsf{tab_fin4})\end{aligned}$$

with $\mathsf{tab_add4}_v[i] = (i+v) \mod 16$, $\mathsf{tab_fin4}_V[16i+j] = \mathsf{tab_add4}[16\times\mathsf{tab_add4}_{\lfloor V/16\rfloor}[i]+\mathsf{tab_car4}_{V\&15}[j]]$, $\mathsf{tab_car4}_v[i] = \lfloor (i+v)/16 \rfloor$ and $\mathsf{tab_add4}[16i+j] = (i+j) \mod 16$. Following the notations in Sect. 5.3.2, this can further be optimized as

$$(\bar{c}_0, \bar{c}_1) = \mathsf{MVLUTeval}(c_1; \perp, c_0; \mathsf{tab_add4}_{V\&15}, \mathsf{tab_fin4})$$

which has the effect of factoring an additional BlindRotate (resulting in 2 BlindRotate vs 3 if (5.1) is used). In a similar spirit, bitwise instructions, e.g. ANDi, can simply be implemented with two calls to SimpleBoot leading, again, to 2 BlindRotate vs 3 when (5.1) is used. We have considered such optimizations on a case-by-case basis, resorting to (5.1) only when we found no better options. †

Lastly, we provide an additional univariate XOP instruction taking a user-defined 256×8 bits table rather than an immediate value V as input. For example, this instruction can be used to evaluate the $GF(256)$ multiplications and the Sbox of the AES algorithm (that is to say, exactly what we did in our first contribution!). A variant of this latter instruction, XOPN(ibble) also takes a user-defined 256×4 bits table as input to evaluate custom conditions following (5.2). Table 5.2 provides a synthetic (yet exhaustive) list of the univariate instructions we have implemented.

5.3.4 Implementing Bivariate Instructions

Bivariate Instructions Basics

We now turn to bivariate instructions, which take *two* input ciphertexts (each with an 8-bit cleartext payload). These correspond to additions (ADD), left shift or rotation by an encrypted number of positions (SHL or ROL), etc. In Section 5.3.2, we have already seen how to perform bitwise instructions. As another example, let us consider instruction ADD which, given two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$, returns $\bar{C} = (\bar{c}_0, \bar{c}_1) \in [\![\bar{h}]\!] \times [\![\bar{l}]\!]$ such that $16\bar{h} + \bar{l} = (16h+l+16h'+l') \mod 256$. This instruction is performed with two calls to LUTeval and one call to MVLUTeval:

$$\begin{aligned}c_s &= \mathsf{LUTeval}(c_0, c'_0, \mathsf{tab_add})\\ (\bar{c}_1, c_c) &= \mathsf{MVLUTeval}(c_1; c'_1, c'_1; \mathsf{tab_add}, \mathsf{add_carry})\\ \bar{c}_0 &= \mathsf{LUTeval}(c_s, c_c, \mathsf{tab_add}).\end{aligned} \tag{5.3}$$

with $\mathsf{tab_add}[16i+j] = (i+j) \mod 16$, and $\mathsf{add_carry}[16i+j] = \lfloor \frac{i+j}{16} \rfloor$.

Arithmetic inst.	ADD(i) (addition of two bytes); SUB(i); MUL(i); MULM(i) (most sig. *byte* of the product of two bytes); DIV4(i) (division of an encrypted byte by an encrypted nibble); DIV(i) (division of an encrypted byte by another encrypted one); MOD4(i) (modulo of an encrypted byte by an encrypted nibble); MOD(i) (modulo of an encrypted byte by another encrypted byte)
Bitwise inst.	AND(i); OR(i); (U)SHL(i) (shift an encrypted byte (signed or unsigned) left by an encrypted 8-bit index), ROL(i) (rotate an encrypted byte left by an encrypted 8-bit index); (U)SHR(i); ROR(i)
Test inst.	EQ(i) (test if two ciphertexts encrypt the same byte); GT(i) (test if the first ciphertext encrypts an 8-bit value greater than the one encrypted by the second ciphertext); LT(i); GTE(i); LTE(i)
Other inst.	MIN(i) (minimum of two encrypted bytes); MAX(i); CDUP(i); NCDUP(i); CSEL (conditional selection); ABS (absolute value of an encrypted signed byte); NEG (returns the opposite of an encrypted signed byte); XOP; XOPN; DC (binary decomposition of a ciphertext); RC (recomposition of a ciphertext)

Table 5.2: List of our instructions. For each instruction denoted by INSTR(i), INSTR is the bivariate instruction taking two encrypted inputs, and INSTRi is the variant taking as inputs an encryption of a byte and a cleartext one. Instructions denoted by (U)INSTR(i) have an unsigned and a signed version.

ADDZ. In FHE computations, we often have to perform the addition of two encrypted values with an unknown one of them encrypting 0. This does not make any difference on a cleartext processor, and special instructions dedicated to such cases are usually not included. However, when working over encrypted data, the absence of carry propagation can make an important difference in execution timings. Using our set of bootstrapping tools, we can design a specific instruction ADDZ that would compute the addition of two 8-bit payload ciphertexts but without the call to MVLUTeval in Eq. (5.3) above. This instruction thus allows better efficiency when performing the addition of two ciphertexts under the assumption that at least one of them belongs to $[\![0]\!]$.

As for the univariate case (Section 5.3.3), bivariate test instructions output only a single (encrypted) nibble with a single-bit payload, i.e., ciphertexts of the form $(\perp, c_1)$ with $c_1 \in [\![0]\!] \cup [\![1]\!]$. The CDUP ("Conditional DUPplication") instruction is not a test instruction per se (as it does not return an encryption of a boolean value), but plays an important role in the evaluation of conditional assignment and, as such, is fundamental in the context of FHE calculations. Given an encrypted boolean $(\perp, c_1) \in [\![l]\!]$ and an input $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$, CDUP produces ciphertext $\bar{C} = (\bar{c}_0, \bar{c}_1) \in [\![\bar{h}]\!] \times [\![\bar{l}]\!]$ with $\bar{h} = h'$ and $\bar{l} = l'$ when $l = 1$ (i.e. when the input boolean is true) and $\bar{h} = \bar{l} = 0$ when $l = 0$ (we leave the instruction behavior unspecified when $l > 1$). Essentially, CDUP can be implemented by a single call to MVLUTeval:

$$\mathsf{CDUP}((\perp, c_1), C') = \mathsf{MVLUTeval}(c_1; c'_0, c'_1; \mathsf{tab_sel1}, \mathsf{tab_sel1}),$$

with $\mathsf{tab_sel1}[16i + j] = j$ if $i = 1$ and 0 otherwise.
Conversely, instruction NCDUP behaves similarly except that it outputs (encryption of) 0 when the input boolean is true. As such, it is implemented exactly as CDUP but using table $\mathsf{tab_sel0}[16i + j] = j$ if $i = 0$ and 0 otherwise.
Lastly, we provide a single *trivariate* instruction CSEL ("Conditionnal SELection") which, given an encrypted boolean $(\perp, c_1) \in [\![b]\!]$ ($b \in \{0, 1\}$) and *two* inputs $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$ and $C'' = (c''_0, c''_1) \in [\![h'']\!] \times [\![l'']\!]$, produces ciphertext $\bar{C} = (\bar{c}_0, \bar{c}_1) \in [\![\bar{h}]\!] \times [\![\bar{l}]\!]$ such that $\bar{h} = bh' + (1 - b)h''$ and $\bar{l} = bl' + (1 - b)l''$. Interestingly, even if it is a trivariate instruction, CSEL can be implemented rather efficiently by factoring 4 blindRotate in a single call to MVLUTeval:

CSEL

$$(\tilde{c}_0, \tilde{c}_1, \tilde{c}_2, \tilde{c}_3) = \mathsf{MVLUTeval}(c_1; c'_0, c'_1, c''_0, c''_1; \mathsf{tab_sel1}, \mathsf{tab_sel1}, \mathsf{tab_sel0}, \mathsf{tab_sel0})$$
$$(\bar{c}_0, \bar{c}_1) = \mathsf{ADDZ}((\tilde{c}_0, \tilde{c}_1), (\tilde{c}_2, \tilde{c}_3))$$

This gives us the conditional assignment instruction needed to emulate if-then-else constructs on our FHE processor abstraction. Note that we also provide instructions CDUPi, NCDUPi, and CSELi, which all take an encrypted boolean as input and either one or two cleartext values. We detail these instructions in Appendix B.

A Homomorphic Division Operator

We now illustrate how our approach can be used to lead to a division operator between two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$. For simplicity's sake, we consider the unsigned division. This operator returns $\bar{C} = (\bar{c_0}, \bar{c_1}) \in [\![\bar{h}]\!] \times [\![\bar{l}]\!]$ such that $16\bar{h} + \bar{l} = \lfloor (16h + l)/(16h' + l') \rfloor$.

Let $16\bar{h} + \bar{l} = Q = \sum_{i=0}^{7} q_i 2^i = \sum_{i=0}^{7} p_i$ with $q_i \in \{0, 1\}$. We then have

$$q_i = \begin{cases} 1 & \text{if } 2^i(16h' + l') \leq (16h + l) - \sum_{j=i+1}^{7} q_j \times (16h' + l')2^j, \\ 0 & \text{otherwise.} \end{cases}$$

Then, Q can be naively obtained using a carryless addition of the $p_i = q_i 2^i$'s. But we note that if $h' \neq 0$, the MSN of $\frac{16h+l}{16h'+l'}$ is always 0. If $h' = 0$, then the MSN of the quotient is given by $\frac{h}{l'}$. So instead of computing the q_i's for $i \in \{7, 6, 5, 4\}$, it is sufficient to compute

$$\bar{c_0} = \mathsf{LUTeval}(\mathsf{SimpleBoot}(c'_0, \mathsf{tab_is_zero}), \mathsf{LUTeval}(c_0, c'_1; \mathsf{tab_div}); \mathsf{tab_mul}).$$

Then, to compute the LSN of the quotient, we must follow the algorithm presented below, starting from C_t rather than C. Indeed, after computing $\bar{c_0}$, the value that will be compared to obtain the LSN of the quotient must be updated following some operations:

$$\begin{aligned} c_s &= \mathsf{LUTeval}(\bar{c_0}, c'_1; \mathsf{tab_mul}) \\ c_t &= \mathsf{LUTeval}(c_0, c_s, \mathsf{tab_sub}) \\ C_t &= (c_t, c_1). \end{aligned}$$

All that remains is to proceed with the description of the instruction DIV proposed in the following blue box.

Remark 4. Let us further consider the case where it is known that $h' = 0$ and let $q_0 = \lfloor 16h/l' \rfloor$, $q_1 = \lfloor l/l' \rfloor$, $r_0 = 16h \mod l'$ and $r_1 = l \mod l'$, then the division algorithm may be significantly simplified due to the following relation, which holds $\forall (h, l, l') \in \{0, \cdots, 15\}^3$,

$$\left\lfloor \frac{16h + l}{l'} \right\rfloor = \underbrace{\left\lfloor \frac{16h}{l'} \right\rfloor}_{q_0} + \underbrace{\left\lfloor \frac{l}{l'} \right\rfloor}_{q_1} + \underbrace{\left\lfloor \frac{16h_r}{l'} \right\rfloor}_{\epsilon_0} + \underbrace{\left\lfloor \frac{l_r}{l'} \right\rfloor}_{\epsilon_1}, \tag{5.4}$$

with $h_r = \left\lfloor \frac{r_0 + r_1}{16} \right\rfloor$ and $l_r = (r_0 + r_1) \mod 16$. This simplified division requires 20 blind rotations and 12 key switches versus 97 and 56 for a full-blown division. See Table 5.3. †

DIV

// the ciphertext $C_t = (c_t, c_1)$ has been previously computed following the method presented above
$C_t = (c_t, c_1)$
$c_q \in [\![0]\!]$
for $i = 3$ to 0
$C_m = (c_{m_0}, c_{m_1}) = \mathsf{SHLi}(C', i)$ // Shift Left by a cleartext index
$c_g = \mathsf{GTE}(C_t, C_m)$ // Greater Than or Equal
$c_b = \mathsf{LUTeval}(c'_0, c_g, \mathsf{tab_and_mulm_zero})$
$C_s = \mathsf{MVLUTeval}(c_b; c_{m_0}, c_{m_1}; \mathsf{tab_mul_lsn}, \mathsf{tab_mul_lsn})$
$C_t = \mathsf{SUB}(C_t, C_s)$
$c_q = \mathsf{LUTeval}(c_q, c_b, \mathsf{tab_add_}q_i)$

$\mathsf{tab_and_mulm_zero}$ is a 256-element table with $\mathsf{tab_and_mulm_zero}[16k + j] = (((k << i) >> 4) == 0)\&(j == 1)$, that we use to test if the overflow produced by the multiplication $2^i(16h' + l')$ is zero and if $2^i(16h' + l') \leq (16h + l) - \sum_{j=i+1}^{7} q_j 2^j$. Indeed, the condition for $q_i = 1$ is satisfied if and only if the multiplication does not produce any overflow. $\mathsf{tab_add_}q_i$ is a 256-element table such that for $k, j \in \{0, \cdots, 15\}$, $\mathsf{tab_add_}q_i[16k + j] = k + (j \neq 0) \cdot 2^i$ that we use to add the new q_i to c_q. Finally, note that for $i = 0$, SHLi does nothing.

5.4 Other Types of Ciphertexts

5.4.1 Working with Signed Inputs

In this section, we consider 8-bit messages decomposed into two 4-bit digits, but in signed representation using two's complement. This way, we can encrypt messages in $\mathcal{M}^- = [\![-128, 127]\!]$. This, of course, requires the user to know whether they are using signed or unsigned representation to be able to interpret the decrypted messages correctly. Note that the used TFHE parameters do not change, as it is only a matter of semantics. This way, new tables are required to perform additions, multiplications, shifts,... and new operations such as the negation NEG or the absolute value ABS. For many operators, these operations are very similar to their unsigned variants. For instance, the signed right logical shift (SHRi) is implemented by duplicating the sign bit instead of injecting zeroes on the left. Thus, the only differences are in the tables tab_inst_msn and tab_inst_lsn (recall Equation (5.1)) contents. Apart from this, we proceed similarly to the unsigned shift. For further details on how we implement SHRi and SHR, see Section B.2.8. More generally, appendix B presents all our instructions implementation details, including those instructions working over signed data.

5.4.2 Support for Fixed-point Arithmetic

We can also apply this work's approach to ciphertexts encrypting fixed-point arithmetic values. To do so, we can work with 16-bit data where we use 8 bits for the integer part and 8 bits for the decimal part of a fixed-point number. In addition, we need an encoding layer adapted to the semantic of this representation. We consider that the integer part can be signed or unsigned and that the decimal part is always positive. For example, 4.6 is represented as $(4, \lfloor 256 \times 0.6 \rfloor = 153)$ and -4.6 as $(-5, \lfloor 256 \times 0.4 \rfloor = 102)$. We note the ciphertexts and associated plaintexts corresponding to encryptions of such 16-bit messages with bold capital letters: $\boldsymbol{C} = (c_0, c_1, c_2, c_3) \in [\![h]\!] \times [\![l]\!] \times [\![o]\!] \times [\![k]\!]$ is an encryption of the 16-bit message $\boldsymbol{T}$ encoding $16h + l + \frac{16o+k}{256}$. For example, an encryption of $\frac{1}{256} = 0.00390625$ will be $\boldsymbol{C} \in [\![0]\!] \times [\![0]\!] \times [\![0]\!] \times [\![1]\!]$. This approach enables the implementation of new functions, such as decimal division or a fixed-precision sigmoid (Section 5.6).
As an example, let us consider the instruction DIV_DECi that performs the decimal division by a cleartext 8-bit value D (assuming unsigned input semantics for simplicity's sake). This kind of operation is often used when computing basic statistics when the sample size is known. Given $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and D, this instruction outputs $\bar{C} = (\bar{c}_0, \bar{c}_1) \in [\![\bar{h}]\!] \times [\![\bar{l}]\!]$ and $\tilde{C} = (\tilde{c}_0, \tilde{c}_1) \in [\![\tilde{h}]\!] \times [\![\tilde{l}]\!]$ such that

$$16\bar{h} + \bar{l} = \mathsf{tab_int}_D[16h + l] = \left\lfloor \frac{16h + l}{D} \right\rfloor$$

and

$$16\tilde{h} + \tilde{l} = \mathsf{tab_dec}_D[16h + l] = \left\lfloor 256 \frac{(16h + l) \mod D}{D} \right\rfloor.$$

With only 5 BlindRotate, we then compute $(\bar{c}_0, \bar{c}_1, \tilde{c}_0, \tilde{c}_1)$ as the result of

$$\mathsf{MVLUTeval}(c_0; c_1, c_1, c_1, c_1; \mathsf{tab_int_msn}, \mathsf{tab_int_lsn}, \mathsf{tab_dec_msn}, \mathsf{tab_int_lsn})$$

We also propose instructions to perform ciphertext-ciphertext additions (ADD_DEC) and cleartext-ciphertext multiplication (MUL_DECi) on this type of data.

5.4.3 Input/Output

In this section, we consider the issue of getting data in and out of our processor abstraction in the setting where it is deployed on a remote server and available to a *client*, which sends input ciphertexts to the server (which we refer to as *uplink* input transmissions from the client to the server) and expects output ciphertexts in return (which we refer to as *downlink* output transmissions from the server back to the client), as the results of some valuable computations. We then wish to avoid the naive approach, which consists of the client sending its encrypted input data by transferring a full TLWE ciphertext for each payload nibble (similarly on the downlink).

Still, in this naive setting (and considering the parameters in Table 2.1), our approach is more efficient than the "standard TFHE gate approach" as we require transmitting two TFHE ciphertexts with a 4-bit payload (for a total size of 65600 bits) by opposition to eight ciphertexts with a single bit payload (accounting for a total of 161536 bits using the default TFHELib parameters).

Uplink Input Data Transmission

On the uplink, a standard approach is to resort to transciphering to remove the transmission overhead [CCF+18, BBS22, BCBS23, PJH23], at the cost of homomorphically running a symmetric algorithm decryption function (which can then be easily "coded" using our instruction set). If we accept a slightly higher transmission overhead, a computationally lighter approach consists of simply synchronizing the client and server using a PRF to avoid sending the a term of the TLWE pairs (i.e., both the server and the client are able to compute on their own the a vector associated to a given $b = \langle a, s\rangle + \frac{q}{16}m + e$) and thus to transmit only the unique coefficient b. The uplink expansion factor, therefore, becomes independent of the n parameter. Since the ciphertext and plaintext moduli, respectively, are $q = 2^{32}$ and $B = 16$ in our TFHE parameter setting for basis-16 (Table 2.1), this leads to an expansion factor of only $\frac{32}{4} = 8$, which is reasonable by "FHE standards".

Downlink Output Data Transmission

Remark that none of the above approaches are applicable to reduce the overhead of encrypted results transmission from the server to the client. Indeed, transciphering allows the conversion of data encrypted under some (usually symmetric) scheme towards an *homomorphic* scheme, but not the other way around. Besides, for results of FHE computations, neither the server nor the client can control the resulting a term, which, therefore, has to be transmitted somehow. Still, to decrease as much as possible the burden of transmitting several encrypted outputs, under the form of TLWE ciphertexts, the server can assemble them as much as possible in TRLWE ones. Thus, we want to assemble $K = 2L$ TLWE results into one or more TRLWE ciphertexts. More precisely, the server assembles up to n TLWE samples into a single TRLWE sample using the usual keyswitch packing whereby n TLWE messages $m_0, \cdots, m_{n-1}$ maps to $m(X) = \sum_0^{n-1} m_i X^i$. Consider that K TLWE ciphertexts have to be transmitted, then, when $K \mod n = 0$, we have an expansion factor of

$$\frac{2\log_2 q}{\log_2 t} \tag{5.5}$$

i.e., with $n = 1024$ and $q = 2^{32}$, this leads to an expansion factor of 16 ($B = 16$). So, the downlink expansion factor is "only" twice that of the uplink (asymptotically).

When, $K \mod n = r > 0$, expansion is given by

$$\frac{2\lfloor k/n \rfloor \log_2 q + (n+r)\log_2 q}{K \log_2 t}.$$

Expansion factor (5.5) is also valid in the asymptotic regime when K is large. Other techniques may be used to further reduce the expansion factor on the downlink, e.g., [BDGM19, BCCS24].

5.4.4 Bit Decomposition and Recomposition

Decomposition (DC)

In some algorithms, it is more interesting for specific operations to work directly over bits. For instance, the symmetric sponge-based cipher ASCON [DEMS21] requires switching from a binary row representation to a column representation. Thus, we must decompose a ciphertext c into eight encryptions of bits. To perform such an operation with our bootstrapping tools, we must decompose our usual ciphertext $C = (c_0, c_1)$ into encryptions of bits. To do so, it is sufficient to decompose c_0 and c_1 each into four ciphertexts. Four tables are thus needed: one per decomposition bit. These tables are easy to precompute as it only requires calculating for $i \in \{0, 1, \cdots, 15\}$, the LUTs corresponding to i `& 0b0001`, (i `& 0b0010`)>> 1, (i `& 0b0100`)>> 2, and (i `& 0b1000`)>> 3. Then, the user can perform an MVB bootstrapping using the four test-vector polynomials given by the four 16-entries tables and extract the four results. Note that this operation is less expensive than a LUTeval call. Indeed, thanks to the use of MVB, we can perform the extraction of the four bits of a nibble with only one BlindRotate and no KeySwitch (that is to say, this operation costs 1 BR and 0 KS vs. 2 BR and 1 KS for a LUTeval evaluation). Finally, even if we now have encryptions of either 0 or 1, these ciphertexts are still manipulated with the parameters corresponding to a basis 16 encryption. This is a crucial principle for a cheap recomposition.

Recomposition (RC)

Once done working on a smaller basis, one should recompose their ciphertext into the initial basis to continue their computations. In our specific case of ciphertexts of basis 16 decomposed 8-bit data, that means that we want to obtain $c' = (c'_0, c'_1)$ from $c = (c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7)$ encrypting the same message m. As previously stated, the individual ciphertexts $c_0, \cdots, c_7$, even encrypting binary values, are still in the 32-value discretized torus. This simplifies the recomposition into 4-bit ciphertexts. Indeed, we have

$$c'_0 = 2^3 \cdot c_0 + 2^2 \cdot c_1 + 2 \cdot c_2 + c_3 \quad \text{and} \quad c'_1 = 2^3 \cdot c_4 + 2^2 \cdot c_5 + 2 \cdot c_6 + c_7.$$

As the multiplication by a power of $2 < 16$ will not result in an overflow (when considering nibbles), we can use a call to SimpleBoot on each c_i being multiplied. In these conditions, we can use the native TFHE TLWE addition to recompose each nibble. Still, this recomposition alone takes longer than a SHLi instruction, so a decompose/shift-for-free/recompose approach is not competitive. Hence, decomposing and recomposing ciphertexts is only efficient when many binary operations have to be performed.

> **Note**
>
> These DC and RC instructions correspond to the Decomposer and Recomposer operations proposed in the Hippogryph framework (Section 4.3), but without the encoding switching!

5.5 Instructions Timings

We have fully implemented our proposed instruction set under TFHElib. We summarize in Table 5.3 the timings obtained on our test machine (a 12th Gen Intel(R) Core(TM) i7-12700H CPU laptop with 64 Gib total system memory with an Ubuntu 22.04.2 LTS server), using only a single core.

Instr.	N_{BR}	N_{KS}	**ms**	**Instr.**	N_{BR}	N_{KS}	**ms**
ANDi/ORi/XORi	2	0	69	AND/OR/XOR	4	2	278
DC	2	0	81	RC	8	0	267
(U)SHLi/(U)SHRi	2	0	72	(U)SHL/(U)SHR	6	4	478
ROLi/RORi	4	0	125	ROL/ROR	9	6	714
EQi	2	0	88	EQ	6	3	393
LT(E)i/GT(E)i	2	1	126	LT(E)/GT(E)	9	5	623
(N)CDUP	3	1	159	CSEL	9	6	694
NEG/ABS	2	1	215	MIN/MAX	16	10	1176
ADDi/SUBi	2	1	137	ADD/SUB	7	4	493
ADDZ	4	2	271	MUL(M)i/DIV(4)i/MOD4i	2	1	133
MODi	3	2	267	MUL	10	6	725
MULM	32	20	2442	DIV4	21	14	1624
DIV	97	56	7711	MOD4	10	6	724
MOD	91	50	7584	(N)CDUPi	1	0	33
XOP	3	2	229	MINi/MAXi	2	1	135

Table 5.3: Mnemonics, BlindRotate and KeySwitch counts as well as execution times for our (T)FHE processor abstraction instruction set.

As a complement to Table 5.3, let us now illustrate the benefits of the successive optimizations that we have proposed and applied by focusing on the ADDi and ADD instructions (as representative examples of non-straightforward but not too overly complex univariate and bivariate instructions).

First, for the ADD instruction, one could start naively by trying to use the TBM relying on the basis-256 functional bootstrapping alone. This approach would require 257 BlindRotate and one packing via a TLWE-to-TRLWE KeySwitch, leading to an overall execution time of around 385 seconds. Doing the same while further relying on the MVB gets this timing down to 3 secs. Then, using our paper approach relying on a two basis-16 digits decomposition of each 8-bit data,

- Using the TBM without MVB leads to a run time of 2.32 secs for that instruction.
- With MVB (i.e. LUTeval), this number gets down to 0.52 secs.
- With the additional factorization allowed by MVLUTeval, we then obtain our final timing of 0.49 secs for the ADD instruction.

Then, regarding the ADDi instruction, one can also naively start by performing a single basis-256 bootstrapping leading to an execution time of 1.5 secs. Then, following our approach, we obtain the following successively refined timings:

- Using the TBM without MVB leads to 1.16 secs.
- Doing the same with MVB (LUTeval) gets that number down to 0.26 secs.
- The MVLUTeval optimization leads to 0.23 secs.
- Then, with our final optimization which factors one more simple bootstrapping (Remark 3, p. 104) we obtain our final timing of 0.13 secs.

Table 5.4 summarizes the above successive timing refinements.

Opt. level	$\mathsf{N_{BR}}$	$\mathsf{N_{KS}}$	Timing (s)
Basis-256/TBM w/o MVB	257	1	385
Basis-256/TBM w. MVB	2	1	3
Basis-16/TBM w/o MVB	68	4	2.32
Basis-16/TBM w. MVB	8	4	0.52
Basis-16/MVLUTeval	7	4	0.49
Basis-256/SimpleBoot	1	1	1.5
Basis-16/TBM w/o MVB	34	2	1.16
Basis-16/TBM w. MVB	4	2	0.26
Basis-16/MVLUTeval	3	2	0.23
Basis-16/Rem. 3	2	1	0.13

Table 5.4: Illustration of the benefits of the successive optimizations proposed and applied in this paper on the ADDi (lower half) and ADD (upper half) instructions.

5.6 From Instructions to Algorithms

To test our instruction set, we now use it to implement a number of (simple) algorithms. Note that in certain cases, it might be more efficient to directly implement these algorithms at the functional bootstrapping level. However, by analogy to a real microprocessor, that would mean coding at the micro-code level rather than at the ISA level. So, in this section, we only use instructions from our set.

5.6.1 Testing a Few Elementary Algorithms

Bubble Sort

Bubble sorting consists of repeatedly comparing consecutive elements in an array and permuting them when incorrectly ordered. One way to perform the conditional swap of two array elements without resorting to an if-then-else construct can be, for example, done using MAX and MIN computations. However, it is more efficient to use GT, CDUP, NCDUP and ADDZ as done in Algo 5. To give an order of magnitude for the execution time, sorting an array of five ciphertexts encrypting 8-bit values using this "sorting in place" algorithm takes around 15 seconds. More execution timings can be found in Table 5.5.
Recall that $\mathsf{CDUP}(c_b, C)$ returns an encryption of $c_b \times C$, and $\mathsf{NCDUP}(c_b, C)$ returns an encryption of $(1 - c_b) \times C$, with c_b the encryption of a Boolean value.

Algorithm 5 BubbleSort

Input: A an array of n encryptions of 8-bit values
Output: A sorted from the smallest value to the largest.

for $i = n - 1$ **to** 0 **do**
  for $j = 0$ **to** $i - 1$ **do**
    $c_b \leftarrow \mathsf{GT}(A[j], A[j+1])$
    $\tilde{C} \leftarrow A[j+1]$
    $C_s \leftarrow \mathsf{CDUP}(c_b, A[j])$
    $A[j+1] \leftarrow \mathsf{NCDUP}(c_b, A[j+1])$
    $A[j+1] \leftarrow \mathsf{ADDZ}(C_s, A[j+1])$
    $C_s \leftarrow \mathsf{CDUP}(c_b, \tilde{C})$
    $A[j] \leftarrow \mathsf{NCDUP}(c_b, A[j])$
    $A[j] \leftarrow \mathsf{ADDZ}(C_s, A[j])$
return A

Maximum/Minimum of an Array

As another simple example, it is easy to use our MIN and MAX homomorphic operators to find the largest or smallest element in a table, as done by Algo 6. With

this algorithm, finding the maximum or minimum of an array composed of five 8-bit encrypted values takes less than 5 seconds (see Table 5.5).

Algorithm 6 Maximum

Input: A an array of n encryptions of 8-bit values
Output: $\bar{C}$ a ciphertext encrypting the largest value in A

$\bar{C} \leftarrow A[0]$
for $i = 1$ **to** $n - 1$ **do**
  $\bar{C} \leftarrow \mathsf{MAX}(\bar{C}, A[i])$
return $\bar{C}$

Average

Thanks to our homomorphic decimal division operator, we are able to precisely compute the average of an array of encrypted values and obtain a fixed-point arithmetic encrypted result. Algo 7 gives an implementation with our instruction set. As shown in Table 5.5, this computation takes less than 4 seconds when tried on a five-element array.

Algorithm 7 Average

Input: A an array of n encryptions of 8-bit values
Output: $\bar{\boldsymbol{C}}$ a ciphertext encrypting the 16-bit value corresponding to the average of the table A

$C_a \leftarrow A[0]$
for $i = 1$ **to** $n - 1$ **do**
  $C_a \leftarrow \mathsf{ADD}(C_a, A[i])$
$C_i \leftarrow \mathsf{DIVi}(C_a, n)$
$C_d \leftarrow \mathsf{DIV_DECi}(C_a, n)$
return $\bar{\boldsymbol{C}} = (C_i, C_d)$

Array Dereferencing and Assignement

Note that dereferencing an array of 256 (or less) *cleartext* values (with an encrypted index) is just an evaluation of our MVLUTeval operator. Further optimizations can be made on a case-by-case basis, for example, if the array to be dereferenced contains fewer than 256 values. It is also feasible to dereference an array of 256 (or less) *encrypted* values with an encrypted index. Indeed, we can use a modified MVLUTeval running directly on encrypted test polynomials, as in the second level of the evaluation of the TBM , where we rotate a new *encrypted* test polynomial by an encrypted index. Dereferencing arrays with more than 256 elements is also possible

but requires a slightly more complex machinery that we do not detail (indeed, as 256 is the size of our plaintext domain we have tools to bootstrap over a 256-element table straightforwardly, but when there are more than 256 elements in the table, a single tree-based bootstrapping is not enough). Lastly, we can also obtain an operator for assigning an array of 256 (or less) encrypted values, still with an encrypted index. That is to say, given an encrypted table tab of size n, an encrypted index $C_i = (c_{i_0}, c_{i_1}) \in [\![i_0]\!] \times [\![i_1]\!]$ and an encrypted value $C_V = (c_{v_0}, c_{v_1}) \in [\![v_0]\!] \times [\![v_1]\!]$, the operation affects the value $V = 16v_0 + v_1$ to $\mathsf{tab}[16i_0 + i_1]$, as proposed in Algo 8. As seen in Table 5.5, the sequential evaluation of this operator on an array of five 8-bit encrypted inputs takes 4.45 seconds. Note that this approach to array dereferencing is *not* competitive with PIR approaches running over RLWE schemes. However, these latter approaches are, by intent, only able to execute PIR requests very efficiently and do not claim to achieve more than that.

Algorithm 8 Assignment

Input: A an array of n encryptions of 8-bit values, an encrypted index C_i and an encrypted value C_V

Output: A modified at index $16i_0 + i_1$

for $j = 0$ **to** $n - 1$ **do**
  $(0, c_b) \leftarrow \mathsf{EQI}(C_i, j)$
  $C_0 \leftarrow \mathsf{CDUP}(c_b; C_V)$
  $C_1 \leftarrow \mathsf{NCDUP}(c_b; A[j])$
  $A[j] \leftarrow \mathsf{ADDZ}(C_0, C_1)$
return A

Squares Sum

As another simple algorithm, let us consider a Sum of Squares computation. It is a rather simple algorithm. However, taking a glimpse at Section 5.7, it already induces a number of gates large enough for the Boolean circuit approaches to not be competitive anymore. By contrast, with our instruction set, it is (also) straightforward to implement (see Algo 9) and more efficient. Indeed, as seen in Table 5.5, the sequential evaluation of this operator on an array of five 8-bit encrypted inputs takes 5.66 seconds when the best Boolean circuit-based approach takes 7.6 secs (Table 5.7).

Loops

For completion, we highlight a technique to perform (encrypted) data dependant loop termination when a bound B is known on the total number of iterations. Let S denote the state of a program, then a statement of form "$\mathsf{while}\ c(S)\ \mathsf{do}\ S := f(S)$" can be rewritten as "$\mathsf{for}\ 0 \leq i < B\ \mathsf{do}\ \mathsf{if}\ c(S)\ \mathsf{then}\ S := S\ \mathsf{else}\ S := f(S)$". In essence,

Algorithm 9 SquaresSum

Input: A an array of n encryptions of 8-bit values
Output: C the encryption of the sum of the squares of the n values of A

$C_s \leftarrow \mathsf{MUL}(A[0], A[0])$
for $j = 1$ **to** $n-1$ **do**
  $C_t \leftarrow \mathsf{MUL}(A[j], A[j])$
  $C_s \leftarrow \mathsf{ADD}(C_s, C_t)$
return C_s

that latter form computes a fixed point after condition $c(S)$ reaches a true value, and the inner if-then-else statement can then be done via a CSEL instruction.

5.6.2 Evaluation of an Elementary Neuron

We now turn to the homomorphic evaluation of an elementary neuron, as usually found in convolutional neural networks. Our simple neuron has two encrypted fixed-precision inputs representing encryptions of numbers, $\boldsymbol{F_1}$ and $\boldsymbol{F_2}$, in $[-1, 1]$ (each over 16 bits as in Section 5.4.2) and one encrypted fixed-precision output of the same form. *We emphasize that the output of our neuron can be fed to another one, enabling the evaluation of larger networks over encrypted data.* From an operational viewpoint, the two encrypted inputs are first multiplied by cleartext fixed precision weights in $[-1, 1]$ ($\boldsymbol{W_1}$ and $\boldsymbol{W_2}$, respectively), which may either be cleartexts or ciphertexts. The sum of these products is then fed into an activation function, in this case, the sigmoid, noted σ (which takes an encrypted fixed-precision value as input and evaluates the sigmoid at that point). In summary, specified over cleartext values, we have to evaluate

$$\mathsf{neuron}(\boldsymbol{F_1}, \boldsymbol{F_2}) = \sigma(\boldsymbol{F_1} \cdot \boldsymbol{W_1} + \boldsymbol{F_2} \cdot \boldsymbol{W_2}).$$

Let $\boldsymbol{C_{F_1}} = (c_0, c_1, c_2, c_3) \in [\![h]\!] \times [\![l]\!] \times [\![o]\!] \times [\![k]\!]$, (meaning, as in Section 5.4.2, that $\boldsymbol{C_{F_1}}$ encrypts the value $\boldsymbol{F_1} = 16h + l + \frac{16o+k}{256}$) and $\boldsymbol{C_{F_2}} = (c'_0, c'_1, c'_2, c'_3) \in [\![h']\!] \times [\![l']\!] \times [\![o']\!] \times [\![k']\!]$. This way, we have to compute two cleartext-ciphertext decimal multiplications, one homomorphic decimal addition, and the homomorphic evaluation of the sigmoid.

The most complicated part of that computation is the homomorphic evaluation of the sigmoid, taking as input a ciphertext corresponding to a 16-bit fixed-point arithmetic value. To do so, we evaluate a discretized sigmoid $\tilde{\sigma}((i, j))$ on several non overlapping intervals $(-\infty, -6)$, $[-6, -5)$, ..., $[5, 6)$, $[6, \infty)$ as

$$\tilde{\sigma}((i,j)) = \begin{cases} (0,0) & \text{if } i < -6, \\ (0, \mathsf{tab_sig}_i[j]) & \text{if } i \in \{-6, -5, ..., 4, 5\}, \\ (1,0) & \text{if } i \geq 6, \end{cases}$$

with $\mathsf{tab_sig}_i[j] = \sigma(i + j/256)$ and then test the input to select the appropriate value among these. However, because we do not have any way to branch on conditions over

encrypted data, each of the above (mutually exclusive yet collectively exhaustive) possibilities must be computed, multiplied by an encrypted boolean, and then xored to obtain the final result. It follows that, for a ciphertext $\boldsymbol{C} = (c_0, c_1, c_2, c_3) \in [\![h]\!] \times [\![l]\!] \times [\![o]\!] \times [\![k]\!]$, we obtain the encrypted result $\tilde{\mathbf{C}}$ of the evaluation of the sigmoid SIG by computing:

$$\bar{\boldsymbol{C}} = (\bar{C}, \tilde{C}) = (\mathsf{GTEi}((c_0, c_1), 6), \bigoplus_{i \in \{-6, \cdots, 5\}} \mathsf{CDUP}(\mathsf{EQi}((c_0, c_1), i), \mathsf{SIGLUT}^{(i)}((c_2, c_3))))$$

with $\bar{C}$ and $\tilde{C}$, the respective encryptions of the integer and decimal parts of the result. In the above equation, the $\bigoplus$ operator corresponds to multiple calls to our XOR[3] instruction and $\mathsf{SIGLUT}^{(i)}$ is the homomorphic evaluation of the LUT corresponding to the decimal values of $\sigma\left(i + \frac{16o+k}{256}\right)$.

Note that from an instruction set perspective, $\mathsf{SIGLUT}^{(i)}$ can be performed by means of the XOP (user-defined) univariate instruction discussed in Section 5.3.3 using the above twelve $\mathsf{tab_sig}_i$ tables. This implementation of the sigmoid takes about 10 seconds to compute. With less precise encrypted inputs and outputs, homomorphic sigmoid evaluation can be less costly [TCBS23a], but here, we prioritize accuracy and the ability to feed a neuron output into another neuron without additional conversion over faster execution. Based on this, we have been able to evaluate one neuron in about 16 seconds, so the evaluation of the sigmoid alone represents two-thirds of that cost. For comparison, we have also implemented a homomorphic Heaviside function that operates on encrypted inputs representing 16-bit fixed-point arithmetic values. Using this much simpler activation function, we can get the execution timing from 16 seconds down to under 7 seconds, as shown in Table 5.5.

[3]Note that this sum can also be computed with our ADDZ instruction.

Note

Before this PhD, I was lucky enough to obtain an internship within the same team, whose subject was "Can RNN go homomorphic?". The idea was to use TFHE's functional bootstrapping to perform the activation functions (sigmoid and hyperbolic tangent) of an LSTM cell. I encountered several challenges while attempting to perform a homomorphic evaluation of these functions. By discretizing the sigmoid into 16 steps, I was able to evaluate it without a severe loss of accuracy on the final result. However, because I used 8 bits of precision (decomposed into two nibbles), I needed to map the evaluated values from the range $[0, 1[$ into our discretized plaintext space of $\{0, 1, \ldots, 15\}$, which complicated the subsequent operations (additions and multiplications). This led to a semantic issue that needed to be addressed and was left as an open perspective. Nonetheless, these findings were published in an article at the beginning of my thesis [TCBS23a], which concluded: "**This work is a first step. However, an issue remains. Considering our non-standard interpretation of the values in $\{0, \frac{1}{16}, \frac{2}{16}, \cdots, \frac{15}{16}\}$ in the activation function outputs means that the subsequent addition and multiplication operations cannot be performed directly. Thus, several perspectives have to be investigated for the complete network execution. We can either switch back to the standard interpretation at carefully chosen points during the network execution (which may cost an additional bootstrapping per conversion) or propose new LUT-based operators for performing additions and multiplications directly in the non-standard interpretation (or a combination of both). Both approaches will incur additional bootstrappings, which have not been considered in our coarse-grained estimations.**". With `Mad Panthera`, we solve this issue and propose a *complete neuron execution with even more precision* than in our previous work.

5.7 Comparison with Other Approaches

In this section, we compare the performances of our approach to the following projects: Juliet [GMT24], E3 [CMG+18], Cingulata [CDS15a], and Concrete [Zam22]. We aim to compare the execution timings with ours on the set of algorithms from Section 5.6. An average timing is computed on a hundred samples for each framework and algorithm. All the results are summarized in Table 5.7.

Juliet [GMT24] Juliet[4] is a general-purpose homomorphic computation framework. It provides C++ functions corresponding to instructions implemented as

[4]https://github.com/TrustworthyComputing/Juliet

Table 5.5: Execution times (in seconds) of different homomorphic algorithms on arrays of size n, and expecting timings depending on n.

Algorithm	$n = 2$	$n = 5$	$n = 10$	Expected timings
BubbleSort	1.51	15.22	68.47	$\frac{n(n-1)}{2} \times (4 \times 0.160 + 2 \times 0.270 + 0.390)$
Minimum/Maximum	1.20	4.71	11.14	$(n-1) \times 1.176$
Average	0.78	2.36	4.97	$(n-1) \times 0.493 + 2 \times 0.133$
ArrayAssignment	1.37	4.45	6.88	$n \times (0.088 + 2 \times 0.159 + 0.271)$
SquaresSum	2.12	5.66	12.30	$n \times 0.725 + (n-1) \times 0.493$
	Times			
Neuron with Sigmoid	15.42	-	-	-
Neuron with Heaviside	6.67	-	-	-

Boolean circuits over ciphertexts using TFHElib. Juliet also supports GPU acceleration, but as we perform single-thread CPU computations, we have not activated this feature for fair comparison. Juliet also gives a relatively small set of around 20 instructions (working over encrypted data); it is then relevant to compare our approaches on the instructions we have in common:

- ADD, SUB, MUL, MOD
- AND, OR
- EQ, GT, LT, GTE, LTE (these instructions are all packed in COMP in Juliet)

Additionally, many of our instructions are not supported in Juliet, particularly DIV, MOD as well as SHL and SHR (variants for shifting an encrypted value by an *encrypted* offset). Table 5.6 shows that with the same parameters for TFHE, our approach is, on average, 74% faster on unitary instruction execution.

Operation	This work	Juliet
AND	0.278	0.268
OR	0.278	0.254
ADD	0.493	1.012
SUB	0.493	1.231
MUL	0.725	5.334
EQ/COMP	0.393	2.195

Table 5.6: Average (over 100 runs) execution timings (in seconds) of different homomorphic instructions on encrypted inputs using Juliet and the present work's approach.

Cingulata [CDS15a, ACS20] Cingulata[5], formerly known as Armadillo [CDS15b], is a toolchain and run-time environment (RTE) for implementing applications running over HE. Cingulata provides high-level abstractions and tools to facilitate the implementation and execution of applications running over encrypted data. In Cingulata, programs are expressed in high-level C++ and automatically turned into optimized Boolean circuit form. Its runtime environment then performs the homomorphic execution over the selected FHE scheme, which, for this work, has been the gate-based variant of TFHE (Cingulata also relies on the TFHElib library). Additionally, even if it is not supported natively, we have implemented the homomorphic integer division given in 5.3.4 using Cingulata for further comparisons (see Table 5.8).

As shown in Table 5.7, Cingulata sometimes outperforms our approach: For the Maximum and BubbleSort algorithms, it is faster by an average factor of 60% (but note that for these algorithms corresponding to small Boolean circuits, our approach still outperforms both Juliet and Concrete), as for the SquaresSum and Sigmoid algorithms, our approach is more efficient by an average factor of 70%. It turns out that Cingulata performs very well on homomorphic calculations that lead to small Boolean circuits (which it executes with a faster bootstrapping). In contrast, our approach is faster when the size of the underlying Boolean circuit increases, i.e., when the ratio between the Boolean circuit size and the number of basis 16 operations is above the time ratio between basis 16 and basis 2 bootstrappings.

E3 [CMG+18] E3 (Encrypt-Everything-Everywhere)[6] is a compilation framework similar in spirit to Cingulata. E3 supports bridging: homomorphic evaluation is performed by mixing both arithmetic and binary circuits, which speed up the computation in certain cases. E3, however, does not allow easy setting of the parameters for TFHE and presently uses the default TFHElib parameters, achieving only 118 bits of security (this gives E3 a slight advantage over our approach, which uses larger parameters to achieve 128 bits of security). To the best of our knowledge, E3 is the only other framework that supports homomorphic division (hence, we can compare the two approaches when computing the homomorphic average of an array). Note that in Table 5.7, for the `Average` algorithm, we perform a homomorphic decimal division with our framework, and only an integer division with E3. Like Cingulata, E3 outperforms our approach for the same algorithms. During our tests, Cingulata consistently outperformed E3 (except on the Average algorithm, on which our approach outperforms both E3 and Cingulata). As Sigmoid was complex to implement, we only implemented it for the most competitive approach to ours: Cingulata. (see Table 5.7).

[5] https://github.com/CEA-LIST/Cingulata
[6] https://github.com/momalab/e3

Concrete [Zam22] Concrete [7] by Zama is also an FHE compiler. It is built on an LLVM-based compiler that can transform functions running over plaintexts into a mix of boolean circuits for so-called leveled operations (addition, multiplication, linear application) and LUTs (that they call TLU) for nonlinear operations that will eventually run over their custom implementation of TFHE. Its Python interface makes it easy to use: we write functions over plaintexts, compile them, and run them over ciphertexts in a single Python script. As E3, Concrete does not let the user control or even access the parameters of the TFHE scheme, meaning that we lack complete visibility to compare their parameters with ours. However, as both parameter sets achieve 128-bit security, we assume that the comparison remains fair.

As stated in Concrete's documentation, the tool is most efficient for computations that can be performed using leveled (in their terminology) operations rather than LUTs. This is confirmed in Table 5.7 where Concrete is competitive on the SquaresSum algorithm.

Algorithm	This work	Juliet	E3	Cingulata	Concrete
Maximum	4.74	7.91	3.13	1.87	21.52
BubbleSort	15.22	31.62	12.70	6.18	78.07
SquaresSum	5.66	25.10	8.82	8.40	7.69
Average	2.36	-	3.57	10.31	-
Sigmoid	8.98	-	-	969	-

Table 5.7: Average (over 100 runs) execution timings (in seconds) of different homomorphic algorithms on arrays of 5 encrypted inputs using various approaches. Entries marked with a '-' are so due to lack of support of a unitary operation (e.g., division for Average).

Concluding Remarks The rationale for proposing an 8-bit instruction set (working over encryptions of two nibbles) stems from the facts that TFHE can only handle small plaintext domains without inducing huge parameters and that the computational cost of TFHE bootstrapping depends heavily on the plaintext size (as shown in Table 5.1). So, proceeding gate-by-gate over encryptions of bits (the FHE compilers' approach) means performing large numbers of FHE operations with a fast bootstrapping (≈7 ms), while proceeding as we do requires performing comparatively fewer operations with a more costly bootstrapping (≈29 ms). The question is, therefore, whether this extra bootstrapping cost can be amortized in the instruction set approach, leading to better performances. Our experimental results in Tables 5.7 and 5.8 show that our approach amortizes this high bootstrapping cost and outperforms the other state-of-the-art compilers. This is, for example, the case for the

[7] https://github.com/zama-ai/concrete

SquaresSum algoritm (between ×1.3 and ×4.4 better), which uses MUL, an operation that requires a larger number of Boolean gates. This is further exemplified by the more complex DIVi and DIV instructions (respectively ×50 and ×1.5 faster) and the Sigmoid (×100 faster). Still, Tables 5.7 and 5.8 indeed show that the Boolean circuit-oriented approach is more efficient only for algorithms that do not suffer from a significant expansion when represented as a Boolean circuit. However, note that, even on the algorithms with small Boolean circuits, our approach still beats Juliet and Concrete.
The above results mean that there is a significant space in which our approach outperforms the Boolean circuit approach implemented by the FHE compilers.

	This work			Cingulata	
Algorithm	N_{BR}	N_{KS}	Time(s)	N_{BR}'	Time(s)
DIVi	2	1	**0.13**	700	7.38
DIV	97	56	**7.71**	1100	11.17
ADD	7	4	0.49	35	**0.36**
MUL	10	6	**0.72**	183	2.14
CSEL	9	6	0.69	0	**0.001**
Maximum	64	40	4.74	184	**1.87**
BubbleSort	260	110	15.22	620	**6.18**
SquaresSum	78	46	**5.66**	821	8.40
Average	32	18	**2.36**	1000	10.31
Sigmoid	156	60	**8.98**	106652	969

Table 5.8: Average (over 100 runs) execution timings and number of bootstrappings of different homomorphic algorithms on arrays of 5 encrypted inputs (when applicable) using Cingulata and our approach. *Note that the top part of the above table provides a comparison of our approach with one relying on "standard TFHE gates" (as Cingulata exactly implements this approach with optimized Boolean circuits) over a set of representative instructions (implemented as small unitary programs in Cingulata).*

5.8 Conclusion and Perspectives

In this work, we have essentially shown that a very limited set of functional bootstrapping patterns is both versatile and optimal to build a complete conventional-looking assembly language for manipulating (T)FHE encryptions of 8-bit data. In terms of perspectives, this reveals several functional bootstrapping operators of increasing complexity which may be appropriate targets for further works on advanced software optimizations or hardware implementations in an intent, e.g., to provide a wide range of higher level instructions to the user while maintaining a small number

of hardware operators (also leveraging on the fact that TFHE needs smaller parameters compared to the RLWE schemes). Indeed, our approach would directly benefit from further efficiency improvements in the baseline TFHE bootstrapping but also in the higher-level LUTeval or MVLUTeval operators. Beyond this, the approach can also benefit from the ability to run several such primitives in parallel, ideally by exploiting the low-level SIMD instructions offered by modern processors or dedicated HW.

Another important perspective is to further investigate several values for the bootstrapping error probability to consider the recent attacks in [CSBB24, CCP+24]. Indeed, our parameters achieve "only" a 2^{-40} bootstrapping error probability. Although parameters have been proposed in [CSBB24] for a 2^{-128} bootstrapping error probability, showing a 20% overhead in the baseline bootstrapping, they are valid only for $B = 2$. Finding a parameter set for basis $B = 16$ achieving such a low probability remains challenging (due to the necessary increase in polynomial degree and ciphertext modulus), and in that regime, basis 4 might be the optimal choice. So achieving immunity against these recent attacks may, therefore, have an impact that remains to be studied in depth.

Chapter 6

Running Other Block Ciphers with TFHE

Introduction

In this work, we leverage on two methods which have successfully been applied to the homomorphic evaluation of AES [TCBS23b, BBB+25b] to study several state-of-the-art symmetric block ciphers (namely CLEFIA, PRESENT, PRINCE, SIMON, SKINNY). That is to say, we implement a representative set of symmetric block ciphers using TFHE. These implementations allow us to compare the efficiency of this set of symmetric schemes and to categorize them. We highlight the characteristics of block ciphers that are fast to execute in the homomorphic domain and those that are particularly costly. Finally, this classification of operation types enables us to sketch out what the ideal block cipher for transciphering homomorphic data in integer mode might look like.

This work has been the object of a paper which is currently in submission:**AES is Not Enough: the Block Ciphers Zoo Goes Homomorphic**, D. Trama, A. Boudguiga, R. Sirdey.

6.1 Method and Tools

To compare the efficiency of different standardized block ciphers, we propose using the same methods and tools as outlined in the contributions on AES (see Sections 4.2 and 4.3). In this work, we explain how to construct a generic method based on the "full-LUT" approach (a contribution presented in Section 4.2) for the homomorphic evaluation of any block cipher. It is important to note that while this approach may not be the fastest in the state of the art, it serves as a helpful metric for comparing different schemes. Additionally, we then conduct a second study on the same block ciphers using the Hippogryph framework (a contribution presented in Section 4.3). This second study allows us to determine whether the conclusions drawn from the

first approach are consistent or if new and interesting insights emerge from the second framework.

6.1.1 The full-LUT Approach

Building on the study presented in [TCBS23b] (Section 4.2), which has been validated and explored in greater detail in [TCB+25] (Section 5), we work with 8-bit plaintexts, which are divided into two 4-bit digits known as nibbles. Remember that this approach involves transforming all AES steps into LUT evaluations, including XOR operations and multiplication in $GF(256)$.

Reminder on the Evaluation of an 8-bit Sbox As we rely on [TCBS23b] to study block ciphers, we adopt the same type of ciphertexts, that is to say, we work with encryptions of nibbles (4-bit entities). We present here how to evaluate an 8-bit Sbox on 4-bit payload ciphertexts using TBM and MVB. An 8-bit message M is first decomposed into two 4-bit digits h and l such that $M = 16h + l$. The result of the evaluation of the Sbox sbox on M is thus given by $\mathsf{sbox}[M] = \mathsf{sbox}[16h + l] = 16h' + l'$, with $h' = \lfloor \frac{\mathsf{sbox}[M]}{16} \rfloor$ and $l' = \mathsf{sbox}[M] \pmod{16}$. We thus need two tables $\mathsf{sbox_msn}$ and $\mathsf{sbox_lsn}$ such that for $h, l \in \{0, \cdots, 15\}^2$ $\mathsf{sbox_msn}[16h + l] = \lfloor \frac{\mathsf{sbox}[16h+l]}{16} \rfloor$ and $\mathsf{sbox_lsn}[16h + l] = \mathsf{sbox}[16h + l] \pmod{16}$ to obtain the two nibbles composing the result. If we denote the encryption of M as $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$. Then, to homomorphically evaluate sbox on C, we have to call $\mathsf{MVLUTeval}(c_0; c_1, c_1; \mathsf{sbox_msn}, \mathsf{sbox_lsn})$, which costs 3 BR and 2 KS.

6.1.2 The Hippogryph Framework

The Hippogryph framework presented in Section 4.3 improves on the full-LUT approach, and even builds on it. We exploit this framework on top of the full-LUT approach to see if two different methods can produce the same results. Remind that the Hippogryph framework combines the byte-oriented full-LUT approach with a generalization of the boolean-oriented p-encodings method of [BPR24]. This combination allows us to capitalize on the strengths of both approaches. The p-encodings method [BPR24] encrypts data bit by bit and performs only boolean operations. It takes advantage of the fact that, in the plaintext space $\mathbb{Z}_2$, the native TFHE addition of TLWE ciphertexts effectively performs XOR. This allows the linear operations of the AES ($\mathsf{MixColumns}$ and $\mathsf{AddRoundKey}$) to be executed efficiently with minimal cost (almost for free when compared to the cost of bootstrapping) by using only the native TFHE addition. In the Hippogryph framework, the Sbox is performed with LUTs, as in [TCBS23b], and the rest of the algorithm is performed with the p-encodings method. Hippogryph thus requires decompositions and recompositions of the 4-bit payload ciphertexts. These two operations that we note DC and RC have been detailled in the ISA proposed in Section 5.4.4 and rely on programmable boot-

strapping. So, to simulate the Hippogryph framework, we only have to count the number of BR and KS to be executed. The decomposition of a 4-bit ciphertext c into four binary ciphertexts requires 1 BR and 0 KS(thanks to the MVB, we factorize the BlindRotate needed to obtain each bit of the decomposition) and the recomposition requires 4 BR and 0 KS (for each encryption of bit c_i, we compute with a classical bootstrapping $2^i \cdot c_i$ and then add the four new ciphertexts with the native TFHE addition). Knowing the execution timings of a BR (about 0.03 seconds) and of a KS (about 0.07 seconds) will help us imagine the efficiency of the framework and help us to draw some more conclusions.

6.2 Running Block Ciphers over TFHE

In this section, we present each studied block cipher. First, we provide a brief overview of the specifications and structures of each cipher. Next, we explain the work done to implement them efficiently using the full-LUT approach. Building on the results from the full-LUT method, we then leverage the Hippogryph framework for a second analysis. Finally, we highlight key conclusions and findings from both studies on the block ciphers.

6.2.1 CLEFIA

Specification.

CLEFIA (ISO/IEC 29192-2 standard) [SSA+07] is a 128-bit block cipher that supports key lengths of 128, 192, and 256 bits. It consists of two main components: a data processing part and a key scheduling part. CLEFIA uses a generalized Feistel structure with four data lines, each 32-bit wide. Additionally, it includes key whitening processes at the beginning and the end of the encryption. The number of rounds in CLEFIA varies depending on the key length: 18 rounds for a 128-bit key, 22 rounds for a 192-bit key, and 26 rounds for a 256-bit key. CLEFIA relies on two functions F_0 and F_1 that follow the same structure and use two 8-bit Sboxes in step 2. Figure 6.1 gives some more details about CLEFIA's specification.

Full-LUT Methodology.

To the best of our knowledge, this is the first attempt to evaluate CLEFIA in the homomorphic domain. We already know how to evaluate XOR and 8-bit Sbox with the full-LUT approach. So what remains to be transformed into a succession of LUTs is the matrix multiplication in $GF(256)$. To do so, we precompute the LUTs corresponding to multiplications by `0x02`, `0x04`, `0x06`, `0x08` and `0x0a` in $GF(256)$. Indeed, all other values are not used for the multiplication. For instance, M_0 is

$F_0 : (RK_{(32)}, x_{(32)}) \mapsto y_{(32)}$

Step 1. $T \leftarrow RK \oplus x$
Step 2. Let $T = T_0 \mid T_1 \mid T_2 \mid T_3,\ T_i \in \{0,1\}^8$
$T_0 \leftarrow S_0(T_0)$,
$T_1 \leftarrow S_1(T_1)$,
$T_2 \leftarrow S_0(T_2)$,
$T_3 \leftarrow S_1(T_3)$
Step 3. Let $y = y_0 \mid y_1 \mid y_2 \mid y_3,\ y_i \in \{0,1\}^8$
$^t(y_0, y_1, y_2, y_3) = M_0\ ^t(T_0, T_1, T_2, T_3)$

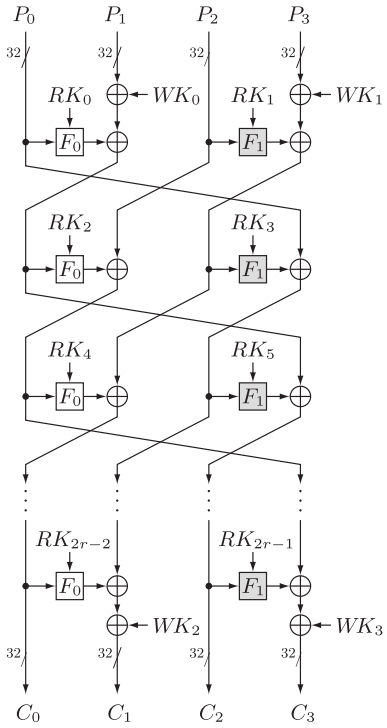


Figure 6.1: Figure extracted from [SSA+07].

defined as follows:

$$M_0 = \begin{bmatrix} \texttt{0x01} & \texttt{0x02} & \texttt{0x04} & \texttt{0x06} \\ \texttt{0x02} & \texttt{0x01} & \texttt{0x06} & \texttt{0x04} \\ \texttt{0x04} & \texttt{0x06} & \texttt{0x01} & \texttt{0x02} \\ \texttt{0x06} & \texttt{0x04} & \texttt{0x02} & \texttt{0x01} \end{bmatrix}$$

Each multiplication in $GF(256)$ is transformed into two tables: one that gives the most significant nibble of the multiplication (msn), and one that gives the least significant nibble of the result (lsn). For instance, from an 8-bit message $M = 16h+l$, we can obtain $M \times \texttt{0x02}$ by evaluating $16 \times \mathsf{mul_02_msn}[16h+l] + \mathsf{mul_02_lsn}[16h+l]$. In the homomorphic domain with $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$, this correspond to a call to $\mathsf{MVLUTeval}(c_0; c_1, c_1; \mathsf{mul_02_msn}, \mathsf{_mul_02_lsn})$.
Then, from the four bytes defined in Figure 6.1 by $T_0|T_1|T_2|T_3$ we can easily obtain $y_0 = T_1 \oplus (T_2 \times \texttt{0x02}) \oplus (T_3 \times \texttt{0x04}) \oplus (T_4 \times \texttt{0x06})$. Note that each 8-bit payload ciphertext T_i will be multiplied in $GF(256)$ by the three values `0x02`, `0x04`, and `0x06`. So we can optimize the computation by calling $\mathsf{MVLUTeval}$ on each T_i. More precisely, if we note $T_i = (c_{0_i}, c_{1_i})$ where c_{0_i} and c_{1_i} are nibble encryptions, then the call corresponding to each T_i is:

$$\begin{aligned} \mathsf{MVLUTeval}(\quad & c_{0_i}; c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}; \\ & \mathsf{mul_by_0x02_msn}, \mathsf{mul_by_0x02_lsn}, \mathsf{mul_by_0x04_msn}, \\ & \mathsf{mul_by_0x04_lsn}, \mathsf{mul_by_0x06_msn}, \mathsf{mul_by_0x06_lsn}). \end{aligned}$$

Even better, if we create six new tables Sbox02_msn, Sbox02_lsn, Sbox04_msn, Sbox04_lsn, Sbox06_msn and Sbox06_lsn, corresponding to the Sbox respectively multiplied by 0x02, 0x04 and 0x06, we can factorize calls to MVLUTeval one step further. Indeed, as the matrix multiplication directly follows the SBox evaluation, we can precompute the multiplication following the Sbox. Then, for each T_i we perform only one call to MVLUTeval that will give us all the values needed to compute steps 2 and 3 of F_0. For one T_i this one call would give Sbox$[T_i]$, Sbox$[T_i] \times$ 0x02, , Sbox$[T_i] \times$ 0x04 and , Sbox$[T_i] \times$ 0x06. Such a call can be defined in the following way:

$$\begin{aligned}\mathsf{MVLUTeval}(\quad & c_{0_i}; c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}, c_{1_i}; \\ & \mathsf{Sbox_msn}, \mathsf{Sbox_lsn}, \mathsf{Sbox02_msn}, \mathsf{Sbox02_lsn}, \\ & \mathsf{Sbox04_msn}, \mathsf{Sbox04_lsn}, \mathsf{Sbox06_msn}, \mathsf{Sbox06_lsn}).\end{aligned}$$

Thus, to obtain all the terms needed to obtain the y at the end of an F_0 evaluation, it requires $4 \times$ (9 BR,8 KS) + $4 \times 3 \times 2 \times$ (2 BR, 1 KS) = 84 BR and 56 KS instead of $4\times$(3 BR, 2 KS)+$4\times$(7 BR, 6 KS)+$4\times3\times2\times$(2BR, 1 KS) = 88 BR and 56 KS.

With such an optimization, one complete evaluation of F_0 costs 100 BR and 64 KS (if we add the counts for XORs on 32-bit words in step 1 of F_0). Thus a whole evaluation, containing 36 calls to F_i and 40 XORs would cost 4240 BR and 2624 KS, which takes about 311 seconds.

Hippogryph Approach.

We assume there exist, as for the AES, a boolean circuit corresponding to the matrix multiplication, which allows to compute this state nearly for free with the Hippogryph framework. With such framework, the only bootstrappings to compute are the ones for the Sboxes, and the ones to decompose (DC) and recompose (RC) the messages.
We perform RC at the beginning of an F_i on 4 bytes, thus costing $4 \times 2 \times$ (4 BR, 0 KS). After step 2, we perform DC again on 4 bytes, costing $4 \times 2 \times$ (1 BR, 0 KS). And of course, we perform the Sbox at step 2 on each byte, which costs $4 \times$ (3 BR, 2 KS). Thus, a full F_i execution costs a total of 52 BR and 8 KS, and 36 F_i executions cost 1872 BR and 288 KS. Estimated timings are given in Table 6.1.

Concluding Remarks.

Always look everywhere for small optimization. For instance, here our trick saves us 4 BR by F_i. It can seem to be nothing, but on a complete execution, it saves us 144 BR, that is to say about 4.5 seconds. Finally, let us note that with the Hippogryph approach, CLEFIA seems to be more efficient than the AES.

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$P(i)$	0	16	32	48	1	17	33	49	2	18	34	50	3	19	35	51
i	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$P(i)$	4	20	36	52	5	21	37	53	6	22	38	54	7	23	39	55
i	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47
$P(i)$	8	24	40	56	9	25	41	57	10	26	42	58	11	27	43	59
i	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63
$P(i)$	12	28	44	60	13	29	45	61	14	30	46	62	15	31	47	63

Figure 6.2: The bit permutation used in PRESENT: bit i of state is moved to bit position $P(i)$. This figure is extracted from [BKL+07].

6.2.2 PRESENT

Specification.

PRESENT (ISO/IEC 29192-2 standard) [BKL+07] is an example of a Substitution-Permutation network consisting of 31 rounds working on a 64-bit state matrix (meaning that we have to compute two executions of PRESENT to obtain 128 bits). Each round includes an XOR operation that incorporates a round key (K_i) for ($1 \leq i \leq 32$), with (K_{32}) being used for post-whitening. Each round also features a linear bitwise permutation and a non-linear substitution layer. The non-linear layer utilizes a single 4-bit S-box, which is applied on the 16 nibbles of the state during each round. Figure 6.2 presents the bit permutation of the cipher, also called *pLayer*. The execution of PRESENT can be resumed with the following algorithm:

generateRoundKeys()
for $i = 1$ **to** 31 **do**
    addRoundKey(STATE,K_i)
    sBoxLayer(STATE)
    pLayer(STATE)
addRoundKey(STATE,K_{32})

Full-LUT Methodology.

To the best of our knowledge, this is the first attempt to evaluate PRESENT in the homomorphic domain. Note that the Sbox of PRESENT is a 4-bit Sbox, so there is no need for the TBM to evaluate it with our framework. Indeed, as we work with 4-bit payload ciphertext, a classical bootstrapping is sufficient to evaluate the Sbox on encryptions of nibbles. This is great, as the evaluation of the Sbox on an 8-bit payload ciphertext then costs only 2 BR, instead of 3 BR and 2 KS for an 8-bit Sbox. That is to say, it takes about 0.06 seconds to evaluate instead of about 0.23 seconds. The really difficult part to process is the so-called *pLayer* of PRESENT. This pLayer

is a bit permutation that is hard to compute with 4-bit chunks. In fact, the most efficient way to homomorphically compute it with 4-bit payload ciphertexts is to decompose them into binary ciphertexts, perform the permutation on these binary ciphertexts and then recompose the nibbles. Thus, the permutation itself can be seen as "free" (we only move the binary ciphertexts), but we have to compute several decompositions and recompositions. A decomposition (DC) takes 1 BR and 0 KS per nibble, and a recomposition (RC) takes 4 BR and 0 KS. These are rather cheap operations as they do not rely on TBM, but only on simple bootstrapping.
So one round thus costs 16 homomorphic XORs, 16 4-bit Sbox evaluations and 16 DC/RC to compute the pLayer. That is to say, one round costs $16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (0\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) = 128\ \mathsf{BR}$ and 16 KS. As there are 31 rounds, a complete execution costs 3968 BR and 288 KS and takes about 154 seconds. But as it only gives a 64-bit results, we have to compute the full execution twice to obtain 128 bits.

Hippogryph Approach.

With the Hippogryph framework, only the Sbox is computed with TFHE's programmable bootstrapping. So we have to recompose 4-bit payload ciphertexts to evaluate the Sbox and then to decompose them to perform the pLayer and the XOR with the round keys. Note that even in the full-LUT paradigm, we have to perform these decompositions and recompositions to efficiently perform the pLayer.
Thus one round costs $16 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) = 96$ BR and 0 KS. Thus a complete evaluation requires 2976 BR and 0 KS. Estimated timings are given in Table 6.1.

Concluding Remarks.

What we learn from PRESENT is that to be efficient in the homomorphic domain, a cipher must have a good trade-off between the number of operations per rounds and the number of rounds. For instance, here a round is really light, even in the homomorphic domain (it takes less than 5 seconds in full-LUT version, and less than 3 seconds with Hippogryph), but we have to repeat it 31 times. If it was only 10 such rounds –as for the AES–, it would cost 1280 BR and 160 KS (about 50 seconds) in full-LUT mode, or 960 BR and 0 KS (about 29 seconds) with the Hippogryph framework.

6.2.3 PRINCE

Specification.

PRINCE [BCG+12] is a 64-bit block cipher that uses a 128-bit key. The key is divided into two parts of 64 bits each, represented as $k = k_0 || k_1$ (with $||$ the symbol for concatenation), and is extended to 192 bits through the mapping $(k_0 || k_1) \rightarrow$

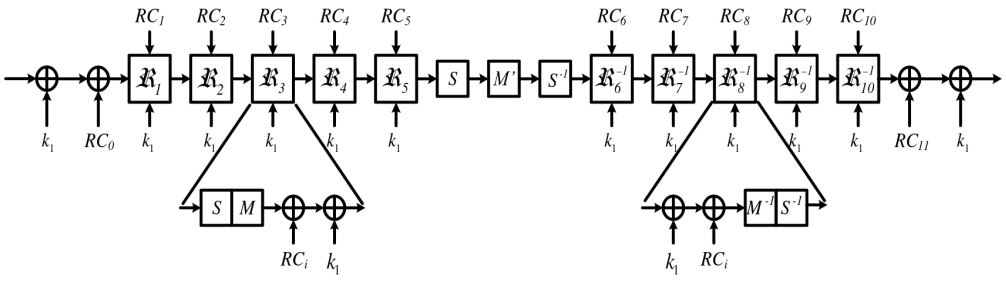


Figure 6.3: PRINCE$_{core}$, reproduced from [BCG+12].

$(k_0||k_0'||k_1) \coloneqq (k_0||(k_0 \gg 1) \oplus (k_0 \ll 63)||k_1)$. The first two subkeys, k_0 and k_0', serve as whitening keys, while the key k_1 is the 64-bit key for a 12-round block cipher known as PRINCE$_{core}$. Each round of PRINCE$_{core}$ includes a key addition, an S-box layer, a linear layer, and the addition of a round constant. Figure 6.3 presents the PRINCE$_{core}$ features. During the key addition phase, the 64-bit state is XORed with the 64-bit key k_1. The S-layer utilizes a single 4-bit S-box. The cipher incorporates M and M' layers, where the 64-bit state is multiplied by a 64×64 bits matrix M (or M'). Lastly, in the Round Constant addition step, a 64-bit round constant RC is XORed with the state.

Full-LUT Methodology.

PRINCE has already been studied for homomorphic purposes in the past. For instance, Doröz et al. [DSES14] proposed an implementation relying on a generalization of NTRU that took 57 minutes to execute. More recently, Bendoukha et al. [BSS+23] also studied the PRINCE block cipher. For an optimized full-LUT evaluation, the original PRINCE algorithm must be carefully analyzed. The first intuition when looking at the algorithm is that the two XORs (one of the state with a round constant, and one with a round key) can be factorized into one single XOR. To do so, the client can compute the XOR of the round constants with the corresponding round keys, obtaining a 64-bit entitie that we denote $RCk_i = k_1 \oplus RC_i$. As these computations are done in the cleartext space, this can be done very efficiently. Then instead of sending FHE encryptions of the round keys to the server, the client will send FHE encryptions of the result of the XOR of the round keys with the round constants, which represents the same amount of data. Lastly, we remind that before the execution of PRINCE$_{core}$, the 64-bit state must be XORed with the 64-bit key k_0, and at the end of PRINCE$_{core}$, the resulting 64-bit state is XORed with the 64-bit key k_0'. Note that these additional XORs do not impact our homomorphic execution of the framework. Indeed, as the first XOR with k_0 directly precedes the first XOR of PRINCE$_{core}$, the client can precompute $RCk_0' = k_0 \oplus k_1 \oplus RC_0$ to replace RCk_0 and send it to the server along with the other RCk_i. Similarly, as the XOR

with the 64-bit key k'_0 is the last operations and directly follows the XOR of the state with RCk_{11}, the client can precompute $RCk'_{11} = k'_0 \oplus k_1 \oplus RC_{11}$ and send it to the server instead of RCk_{11}. With this small optimization, the number of homomorphic XOR remains the same, and we even managed to decrease the number of FHE encryptions to send to the server.

Then, what remains to do is to transform the linear layer, consisting of a permutation similar to the AES ShiftRows and a matrix multiplication into a succession of LUTs. As the permutation permutes nibble, it is free in our design. So the one thing that really needs to be translated into LUTs is the matrix multiplication. We only propose here the most efficient solution we found, but several other options exist.
We note $c = (c_0c_1c_2c_3c_4c_5c_6c_7c_8c_9c_{10}c_{11}c_{12}c_{13}c_{14}c_{15})$ the encryption of the 64-bit state where each c_i is the encryption of a nibble. We want to compute the multiplication of c with M', with M' a 64×64 diagonal matrix which diagonal blocks are $(\hat{M}^{(0)}, \hat{M}^{(1)}, \hat{M}^{(1)}, \hat{M}^{(0)})$.
To do so, we create from the 16×16 bits matrices $\hat{M}^{(0)}$ and $\hat{M}^{(1)}$ the 16×16 bits matrices $\tilde{M}^{(0)}$ and $\tilde{M}^{(1)}$ by computing $\tilde{c}_{i,j} = c_i \wedge (15 - 2^j)$ for $i \in \{0, 1, \cdots, 15\}$ and $j \in \{0, 1, 2, 3\}$. Figure 6.4 shows $\hat{M}^{(0)}$ and $\tilde{M}^{(0)}$.
For instance, with such matrices, the evaluation of the multiplication on c_0 is calculated as follows:

$$\bar{c}_0 = \tilde{c}_{0,0} \oplus \tilde{c}_{1,1} \oplus \tilde{c}_{2,2} \oplus \tilde{c}_{3,3}.$$

That is to say, to obtain the encryption of the evaluation of the matrix multiplication on a nibble, we have to compute the corresponding $\tilde{c}_{i,j}$ and perform three homomorphic XORs. Each $\tilde{c}_{i,j}$ is computed once, with a simple bootstrapping. Note that to compute all $\tilde{c}_{i,i}$, $\tilde{c}_{i,(i+1) \pmod 4}$, $\tilde{c}_{i,(i+2) \pmod 4}$, $\tilde{c}_{i,(i+3) \pmod 4}$, we have to bootstrap with selector c_i. This means we can factorize these bootstrappings with the MVB an obtain four $\tilde{c}_{i,j}$ at the cost of only one BR. This way, the complete multiplication requires one BR per nibble to compute the new matrices (that is to say 16 BR in total) and then three XORs per nibble to actually perform the multiplication (i.e. $3 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 6\ \mathsf{BR}$ and $3\ \mathsf{KS}$ per nibble). So, a complete multiplication requires 112 BR and 48 KS.
A full round composed of the Sbox, the linear layer and the XOR with the new round keys then costs 160 BR and 64 KS. The whole decryption process of PRINCE comprises ten such rounds, one XOR at the beginning of the process and one at the end of the process, and two Sbox evaluation separated by a matrix multiplication (as shown in Figure 6.3), costing 1808 BR and 720 KS.

Hippogryph Approach.

With the Hippogryph framework, we only need the 4-bit payload ciphertexts for the evaluation of the Sbox. We perform multiplications and XORs with binary

$$\hat{M}^{(0)} = \begin{bmatrix}
0\,0\,0\,0 & 1\,0\,0\,0 & 1\,0\,0\,0 & 1\,0\,0\,0 \\
0\,1\,0\,0 & 0\,0\,0\,0 & 0\,1\,0\,0 & 0\,1\,0\,0 \\
0\,0\,1\,0 & 0\,0\,1\,0 & 0\,0\,0\,0 & 0\,0\,1\,0 \\
0\,0\,0\,1 & 0\,0\,0\,1 & 0\,0\,0\,1 & 0\,0\,0\,0 \\
\\
1\,0\,0\,0 & 1\,0\,0\,0 & 1\,0\,0\,0 & 0\,0\,0\,0 \\
0\,0\,0\,0 & 0\,1\,0\,0 & 0\,1\,0\,0 & 0\,1\,0\,0 \\
0\,0\,1\,0 & 0\,0\,0\,0 & 0\,0\,1\,0 & 0\,0\,1\,0 \\
0\,0\,0\,1 & 0\,0\,0\,1 & 0\,0\,0\,0 & 0\,0\,0\,1 \\
\\
1\,0\,0\,0 & 1\,0\,0\,0 & 0\,0\,0\,0 & 1\,0\,0\,0 \\
0\,1\,0\,0 & 0\,1\,0\,0 & 0\,1\,0\,0 & 0\,0\,0\,0 \\
0\,0\,0\,0 & 0\,0\,1\,0 & 0\,0\,1\,0 & 0\,0\,1\,0 \\
0\,0\,0\,1 & 0\,0\,0\,0 & 0\,0\,0\,1 & 0\,0\,0\,1 \\
\\
1\,0\,0\,0 & 0\,0\,0\,0 & 1\,0\,0\,0 & 1\,0\,0\,0 \\
0\,1\,0\,0 & 0\,1\,0\,0 & 0\,0\,0\,0 & 0\,1\,0\,0 \\
0\,0\,1\,0 & 0\,0\,1\,0 & 0\,0\,1\,0 & 0\,0\,0\,0 \\
0\,0\,0\,0 & 0\,0\,0\,1 & 0\,0\,0\,1 & 0\,0\,0\,1
\end{bmatrix} \quad \rightarrow \quad \tilde{M}^{(0)} = \begin{bmatrix}
\tilde{c}_{0,0} & \tilde{c}_{0,1} & \tilde{c}_{0,2} & \tilde{c}_{0,3} \\
\tilde{c}_{1,1} & \tilde{c}_{1,2} & \tilde{c}_{1,3} & \tilde{c}_{1,0} \\
\tilde{c}_{2,2} & \tilde{c}_{2,3} & \tilde{c}_{2,0} & \tilde{c}_{2,1} \\
\tilde{c}_{3,3} & \tilde{c}_{3,0} & \tilde{c}_{3,1} & \tilde{c}_{3,2}
\end{bmatrix}$$

Figure 6.4: $\hat{M}^{(0)}$ is part of the matrix multiplication of PRINCE. To implement it more efficiently, we transform it into the matrix $\tilde{M}^{(0)}$.

ciphertexts. As we perform twelve evaluation of the Sbox; we have to perform twelve RC and twelve DC, as well as 16 BR per Sbox evaluation. That is to say, we have to compute $12 \times 16 \times (5\ \mathsf{BR}, 0\ \mathsf{KS}) + 12 \times (16\ \mathsf{BR}, 0\ \mathsf{KS}) = 1152\ \mathsf{BR}$ and $0\ \mathsf{KS}$. Execution timings are given in Table 6.1.

Concluding Remarks.

Once again, there was an easy way to optimize the algorithm by factorizing the XORs of the round keys with those of the round constants, as well as the pre and post PRINCE$_{core}$ XORs. Also, there exist several ways to transform an operation into a succession of LUTs, so we have to try several times before we find the most optimized one. Finally, as for PRESENT, the 4-bit Sbox is a considerable advantage as it does not require the use of the TBM.

6.2.4 SIMON

Specification.

The SIMON block cipher (ISO/IEC 29167-21 standard) [BTCS+15] operates on pairs of n-bit words, which results in a total block size of $2n$ bits. The value of n can be 16, 24, 32, 48, or 64 bits. For our comparison with AES, we will use 64-bit words.

SIMON utilizes a Feistel structure that only involves left circular shifts (denoted S^i for a shift of an index i), XOR operations, and AND operations on n-bit words. For the 64-bit version, the encryption process requires the execution of 68 rounds, as summarized in Figure 6.5. As SIMON does not require any Sbox evaluation, the full-LUT approach will not perform well. Several attempts to evaluate SIMON in the homomorphic domain can be cited. For instance, a few years ago, Lepoint and Naehrig [LN14] proposed the first implementation in leveled mode, which achieved a 12 minutes latency [LN14]. More recently, Bon et al. [BPR24] studied SIMON and other primitives as use cases for the *p-encodings* approach. They report a 10

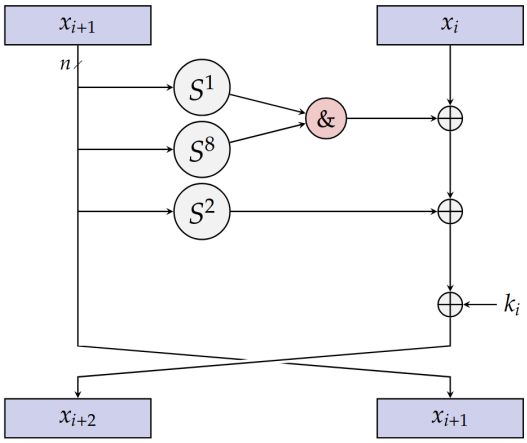


Figure 6.5: Feistel stepping of the SIMON round function extracted from [BTCS+15].

seconds latency.

Full-LUT Methodology.

With the full-LUT approach, homomorphic AND evaluations are conducted in a manner similar to homomorphic XOR evaluations, but utilizing different tables. However, implementing circular shifts poses a greater challenge.

To clarify, for circular shifts, also called rotations, bits that are "shifted out" are reinserted either at the beginning or the end of the word, depending on the direction of the shift. When it comes to executing cleartext-ciphertext rotations (that is to say rotating an encrypted value using a cleartext index), we can optimize the operation on a case-by-case basis. For example, a shift by 4 or any multiple of 4 is nearly free to compute since it involves shifting the entire 4-bit payload ciphertext. In contrast, shifting by an index that is not a multiple of 4 requires additional effort. In the work presented in [TCB+25], the authors propose a set of operations that leverage TFHE's programmable bootstrapping, including an implementation for circular shifts, which we adopt here. The proposed solution is designed for the encryption of bytes divided into two nibbles. To execute a circular shift on an encrypted byte, the authors present an algorithm that requires only 4 BR (two per nibble). One bootstrapping corresponds to the portion that "remains" within the nibble, and the other is for the part that is "shifted out" of the nibble. A SIMON round is composed of three left circular shifts S^1, S^2 and S^8. As 8 is a multiple of 4, we can perform it without any bootstrapping. But to shift with indexes 1 and 2, we have to use the algorithm discuss above. To shift the encryption of one byte, it takes 4 BR. Here, we work with words of 64 bits, that is to say 8 bytes. So to perform one circular shift on 8 bytes, we need to compute $8 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) = 32\ \mathsf{BR}$ and $0\ \mathsf{KS}$. So to perform S^1 and S^2 (recall that S^8 is free in our full-LUT framework), we have to perform 64 BR. In Section 5.3.2, we determined that performing a bitwise operand

on a encrypted nibble requires 2 BR and 1 KS. So performing the three XORs and the AND on SIMON's 64-bit words costs $4 \times 16 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 128\ \mathsf{BR}$ and 64 KS. A full round thus costs 192 BR and 64 KS. As there are 68 such rounds, a complete execution costs 13056 BR and 4352 KS. Execution timings are given in Table 6.1.

Hippogryph Approach.

Note that as SIMON does not rely on Sbox evaluation, there is no need to use 4-bit payload ciphertexts, thus no decomposition or recomposition are needed. So the Hippogryph framework is not really needed, and only some bootstrappings are performed to refresh the ciphertexts (as 68 rounds imply a circuit depth too large to handle with reasonable parameters). This of course induce a super fast evaluation of SIMON with binary ciphertexts for instance with the p-encoding method [BPR24]. For these reasons, we do not simulate the Hippogryph framework for SIMON.

Concluding Remarks.

If the block cipher used for transciphering does not rely on Sbox evaluation, then it is more advantageous to use binary-encoding ciphertexts and Boolean circuits approaches.

6.2.5 SKINNY

Specification.

The lightweight block ciphers of the SKINNY family (ISO/IEC 18033-7 standard) [BJK+16] have 64-bit and 128-bit block versions. In both $n = 64$ and $n = 128$ versions, the internal state is viewed as a 4×4 square array of cells, where each cell is a nibble (in the 64-bit case) or a byte (in the 128-bit case). In the 64-bit variant, 32 rounds are applied to the state, when 40 are applied in the 128-bit variant. Another difference between the two variants is the Sbox. In the 128-bit variant, the authors propose to use an 8-bit Sbox, and for the 64-bit variant they propose a 4-bit Sbox. One round is composed of five operations: an Sbox, two XORs (one with a round constant and one with a round tweakey), a shift of the rows of the state and a matrix multiplication.

Full-LUT Methodology.

To the best of our knowledge, this is the first attempt to evaluate SKINNY in the homomorphic domain. In both variants, we optimize the algorithm by factorizing the two different XORs being applied to the state. As for PRINCE, the client pre-computes the XOR of each round constant with the corresponding keys and send these modified keys to the server. This saves us several bootstrappings.

As with other block ciphers, the most difficult part is to convert the matrix multiplication into a succession of LUT evaluations. But in the SKINNY case, the matrix is a binary matrix, meaning that we only have to correctly XOR the corresponding values to perform the matrix multiplication. More precisely, to obtain four nibbles (resp. four bytes), we have to compute four XORs. This means that the whole multiplication only cost 16 XORs. It is the cheapest matrix multiplication so far.
In the 64-bit case (resp. 128-bit case), the multiplication thus costs $16 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 32\ \mathsf{BR}$ and $16\ \mathsf{KS}$ (resp. $16 \times 2 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 64\ \mathsf{BR}$ and $32\ \mathsf{KS}$). For the Sbox step, we have to perform $16\ \mathsf{BR}$ in the 64-bit version, against $48\ \mathsf{BR}$ and $32\ \mathsf{KS}$ in the 128-bit version. Finally, homomorphically compute the XOR layer requires $16 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 32\ \mathsf{BR}$ and $16\ \mathsf{KS}$ for the 64-bit variant and $16 \times 2 \times (2\ \mathsf{BR}, 1\ \mathsf{KS}) = 64\ \mathsf{BR}$ and $32\ \mathsf{KS}$ for the 128-bit variant.
Thus, a full round of SKINNY-64 costs $80\ \mathsf{BR}$ and $32\ \mathsf{KS}$, and a full round of SKINNY-128 costs $176\ \mathsf{BR}$ and $96\ \mathsf{KS}$. Even if we have to perform two full execution of SKINNY-64 to obtain 128 encrypted bits, it is less costly to run in the homomorphic domain than one complete execution of SKINNY-128. Execution timings are given in Table 6.1.

Hippogryph Approach.

Note that as with PRINCE, the optimization of the factorization of XORs also benefits the Hippogryph framework. Indeed, it allows a smaller depth of Boolean circuit to be performed, and thus smaller parameters.
Still, the Hippogryph framework requires 4-bit payload ciphertexts to compute the SBox with PBS. So the only bootstrappings to perform are those corresponding to the decomposition of ciphertexts (DC), recomposition of ciphertexts (RC) and Sbox evaluation. There is one Sbox evaluation per round regardless of the variant being executed. So, for one round in the 128-bit case, we have to compute $16 \times 2 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times 2 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (3\ \mathsf{BR}, 2\ \mathsf{KS}) = 208\ \mathsf{BR}$ and $32\ \mathsf{KS}$. One round of the 64-bit version costs $16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (4\ \mathsf{BR}, 0\ \mathsf{KS}) + 16 \times (1\ \mathsf{BR}, 0\ \mathsf{KS}) = 96$ BR and $0\ \mathsf{KS}$. These are quite light rounds, but to perform the complete execution, we have to repeat them 40 times for the 128-bit version, and 2×32 times to obtain 128 bits with the 64-bit version.

Concluding Remarks.

An interesting fact to note is that, with both variants, the Hippogryph framework requires more BR than in the full-LUT paradigm, but remains more efficient (between 0.6× and 0.7× faster than the full-LUT method). It is due to the small number of KS required. While one BR takes about 0.03 seconds, one KS takes about 0.07 seconds which is almost twice as the execution time of one BR. So the greater number of BR in the Hippogryph Approach is cushioned by the absence of KS, which ultimately makes the Hippogryph approach more interesting. We also note that

the matrix multiplication costs is very small compared to other block ciphers. And finally, it is very interesting to see that to perform twice the 64-bit variant is almost 2× faster than one execution of the 64-bit variant. As the operations are the same (XORs, Sbox, Shifts,...), the explanation comes from the fact that they are applied on nibbles and not on bytes, which makes them way much faster (very fewer BR, and almost no KS!).

6.3 Results

Our results with the full-LUT approach, presented in Table 6.1, are obtained with a 12th Gen Intel(R) Core(TM) i7-12700H CPU laptop with 64 Gib total system memory with an Ubuntu 22.04.2 LTS server, using a single core. The estimated timings for the Hippogryph framework are given in the same table, for the same set of parameters. With our generic full-LUT method, we show that AES is not the fastest symmetric block cipher. Indeed, PRINCE is faster with both the Full-LUT approach and the Hippogryph framework. Note that the Hippogryph framework only, CLEFIA is also faster than the AES.

6.3.1 Several Categories of Instructions

Using these executions timings and estimated timings, we can draw several additional conclusions. For instance, based on our execution times, we have identified which operations are relatively fast in the homomorphic domain and which are more costly. FHE-friendly features include working on sub-block messages of 4 or 8 bits rather than directly binary messages. In particular, when shifting the rows of the state matrix, we prefer shifts with an index multiple of 4 (and therefore free in our FHE design). A trade-off must also be found between the number of bootstrapping operations per round and the total number of rounds. Finally, a 4-bit Sbox is more than two times faster than an 8-bit Sbox. Note that this final FHE-friendly feature as been recently highlighted in [BBB+25a], where the authors propose a new stream cipher especially designed to be efficient for transciphering with TFHE.

On the FHE-unfriendly side, we have 8-bit Sboxes, lots of ciphertext-to-ciphertext operations (more costly than univariate operations, for instance, the multiplication by 2 in $GF(256)$ in AES' MixColumns), index shifts that are not multiples of 4, a very large number of rounds... We also emphasize one advantage of block ciphers (compared to stream ciphers): the possibility of parallelizing computations. Indeed, for instance with AES calculations can be performed on each byte separately (except for MixColumns), as with PRINCE, PRESENT, and SKINNY. As for Feistel-type block ciphers, such as SIMON and CLEFIA, we can parallelize by the number of words. We have not used this advantage here, as we are mainly interested in comparing the sequential execution times of different schemes.

Cryptosystems	Nb bits in state	nb executions to obtain 128 bits	Full-LUT			Hippogryph		
			$\mathsf{N_{BR}}$	$\mathsf{N_{KS}}$	timings to obtain 128 bits	$\mathsf{N_{BR}}$	$\mathsf{N_{KS}}$	timings to obtain 128 bits
AES	128	1	3776	2112	260	2080	320	84
PRINCE	64	2	1808	1220	210	1152	0	70
PRESENT	64	2	4496	992	408	2976	0	179
SKINNY-128	128	1	7040	3840	480	8192	1280	335
SKINNY-64	64	2	2560	1024	298	3008	0	180
CLEFIA	128	1	5120	2848	353	1872	288	77
SIMON	128	1	18496	4080	840	-	-	-
Unicorn	128	1	1600	640	93	1920	0	58

Table 6.1: Comparison of different block ciphers running over TFHE. Two methods are proposed. Timings are in seconds.

6.3.2 A Fantasy FHE-friendly Block Cipher

Finally, this categorization of block ciphers' operations allows us to highlight the ideal building blocks for a hypothetical Unicorn Cipher that would outperform other block ciphers in the homomorphic domain:

- Subsitution layer: evaluation of a 4-bit Sbox on the state (for example PRINCE's Sbox)
- Permutation layer: SKINNY's MixColumns that only requires XORs (no multiplications).
- Linear layer: AES's ShiftRows that shifts from indexes that are multiples of 4.
- Round Key addition: a simple XOR of the state with the corresponding round key. Note that we can also add a Round Constant Addition operation for free if we factorize it with the Round Key Addition.

Without professing any symmetric encryption security, a 128-bit block size Unicorn Cipher which round tours would be composed of these four operations would only cost 160 BR and 64 KS per round with the full-LUT approach, and 192 BR and 0 KS with the Hippogryph framework. Last line of Table 6.1 resumes the number of BR and KS needed for such a Unicorn cipher that would require ten rounds to execute. We also propose the estimated timings for the two approaches.

Interestingly, the recent Transistor FHE-friendly stream-cipher design [BBB+25a] independently follows guidelines which are consistent with the above ones.

6.4 Conclusion

This work shows that, with two different approaches, there exist standardized block ciphers that can be more efficiently used for transciphering than AES (namely PRINCE with the full-LUT methodology and PRINCE and CLEFIA with the Hippogryph framework). But above all, this work points to two avenues of research that could contribute to efficient transciphering (and more) over TFHE:

- Focus on the operations categorized here as "FHE-unfriendly" and create more efficient FHE operators for them.
- Investigate whether secure block ciphers can be built using only the "FHE-friendly" building blocks that we identified.

Finally, let us emphasize that the method to be used for transciphering does not necessarily have to be the most efficient. Indeed, it should also depend on the type of homomorphic calculations that are being performed after the transciphering. For instance, functional bootstrapping methods generally produce ciphertexts with integer-based encodings, while circuit bootstrapping uses bit encodings. Besides minor latency differences, the choice between integer and bit outputs has a significant impact on further homomorphic evaluation, particularly regarding compatibility with arithmetic versus Boolean circuits. This trade-off indicates that the best approach may depend on the specific use case and the types of subsequent operations involved.

Conclusion

During these three years, I have had the chance to work on a fascinating subject. From the little I knew about homomorphic encryption before my internship, I now find myself rubbing shoulders with the experts in the field. Looking back at the results of my clumsy first steps with TFHE, I can see just how far I have come. What an adventure!

I am delighted to see how my knowledge and understanding of homomorphic encryption have evolved. The contributions of this thesis retrace this journey. It has barely been discussed in this manuscript, but I took my very first steps in the world of research when, following my internship's subject, I tried to perform a homomorphic evaluation of somewhat complicated activation functions with TFHE, without impacting the accuracy of the neural network. The first weeks of my internship were dedicated to learning about the world of AI, with which I was completely unfamiliar. Today, I am still a long way from proposing the next version of ChatGPT, but the knowledge I had acquired was sufficient to finally take a look at homomorphic encryption. When discovering the TFHE scheme, a solution gradually took shape in my mind to evaluate these activation functions. The only problem was that the semantics used did not allow for further evaluation of the neuron. This was the first hurdle along the way and the first time I was challenged as a researcher.

I can confess, and it is no secret to my supervisors, that I was very pleased to leave neural networks and AI behind me when I started my thesis. At the time, the official title was "Optimized and Efficient Homomorphic Transciphering". A question a friend once asked me perfectly reflects the issues I have been working with during this thesis: "But isn't that a bit redundant? Optimized AND efficient?". To which I replied: "No, with homomorphic encryption, you can totally be optimized without being efficient.". And indeed, that is the conclusion of the first contribution of this manuscript, which provides a homomorphic AES execution in 4 minutes. Our method was the most advanced in the state-of-the-art at the time the article was published, and yet the execution time obtained was still counted in minutes! However, it did not matter because these results enabled us to demonstrate, above all, the effectiveness of TFHE's functional bootstrapping. From there, we explored

the idea of a software instruction set that would rely solely on bootstrapping and be universal enough to enable the uninitiated to implement any algorithm on encrypted data. Not only have we succeeded in creating such an instruction set, but we have also shown that it is competitive with Boolean circuit approaches. In particular, as soon as the number of logic gates corresponding to the algorithm to be executed becomes large (i.e., as soon as multiplication comes into play), our approach becomes more efficient. On top of this, we have proposed several instructions that are somewhat unheard of in the homomorphic domain, such as division or modulo calculation. The icing on the cake? Thanks to the instruction set and our optimized bootstrapping operators, we could propose a new evaluation of the sigmoid, better in terms of precision than what I had achieved in my internship, and also enabling us to evaluate an *entire* neural network. It felt like revenge for the long hours spent listening to videos about AI and a nice conclusion to my internship at the same time.

Once our work on `Mad Panthera` was over, we decided to take a look at the "competition" for the fastest AES homomorphic evaluation. Indeed, following the NIST call on threshold encryption with its dedicated part to FHE, a rule has been implicitly established within the community. Whoever has the most efficient homomorphic AES evaluation is the one who proposes the most advanced and efficient homomorphic techniques. Since our first contribution, barely a year earlier, several teams had reduced the execution time to seconds. Caught up in the game, we teamed up with CryptoExperts to return to the competition, and successfully so! With our new Hippogryph approach, we proposed an evaluation in only 32 seconds (and almost 1 second with a little parallelization). So, for now, the crown is French, but for how long? At the speed at which advances are being made in FHE, I am sure someone else will possess the "record" at the time I will be defending.

Still, using `Mad Panthera` and Hippogryph, I could refocus on transciphering, which led me to the results proposed in the last contribution of this manuscript. By studying different standardized block ciphers, we were able to show what the ideal block cipher for TFHE transciphering might look like. Of course, as I am no expert in symmetric cryptography, one of the prospects of this work would be to collaborate with an expert team to bring our Unicorn cipher to life.

But this is not the only perspective opened up by this thesis. Indeed, the `Mad Panthera` contribution, in particular, opens the way to exciting possibilities. Ideally, we would like to join forces with a team of hardware experts to propose a hardware optimization of the processor, which would then no longer be a simple set of software instructions. We have already taken a few steps in this direction by organizing a one-day workshop with hardware team members in May. The discussions have been very enthusiastic, and I hope to participate in further developing this project.

In conclusion, these three years have been a wonderful introduction to the re-

search world. I have added my little stone to the edifice, and there are many prospects for transforming this little pebble into a magnificent cathedral. And I really look forward to it.

Bibliography

[ABB+16] J. Bacelar Almeida, M. Barbosa, G. Barthe, F. Dupressoir, and M. Emmi. Verifying Constant-Time Iplementations. In *25th USENIX Security Symposium (USENIX Security 16)*, 2016.

[ABD16] M. Albrecht, S. Bai, and L. Ducas. A Subfield Lattice Attack on Overstretched NTRU Assumptions. In *CRYPTO*, page 153–178, 2016.

[ABG+24] H. Asi, F. Boemer, N. Genise, M. Haris Mughees, T. Ogilvie, R. Rishi, G. N. Rothblum, K. Talwar, K. Tarbe, R. Zhu, and M. Zuliani. Scalable Private Search with Wally, 2024.

[ACS20] P. Aubry, S. Carpov, and R. Sirdey. Faster Homomorphic Encryption is not Enough: Improved Heuristic for Multiplicative Depth Minimization of Boolean Circuits. In *Topics in Cryptology – CT-RSA 2020*, 2020.

[Ajt05] M. Ajtai. Representing hard lattices with o(n log n) bits. In *ACM Symposium on Theory of Computing*, page 94–103, 2005.

[AKP25] A. Alexandru, A. Kim, and Y. Polyakov. General Functional Bootstrapping using CKKS. In *CRYPTO*, 2025.

[AMDG25] C. Aguilar-Melchor, V. Dyseryn, and P. Gaborit. Somewhat Homomorphic Encryption based on random codes. *Designs, Codes and Cryptography*, 2025.

[AMT22] T. Ashur, M. Mahzoun, and D. Toprakhisar. Chaghri - a FHE-Friendly Block Cipher. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, 2022.

[ARS+15] M. R. Albrecht, C. Rechberger, T. Schneider, T. Tiessen, and M. Zohner. Ciphers for MPC and FHE. In *EUROCRYPT*, 2015.

[ASP14] J. Alperin-Sheriff and C. Peikert. Faster Bootstrapping with Polynomial Error. In *Advances in Cryptology – CRYPTO*, 2014.

[BBB+23a] L. Bergerat, A. Boudi, Q. Bourgerie, I. Chillotti, D. Ligier, J.-B. Orfila, and S. Tap. Parameter Optimization and Larger Precision for (T)FHE. *Journal of Crypto*, 2023.

[BBB+23b] L. Bergerat, A. Boudi, Q. Bourgerie, I. Chillotti, D. Ligier, J-B. Orfila, and S. Tap. Parameter Optimization and Larger Precision for (T)FHE. *Journal of Cryptology*, 36, 2023.

[BBB+25a] J. Baudrin, S. Belaïd, N. Bon, C. Boura, A. Canteaut, G. Leurent, P. Paillier, L. Perrin, M. Rivain, Y. Rotella, and S. Tap. Transistor: a TFHE-friendly Stream Cipher. Cryptology ePrint Archive, Paper 2025/282, 2025.

[BBB+25b] S. Belaïd, N. Bon, A. Boudguiga, R. Sirdey, D. Trama, and N. Ye. Further Improvements in AES Execution over TFHE: Towards Breaking the 1 sec Barrier. *Communications in Cryptology*, 2025.

[BBS22] A.-A. Bendoukha, A. Boudguiga, and R. Sirdey. Revisiting Stream-Cipher-Based Homomorphic Transciphering in the TFHE Era. In *Foundations and Practice of Security: 14th International Symposium, FPS*, page 19–33, 2022.

[BBVY21] S. Banik, K. Barooti, S. Vaudenay, and H. Yan. New Attacks on LowMC Instances with a Single Plaintext/Ciphertext Pair. In *ASIACRYPT*, pages 303–331, 2021.

[BCBS23] A.-A. Bendoukha, P.-E. Clet, A. Boudguiga, and R. Sirdey. Optimized Stream-Cipher-Based Transciphering by Means of Functional-Bootstrapping. In *Data and Applications Security and Privacy XXXVII: 37th Annual IFIP WG 11.3 Conference*, page 91–109, 2023.

[BCCS24] A. Bondarchuk, O. Chakraborty, G. Couteau, and R. Sirdey. Downlink (T)FHE ciphertexts compression. Cryptology ePrint Archive, Paper 2024/1921, 2024.

[BCF+25] C. Brzuska, S. Canard, C. Fontaine, D. H. Phan, D. Pointcheval, M. Renard, and R. Sirdey. Relations Among New CCA Security Notions for Approximate FHE. *IACR Communications in Cryptology*, 2025.

[BCG+12] J. Borghoff, A. Canteaut, T. Güneysu, E. Bilge Kavun, M. Knezevic, L. Ramkilde Knudsen, G. Leander, V. Nikov, C. Paar, C. Rechberger, P. Rombouts, S. S. Thomsen, and T. Yalçin. PRINCE - a Low-Latency

Block Cipher for Pervasive Computing Applications. In *International Conference on the Theory and Application of Cryptology and Information Security*, 2012.

[BCH+24] Y. Bae, J. H. Cheon, G. Hanrot, J. H. Park, and D. Stehlé. Plaintext-Ciphertext Matrix Multiplication and FHE Bootstrapping: Fast and Fused. In *CRYPTO*, page 387–421, 2024.

[BCL+24] L. Bergerat, I. Chillotti, D. Ligier, J.-B. Orfila, A. Roux-Langlois, and S. Tap. New Secret Keys for Enhanced Performance in (T)FHE. In *ACM SIGSAC Conference on Computer and Communications Security*, page 2547–2561, 2024.

[BDGM19] Z. Brakerski, N. Döttling, S. Garg, and G. Malavolta. Leveraging Linear Decryption: Rate-1 Fully-Homomorphic Encryption and Time-Lock Puzzles. In *TCC*, page 407–437, 2019.

[BGV12] Z. Brakerski, C. Gentry, and V. Vaikuntanathan. (leveled) Fully Homomorphic Encryption without Bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. Association for Computing Machinery, 2012.

[BIP+22] C. Bonte, I. Iliashenko, J. Park, H. V. L. Pereira, and N. P. Smart. FINAL: Faster FHE Instantiated with NTRU and LWE. 2022.

[BJK+16] C. Beierle, J. Jean, S. Kölbl, G. Leander, A. Moradi, Thomas Peyrin, Y. Sasaki, P. Sasdrich, and S. M. Sim. The SKINNY Family of Block Ciphers and its Low-Latency Variant MANTIS. In *CRYPTO*, pages 123–153, 2016.

[BKL+07] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin, and C. Vikkelsoe. PRESENT: An Ultra-Lightweight Block Cipher. In Pascal Paillier and Ingrid Verbauwhede, editors, *Cryptographic Hardware and Embedded Systems - CHES 2007*, pages 450–466, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[BMMP18] F. Bourse, M. Minelli, M. Minihold, and P. Paillier. Fast Homomorphic Evaluation of Deep Discretized Neural Networks. In *CRYPTO*, page 483–512, 2018.

[BOS23] T. Balenbois, J.-B. Orfila, and N. Smart. Trivial Transciphering with Trivium and TFHE. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC '23, page 69–78, 2023.

[BP10] J. Boyar and R. Peralta. A new Combinational Logic Minimization Technique with Applications to Cryptology. In *Experimental Algorithms, 9th International Symposium, SEA*, pages 178–189, 2010.

[BPR24] N. Bon, D. Pointcheval, and M. Rivain. Optimized Homomorphic Evaluation of Boolean Functions. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024.

[BPS12] M. Brenner, H. Perl, and M. Smith. How Practical is Homomorphically Encrypted Program Execution? An Implementation and Performance Evaluation. In *IEEE TrustCom*, page 375–382, 2012.

[Bra12] Z. Brakerski. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *Advances in Cryptology – CRYPTO 2012*, 2012.

[BSS+23] A. A. Bendoukha, O. Stan, R. Sirdey, N. Quero, and L. Freitas. Practical Homomorphic Evaluation of Block-Cipher-Based Hash Functions with Applications. In *Foundations and Practice of Security: 15th International Symposium, FPS 2022, Ottawa*, 2023.

[BTCS+15] R. Beaulieu, S. Treatman-Clark, D. Shors, B. Weeks, Jason S., and L. Wingers. The SIMON and SPECK lightweight block ciphers. In *2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2015.

[BV14] Z. Brakerski and V. Vaikuntanathan. Lattice-based FHE as secure as PKE. In *Proceedings of the 5th Conference on Innovations in Theoretical Computer Science*, ITCS '14, 2014.

[CBSZ23] P.-E. Clet, A. Boudguiga, R. Sirdey, and M. Zuber. *ComBo: A Novel Functional Bootstrapping Method for Efficient Evaluation of Nonlinear Functions in the Encrypted Domain*. 2023.

[CCF+18] A. Canteaut, S. Carpov, C. Fontaine, T. Lepoint, M. Naya-Plasencia, P. Paillier, and R. Sirdey. Stream Ciphers: A Practical Solution for Efficient Homomorphic-Ciphertext Compression. In *Journal of Cryptography*, volume 31, page 885–916, 2018.

[CCP+24] J. H. Cheon, H. Choe, A. Passelègue, D. Stehlé, and E. Suvanto. Attacks Against the IND-CPAD Security of Exact FHE Schemes. In *ACM CCS*, page 2505–2519, 2024.

[CDS15a] S. Carpov, P. Dubrulle, and R. Sirdey. Armadillo: a compilation chain for privacy preserving applications. In *Proceedings of the 3rd International Workshop on Security in Cloud Computing*, 2015.

[CDS15b] S. Carpov, P. Dubrulle, and R. Sirdey. Armadillo: a compilation chain for privacy preserving applications. In *Proceedings of the 3rd International Workshop on Security in Cloud Computing*, pages 13–19, 2015.

[CGGI16] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast Fully Homomorphic Encryption Library, August 2016. https://tfhe.github.io/tfhe/.

[CGGI19] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène. TFHE: Fast Fully Homomorphic Encryption Over the Torus. *Journal of Cryptology*, 33, 2019.

[CGRS14] D. B. Cousins, J. Golusky, K. Rohloff, and D. Sumorok. An FPGA Co-processor Implementation of Homomorphic Encryption. In *IEEE HPEC*, pages 1–6, 2014.

[CHK+21] J. Cho, J. Ha, S. Kim, B. Lee, J. Lee, J. Lee, D. Moon, and H. Yoon. Transciphering Framework for Approximate Homomorphic Encryption. In *ASIACRYPT*, page 640–669, 2021.

[CHKS25] J. H. Cheon, G. Hanrot, J. Kim, and D. Stehlé. Ship: A Shallow and Highly Parallelizable CKKS Bootstrapping Algorithm. In *Advances in Cryptology – EUROCRYPT 2025*, pages 398–428, 2025.

[CHMS22] O. Cosseron, C. Hoffmann, P. Méaux, and F.-X. Standaert. Towards Case-Optimized Hybrid Homomorphic Encryption. In *ASIACRYPT*, 2022.

[CIM18] S. Carpov, M. Izabachène, and V. Mollimard. New techniques for Multi-value input Homomorphic Evaluation and Applications. In *Topics in Cryptology – CT-RSA 2019*, 2018.

[CIR22] C. Cid, J. P. Indrøy, and H. Raddum. FASTA – a Stream Cipher for Fast FHE Evaluation. In *CT-RSA*, page 451–483, 2022.

[CJP21] I. Chillotti, M. Joye, and P. Paillier. Programmable Bootstrapping Enables Efficient Homomorphic Inference of Deep Neural Networks. In *Cyber Security Cryptography and Machine Learning*, pages 1–19, 2021.

[CKKS17] J. H. Cheon, A. Kim, M. Kim, and Y. Song. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology – ASIACRYPT*, 2017.

[Cle24] P.-E. Clet. *Contributions to the optimization of TFHE's functional Bootstrapping for the evaluation of non-polynomial operators.* Theses, Université Paris-Saclay, January 2024.

[CLOT21] I. Chillotti, D. Ligier, J-B. Orfila, and S. Tap. Improved Programmable Bootstrapping with Larger Precision and Efficient Arithmetic Circuits for TFHE. In *ASIACRYPT*, page 670–699, 2021.

[CMG+18] E. Chielle, O. Mazonka, H. Gamil, N. G. Tsoutsos, and M. Maniatakos. E3: A Framework for Compiling C++ Programs with Encrypted Operands. Cryptology ePrint Archive, Report 2018/1013, 2018.

[CS19] A. Chatterjee and I. Sengupta. FURISC: FHE Encrypted URISC Design. In *Fully Homomorphic Encryption in Real World Applications*, pages 87–115, 2019.

[CSBB24] M. Checri, R. Sirdey, A. Boudguiga, and J.-P. Bultel. On the practical CPAD security of "exact" and threshold FHE Schemes. In *CRYPTO*, page 3–33, 2024.

[DCP06] Christophe De Cannière and B. Preneel. Trivium: A Stream Cipher Construction Inspired by Block Cipher Design Principles. In *Information Security*, pages 171–186, 2006.

[DEG+18] C. Dobraunig, M. Eichlseder, L. Grassi, V. Lallemand, G. Leander, E. List, F. Mendel, and C. Rechberger. Rasta: A Cipher with Low ANDdepth and Few ANDs per Bit. In *CRYPTO*, page 662–692, 2018.

[DEM15] C. Dobraunig, M. Eichlseder, and F. Mendel. Higher-Order Cryptanalysis of LowMC. In *ICISC*, pages 87–101, 2015.

[DEMS21] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer. Ascon v1.2: Lightweight Authenticated Encryption and Hashing. *Journal of Cryptology*, 34, 2021.

[DGH+23] C. Dobraunig, L. Grassi, L. Helminger, C. Rechberger, M. Schofnegger, and R Walch. PASTA: A Case for Hybrid Homomorphic Encryption. *IACR Trans. Cryptogr. Hardw. Embed. Syst. (TCHES)*, 2023.

[DGHV10] M. Dijk, C. Gentry, S. Halevi, and V. Vaikuntanathan. Fully Homomorphic Encryption over the Integers. In *Advances in Cryptology – EUROCRYPT 2010*, 2010.

[DLMW15] I. Dinur, Y. Liu, W. Meier, and Q. Wang. Optimized Interpolation Attacks on LowMC. In *ASIACRYPT*, page 535–560, 2015.

[DLR16] S. Duval, V. Lallemand, and Y. Rotella. Cryptanalysis of the FLIP Family of Stream Ciphers. In *CRYPTO*, page 457–475, 2016.

[DM15] L. Ducas and D. Micciancio. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *Advances in Cryptology – EUROCRYPT*, 2015.

[DR02] J. Daemen and V. Rijmen. *The Design of Rijndael: AES - The Advanced Encryption Standard*. Springer, 2002.

[DS09] I. Dinur and A. Shamir. Cube Attacks on Tweakable Black Box Polynomials. In *EUROCRYPT*, page 278–299, 2009.

[DSES14] Y. Doröz, A. Shahverdi, T. Eisenbarth, and B. Sunar. Toward Practical Homomorphic Evaluation of Block Ciphers Using Prince. In *Financial Cryptography and Data Security*, 2014.

[ElG85] T. ElGamal. A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms. In *CRYPTO*, pages 10–18, 1985.

[FSF+13] S. Fau, R. Sirdey, C. Fontaine, C. Aguilar-Melchor, and G. Gogniat. Towards Practical Program Execution over Fully Homomorphic Encryption Schemes. In *IEEE 3PGCIC*, pages 284–290, 2013.

[FV12] Junfeng Fan and Frederik Vercauteren. Somewhat Practical Fully Homomorphic Encryption. *IACR Cryptol. ePrint Arch.*, 2012.

[FV14] P.-A. Fouque and T. Vannet. Improving Key Recovery to 784 and 799 Rounds of Trivium Using Optimized Cube Attacks. In *Fast Software Encryption*, pages 502–517, 2014.

[GBA] A. Guimarães, E. Borin, and D. F. Aranha. Revisiting the functional bootstrap in TFHE. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(2):229–253.

[Gen09] C. Gentry. Fully Homomorphic Encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, STOC '09, page 169–178, 2009.

[GH11] C. Gentry and S. Halevi. Implementing Gentry's Fully-Homomorphic Encryption Scheme. In *Proceedings of the 30th Annual International Conference on Theory and Applications of Cryptographic Techniques: Advances in Cryptology*, EUROCRYPT'11, 2011.

[GHBJR23] H. Gilbert, R. Heim Boissier, J. Jean, and J.-R. Reinhard. Cryptanalysis of Elisabeth-4. In *ASIACRYPT*, page 256–284, 2023.

[GHS12] C. Gentry, S. Halevi, and N. P. Smart. Homomorphic Evaluation of the AES Circuit. In *Advances in Cryptology – CRYPTO 2012*, 2012.

[GMAH+23] L. Grassi, I. Manterola Ayala, M. N. Hovd, M. Øygarden, H. Raddum, and Q; Wang. Cryptanalysis of Symmetric Primitives over Rings and a Key Recovery Attack on Rubato. In *CRYPTO*, page 305–339, 2023.

[GMT24] C. Gouert, D. Mouris, and N. G. Tsoutsos. Juliet: A Configurable Processor for Computing on Encrypted Data. *IEEE Transactions on Computers*, pages 1–14, 2024.

[GN] X. Gong and D. Negrut. CryptoEmu: An Instruction Set Emulator for Computation Over Ciphers. Technical report.

[GSPH+21] S. Gorantala, R. Springer, S. Purser-Haskell, W. Lam, R. Wilson, A. Ali, E. P. Astor, I. Zukerman, S. Ruth, C. Dibak, P. Schoppmann, S. Kulankhina, A. Forget, D. Marn, C. Tew, R. Misoczki, B. Guillen, X. Ye, D. Kraft, D. Desfontaines, A. Krishnamurthy, M. Guevara, I. Milinda Perera, Y. Sushko, and B. Gipson. A General Purpose Transpiler for Fully Homomorphic Encryption. Cryptology ePrint Archive, Paper 2021/811, 2021.

[GSW13] C. Gentry, A. Sahai, and B. Waters. Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. In *Advances in Cryptology – CRYPTO*, 2013.

[HKC+20] J. Ha, S. Kim, W. Choi, J. Lee, D. Moon, H. Yoon, and J. Cho. Masta: An HE-Friendly Cipher Using Modular Arithmetic. *IEEE Access*, pages 194741–194751, 2020.

[HKL+22] J. Ha, S. Kim, B. Lee, J. Lee, and M. Son. Rubato: Noisy Ciphers for Approximate Homomorphic Encryption. In *EUROCRYPT*, page 581–610, 2022.

[HKX+25] M.-C. Ho, Y.-T. Ku, Y. Xiao, F.-H. Liu, C.-F. Hsu, M.-C. Chang, S.-H. Hung, and W.-C. Chen. Efficient Design of FHEW/TFHE Bootstrapping Iplementation with Scalable Parameters. In *IEEE/ACM International Conference on Computer-Aided Design*, 2025.

[HL20] P. Hebborn and G. Leander. Dasta – Alternative Linear Layer for Rasta. *IACR Transactions on Symmetric Cryptology*, pages 46–86, 2020.

[IMP18] F. Irena, D. Murphy, and S. Parameswaran. CryptoBlaze: A Partially Homomorphic Processor with Multiple Instructions and Nondeterministic Encryption Support. In *IEEE ASP-DAC*, page 702–708, 2018.

[Joy24] M. Joye. TFHE Public-Key Encryption revisited. In *Topics in Cryptology – CT-RSA*, page 277–291, 2024.

[KS22] K. Kluczniak and L. Schild. FDFB: Full Domain Functional Bootstrapping Towards Practical Fully Homomorphic Encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, page 670–699, 2022.

[LAW+23] F. Liu, R. Anand, L. Wang, W. Meier, and T. Isobe. Coefficient Grouping: Breaking Chaghri and More. In *EUROCRYPT*, 2023.

[LIM21] F. Liu, T. Isobe, and W. Meier. Cryptanalysis of Full LowMC and LowMC-M with Algebraic Techniques. In *CRYPTO*, page 368–401, 2021.

[LKSM24] Fukang Liu, Abul Kalam, Santanu Sarkar, and Willi Meier. Algebraic Attack on FHE-Friendly Cipher HERA Using Multiple Collisions. *IACR Transactions on Symmetric Cryptology*, page 214–233, 2024.

[LM21] B. Li and D. Micciancio. On the Security of Homomorphic Encryption on Approximate Numbers. In *EUROCRYPT*, page 648–677, 2021.

[LMSS23] C. Lee, S. Min, J. Seo, and Y. Song. Faster TFHE Bootstrapping with Block Binary Keys. In *ACM Asia Conference on Computer and Communications Security*, page 2–13, 2023.

[LN14] T. Lepoint and M. Naehrig. A Comparison of the Homomorphic Encryption Schemes FV and YASHE. In *Progress in Cryptology – AFRICACRYPT 2014*, pages 318–335, 2014.

[LSMI21] F. Liu, S. Sarkar, W. Meier, and T. Isobe. Algebraic Attacks on Rasta and Dasta Using Low-Degree Equations. In *ASIACRYPT*, page 214–240, 2021.

[LW23a] F.-H. Liu and H. Wang. Batch Bootstrapping i: A New Framework for SIMD Bootstrapping in Polynomial Modulus. In *EUROCRYPT*, 2023.

[LW23b] F.-H. Liu and H. Wang. Batch Bootstrapping ii: Bootstrapping in Polynomial Modulus Only Requires $\tilde{o}(1)$ FHE Multiplications in Amortization. In *EUROCRYPT*, 2023.

[Mat20] K. Matsuoka. TFHEpp: pure C++ implementation of TFHE Cryptosystem. `https://github.com/virtualsecureplatform/TFHEpp`, 2020.

[Max19] A. Maximov. AES MixColumn with 92 XOR gates. *IACR Cryptol. ePrint Arch.*, 2019.

[MB07] A. Maximov and A. Biryukov. Two trivial attacks on trivium. In *Proceedings of the 14th International Conference on Selected Areas in Cryptography*, page 36–55, 2007.

[MCJS19] P. Méaux, C. Carlet, A. Journault, and F.-X. Standaert. Improved Filter Permutators for Efficient FHE: Better Instances and Iplementations. In *INDOCRYPT*, page 68–91, 2019.

[MGH10] C. A. Melchor, P. Gaborit, and J. Herranz. Additively Homomorphic Encryption with d-Operand Multiplications. In Tal Rabin, editor, *Advances in Cryptology – CRYPTO 2010*, 2010.

[MJSC16] P. Méaux, A. Journault, F.-X. Standaert, and C. Carlet. Towards Stream Ciphers for Efficient FHE with Low-Noise Ciphertexts. In *EUROCRYPT*, pages 311–343, 2016.

[MN24] M. Manulis and J. Nguyen. Fully Homomorphic Encryption Beyond I0ND-CCA1 Security: Integrity Through Verifiability. In *EUROCRYPT*, page 63–93, 2024.

[MS13] S. Mella and R. Susella. On the Homomorphic Computation of Symmetric Cryptographic primitives. In Martijn Stam, editor, *Cryptography and Coding*, 2013.

[Pai99] P. Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *EUROCRYPT*, page 223–238, 1999.

[Pei16] C. Peikert. A Decade of Lattice Cryptography. *Found. Trends Theor. Comput. Sci.*, page 283–424, 2016.

[PJH23] Méaux P., Park J., and V. L. Pereira H. Towards Practical Transciphering for FHE with Setup Independent of the Plaintext Space. *IACR Communications in Cryptology*, 1(1), 2023.

[RAD78] S. Rivest, L. Adleman, and M.L. Dertourzos. On data banks and privacy homomorphisms. In Academic Press, editor, *Foundations of Secure Computation*, 1978.

[Reg05] O. Regev. On lattices, learning with errors, random linear codes, and cryptography. In ACM, editor, *Journal of the ACM (JACM)*, 2005.

[Reg09] O. Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 2009.

[RLW25] X. Ren, Y. Li, and M. Wang. Improved Algebraic Attacks on Round-Reduced LowMC with Single-Data Complexity. In *Selected Areas in Cryptography – SAC*, page 183–204, 2025.

[RS18] C. Rechberger and Hadi Soleimany. Cryptanalysis of low-data instances of full LowMCv2. *IACR Transactions on Symmetric Cryptology*, 2018:163–181, 2018.

[RSA78] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, page 120–126, 1978.

[SMK22] R. Stracovsky, R. Akhavan Mahdavi, and F. Kerschbaum. Faster Evaluation of AES using TFHE. Poster Session, FHE.Org, 2022.

[SSA+07] T. Shirai, K. Shibutani, T. Akishita, S. Moriai, and T. Iwata. The 128-bit Blockcipher CLEFIA. In Alex Biryukov, editor, *Fast Software Encryption*, pages 181–195, 2007.

[SV10] N. Smart and F. Vercauteren. Fully Homomorphic Encryption with Relatively Small Key and Ciphertext Sizes. In *Proceedings of the 13th International Conference on Practice and Theory in Public Key Cryptography, PKC*, page 420–443, 2010.

[TCB+25] D. Trama, P.-E. Clet, A. Boudguiga, R. Sirdey, and N. Ye. Designing a General-Purpose 8-bit (T)FHE Processor Abstraction. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025.

[TCBS23a] D. Trama, P.-E. Clet, A. Boudguiga, and R. Sirdey. Building Blocks for LSTM Homomorphic Evaluation with TFHE. In *Cyber Security, Cryptology, and Machine Learning*, 2023.

[TCBS23b] D. Trama, P.-E. Clet, A. Boudguiga, and R. Sirdey. A Homomorphic AES Evaluation in Less than 30 Seconds by Means of TFHE. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC '23, 2023.

[TKLP25] I. Thakur, A. Karmakar, C. Li, and B. Preneel. A Survey on Transciphering and Symmetric Ciphers for Homomorphic Encryption. Cryptology ePrint Archive, Paper 2025/093, 2025.

[TM13] N. G. Tsoutsos and M. Maniatakos. Investigating the Application of One Instruction Set Computing for Encrypted Data Computation. In *SPACE*, pages 21–37, 2013.

[TM14] N. G. Tsoutsos and M. Maniatakos. HEROIC: Homomorphically EncRypted One Instruction Computer. In *IEEE DATE*, 2014.

[Vie07] M. Vielhaber. Breaking ONE.FIVIUM by AIDA an Algebraic IV Differential Attack. *IACR Cryptol. ePrint Arch.*, 2007.

[WLW+24] B. Wei, X. Lu, R. Wang, K. Liu, Z. Li, and K. Wang. Thunderbird: Efficient Homomorphic Evaluation of Symmetric Ciphers in 3gpp by combining two modes of TFHE. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2024.

[WWL+23] B. Wei, R. Wang, Z. Li, Q. Liu, and X. Lu. Fregata: Faster Homomorphic Evaluation of AES via TFHE. In *Information Security: 26th International Conference, ISC 2023, Groningen, The Netherlands, November 15–17, 2023, Proceedings*, 2023.

[YXS+21] Z. Yang, X. Xie, H. Shen, S. Chen, and J. Zhou. TOTA: Fully Homomorphic Encryption with Smaller Parameters and Stronger Security. Cryptology ePrint Archive, Report 2021/1347, 2021.

[Zam22] Zama. Concrete: TFHE Compiler that converts python programs into FHE equivalent, 2022. `https://github.com/zama-ai/concrete`.

Appendix A

Complete Listing and Timings of Instructions

Instr.	Description	N_{BR}	N_{KS}	Exec. (ms)
ABS	Returns an encryption of the absolute value of an encrypted input.	2	1	215
ADD/SUB	Performs the homomorphic addition (or subtraction) of two encryptions of 8-bit values and returns an 8-bit encrypted result.	7	4	493
ADDi/SUBi	Performs the homomorphic addition (or subtraction) of one 8-bit plaintext to a ciphertext encrypting an 8-bit value and returns the 8-bit encrypted result.	2	1	137
ADDZ	Performs the carryless addition of two 8-bit encrypted inputs. Returns an 8-bit encrypted value.	4	2	271
AND	Computes the logical AND of two 8-bit payload ciphertexts.	4	2	278
ANDi	Computes the logical AND of an 8-bit payload ciphertext and an 8-bit plaintext.	2	0	69
(N)CDUP	Given an encrypted Boolean c_b and an 8-bit payload ciphertext C, returns an encryption of $c_b \times C$.	3	1	159

Instr.	Description	$\mathsf{N_{BR}}$	$\mathsf{N_{KS}}$	Exec. (ms)
CSEL	Given an encrypted Boolean c_b and two 8-bit payload ciphertexts C and C', returns an encryption of $c_b \times C + (1 - c_b) \times C'$.	9	6	694
DC	Decomposition of an 8-bit payload ciphertext into eight ciphertexts encrypting binary values.	2	0	81
(U)DIV4	Calculates the Euclidean division of an 8-bit payload ciphertext by the encryption of a nibble and returns encrypted the quotient.	21	14	1624
(U)DIV(4)i	Calculates the Euclidean division of an 8-bit payload ciphertext by an 8-bit (or 4-bit) plaintext and returns the encrypted quotient.	2	1	133
(U)DIV	Calculates the Euclidean division of an 8-bit payload ciphertext by another 8-bit payload ciphertext and returns the encrypted quotient.	97	56	7711
EQ	Equality test: homomorphically compares two 8-bit payload ciphertexts and returns an encrypted boolean corresponding to the evaluation.	6	3	393
EQi	Equality test: homomorphically compares one 8-bit payload ciphertext with one 8-bit plaintext and returns an encrypted boolean corresponding to the evaluation.	2	0	88
GT(E)/LT(E)	Greater Than (or Equal to)/Less Than (or Equal to): compares two 8-bit payload ciphertexts and returns an encrypted boolean corresponding to the evaluation.	9	5	623
GT(E)i/LT(E)i	Greater Than (or Equal to)/Less Than (or Equal to): compares one 8-bit payload ciphertext with one 8-bit plaintext and returns an encrypted boolean corresponding to the evaluation.	2	1	127

Instr.	Description	N_{BR}	N_{KS}	Exec. (ms)
MAX/MIN	Homomorphically computes the maximum (or minimum) of two 8-bit payload ciphertexts and returns the encrypted result.	16	10	1176
MAXi/MINi	Homomorphically computes the maximum (or minimum) of an 8-bit payload ciphertext and an 8-bit plaintext and returns the encrypted result.	2	1	133
MOD4	Homomorphically calculates the Euclidean division of an 8-bit ciphertext by a 4-bit ciphertext and returns the remainder.	10	6	724
MOD4i	Homomorphically calculates the Euclidean division of an 8-bit payload ciphertext by a 4-bit plaintext and returns the encryption of the remainder.	2	1	133
MOD	Homomorphically calculates the Euclidean division of an 8-bit payload ciphertext by another 8-bit payload ciphertext and returns the encryption of the remainder.	91	50	7584
MODi	Homomorphically calculates the Euclidean division of an 8-bit payload ciphertext by an 8-bit plaintext and returns the encryption of the remainder.	3	2	267
MUL	Multiplies two 8-bit payload ciphertexts and returns the encrypted result (only returns the result modulo 256).	10	6	725
MUL(M)i	Multiplies one 8-bit payload plaintext with an 8-bit payload ciphertext and returns the result on 8 bits.	2	1	133
MULM	Multiplies two 8-bit payload ciphertexts and returns the overhead result on 8-bits.	32	20	2442
NEG	Returns the negation of the input on a signed 8-bit payload ciphertext.	2	1	215
OR	Computes the logical OR of two 8-bit payload ciphertexts.	4	2	278

Instr.	Description	N_{BR}	N_{KS}	Exec. (ms)
ORi	Computes the logical OR of an 8-bit payload ciphertext and an 8-bit plaintext.	2	0	69
RC	Recomposition of eight binary ciphertexts into two 4-bit payload ciphertexts encoding one 8-bit payload ciphertext.	8	0	267
ROL/ROR	Rotates an 8-bit payload ciphertext to the left (or right) by an 8-bit payload encrypted index and returns the rotated ciphertext.	9	6	714
ROLi/RORi	Rotates an 8-bit payload ciphertext to the left (or right) by an 8-bit payload plaintext and returns the rotated ciphertext.	4	0	125
(U)SHL/(U)SHR	Shifts an 8-bit payload ciphertext to the left (or right) by an 8-bit encrypted index and returns the shifted ciphertext.	6	4	478
(U)SHLi/(U)SHRi	Shifts an 8-bit payload ciphertext to the left (or right) by an 8-bit plaintext index and returns the shifted ciphertext.	2	0	72
TZR	Test Zero: Homomorphically tests if an 8-bit payload ciphertext is an encryption of zero.	2	0	88
XOP	User's defined operator.	3	2	229
XOR	Computes the logical XOR of two 8-bit paylaod ciphertexts.	4	2	278
XORi	Computes the logical XOR of an 8-bit payload ciphertext and an 8-bit plaintext.	2	0	69

Appendix B

Instruction Set Implementation Details

B.1 Univariate Operations

Cleartext-ciphertext operations are also called *univariate* operations, as they only take *one* encrypted input. Such operations can be cleartext-ciphertext multiplications in GF(256), cleartext-ciphertext additions in GF(256), cleartext-ciphertext comparisons, ... In such cases, the generic use of our LUT evaluation tools is as follows.

In order to homomorphically compute the evaluation of a univariate function $f : \mathcal{M} \to \mathcal{M}$ on an 8-bit encrypted input $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$, we have to create basis 16 tables from tab_f, the basis 256 LookUp Table corresponding to f. Indeed, we need one table $\mathsf{tab}_{\text{MSN}}$ to compute the MSN of $f(M) = f(16h + l)$ and one $\mathsf{tab}_{\text{LSN}}$ to compute its LSN part.
Such tables are easily defined by $\mathsf{tab}_{\text{MSN}}[i] = \lfloor \frac{\mathsf{tab}_f[i]}{16} \rfloor$ and $\mathsf{tab}_{\text{LSN}}[i] = \mathsf{tab}_f[i]\%16$ for $i \in \{0, \cdots, 255\}$. Note that these tables can be efficiently generated on the fly. In fact, compared to the cost of bootstrapping, this operation is almost free of charge and offers a good time-memory compromise.
Then the naive way to evaluate f is to first compute $\bar{c_0} = \mathsf{LUTeval}(c_0, c_1; \mathsf{tab}_{\text{MSN}})$, which gives us the encryption of the MSN part of $f(M)$. Then we can similarly obtain the encryption of the LSN part of $f(M)$ by computing $\bar{c_1} = \mathsf{LUTeval}(c_0, c_1; \mathsf{tab}_{\text{LSN}})$. Thus, we obtain $\bar{C} = (\bar{c_0}, \bar{c_1}) \in [\![\bar{h}]\!] \times [\![\bar{l}]\!]$, such that $f(M) = 16\bar{h} + \bar{l}$.
It is important to see that as the calls to $\mathsf{LUTeval}$ are applied on the same encrypted inputs, they can be factorized in a single call to $\mathsf{MVLUTeval}$, giving us an *optimized* execution of the homomorphic evaluation of f. In the rest of this Section, we only present such optimized versions of our operators.
Following the same logic, we provide a user-defined operation XOP allowing one to homomorphically evaluate any univariate function on an 8-bit encrypted input of his choice.

XOP

// takes a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and a LUT tab_f
// $\mathsf{tab}_{\text{MSN}}$ and $\mathsf{tab}_{\text{LSN}}$ are created on the fly depending on tab_f
$(\bar{c_0}, \bar{c_1}) = \mathsf{MVLUTeval}(c_0; c_1, c_1; \mathsf{tab}_{\text{MSN}}, \mathsf{tab}_{\text{LSN}})$

Almost all univariate instructions can be implemented using the structure of the optimized XOP operation. However, this is often not optimal. For instance, we here further detail the cleartext-ciphertext addition (ADDi).
In the case of addition modulo 256 of $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ with an 8-bit plaintext $T = 16u + v \in \mathcal{M}$, we have to create the tables corresponding to the function

$$\begin{array}{rccc} f & : & \mathcal{M} & \to & \mathcal{M} \\ & & M & \mapsto & M + T \end{array}$$

The one giving the most significant nibble of computation is $\mathsf{tab_add}_{\text{MSN}}$ such that $\mathsf{tab_add}_{\text{MSN}}[i] = \lfloor \frac{i+T}{16} \rfloor$ for $i \in \{0, \cdots, 255\}$. Similarly, the table giving the least significant nibble of the computation is $\mathsf{tab_add}_{\text{LSN}}$ such that $\mathsf{tab_add}_{\text{LSN}}[i] = (i + T) \pmod{16}$ for $i \in \{0, \cdots, 255\}$.
Using these tables (that can be computed on the fly), we can execute the ADDi operation following the pattern of XOP. But, if we twist our MVLUTeval tool just a bit (following Remark 3), we can obtain an even more optimized version of these operators. To compute the LSN part of the homomorphic addition of $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $T = 16u + v$, the table $\mathsf{tab}_{\text{LSN}}$ such that for $i \in \{0, \cdots, 15\}$, $\mathsf{tab}_{\text{LSN}}[i] = i + v\%16$ is sufficient. That means that a call to SimpleBoot on c_1 can give us the LSN part of the computation (instead of a call to LUTeval). To factorize this bootstrapping with the one implied by the computation of $\bar{c_0}$ we modify the MVLUTeval instruction: we create $\mathsf{MVLUTeval}_2$ so that the MVB method is used with selector c_1 on $\mathsf{tab}_{\text{MSN}}$ and $\mathsf{tab}_{\text{LSN}}$. Then we apply the extractions: we get 17 new ciphertexts. Sixteen of these ciphertexts (corresponding to the first part of the evaluation of $\mathsf{tab}_{\text{MSN}}$ are used to proceed to the final bootstrapping (giving an encryption of $\mathsf{tab}_{\text{MSN}}[16h + l]$), and the last ciphertext is $\bar{c_1} \in [\![l + u \pmod{16}]\!]$. The generic implementation of such optimized versions is as follows:

ADDi, SUBi, LT(E)i, GT(E)i, MULi, MULMi, DIV(4)i, MOD(4)i, MINi, MAXi, NEG, ABS, ...

// takes a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and an 8-bit immediate value T
// $\mathsf{tab}_{\text{MSN}}$ and $\mathsf{tab}_{\text{LSN}}$ are created on the fly
$(\bar{c_0}, \bar{c_1}) = \mathsf{MVLUTeval}_2(c_1; c_0, c_0; \mathsf{tab}_{\text{MSN}}, \mathsf{tab}_{\text{LSN}})$

This optimization saves us one bootstrapping and one TLWE to TRLWE keyswitch.

B.1.1 Shifts ((U)SHLi, (U)SHRi)

In logical (unsigned) shift, zeros are inserted to replace displaced bits as well in signed representation as in unsigned representation. In the specific case of cleartext-ciphertext logical shift, one wants to shift an 8-bit encrypted input $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ by a plaintext $K \in \mathcal{M}$. To be as efficient as possible and avoid as many unnecessary FHE calculations as possible, we must proceed by considering different cases based on the value of the plaintext index. Indeed, since we work with messages of only 8 bits, an offset index greater than or equal to 8 would not require any homomorphic calculation: it would suffice to return encryption of 0, that is to say $\bar{C} = (\bar{c_0}, \bar{c_1}) \in [\![0]\!] \times [\![0]\!]$. Similarly, if the offset index is $K = 16 \cdot 0 + 4$, it is sufficient to simply return $\bar{C} = (\bar{c_0}, \bar{c_1}) \in [\![l]\!] \times [\![0]\!]$ or $\bar{C} = (\bar{c_0}, \bar{c_1}) \in [\![0]\!] \times [\![h]\!]$ depending on the direction of the shift. Finally, if the index is $K = 16 \cdot 0 + 0$, then the output is the unmodified input ($\bar{C} = C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$). Now, if $K \in \{1, 2, 3, 5, 6, 7\}$, we have to use LUTs to compute the new MSN and LSN of the ciphertext. As the offset index K is known, the tables are easy to compute. For instance for the unsigned shift to the left (USHLi), for $i \in \{0, \cdots, 15\}$, we have $\mathsf{tab_ushli}_{\text{MSN}}[i]$ = `((i<<K) >> 4) &0xf` and $\mathsf{tab_ushli}_{\text{LSN}}[i]$ = `(i<<K) &0xf`. This way, we can compute $\mathsf{USHLi}(C, K)$ by means of SimpleBoot and native TFHE addition LweAdd:

USHLi

// takes a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and an 8-bit immediate value K
$\bar{c_1} = \mathsf{SimpleBoot}(c_1, \mathsf{tab_ushli}_{\text{LSN}})$
$c_t = \mathsf{SimpleBoot}(c_0, \mathsf{tab_ushli}_{\text{LSN}})$
$c_u = \mathsf{SimpleBoot}(c_1, \mathsf{tab_ushli}_{\text{MSN}})$
$\bar{c_0} = \mathsf{LweAdd}(c_t, c_u)$

The final result is $\bar{C} = (\bar{c_0}, \bar{c_1}) = \mathsf{USHLi}(C, T) \in [\![\mathsf{tab_shli}_{\text{MSN}}[16h+l]]\!] \times [\![\mathsf{tab_shli}_{\text{LSN}}[16h+l]]\!]$. Here we can use the native TFHE addition for two reasons. First, the noise of ciphertext after a classic bootstrapping is small enough (regarding our parameters set designed especially to work for ciphertexts obtained following a tree-based method), and second, this is a carryless addition, so there will not be any carry to propagate, meaning that the result stays an encryption of a nibble value. The cleartext-ciphertext logical shift to the right and arithmetic shift to the left are computed in a similar way. In the case of signed right arithmetic (signed) shift (SHRi), the sign bit is replicated to fill in all the vacant positions. The tables to use are thus the following:

- $\mathsf{tab_shri}_{\text{LSN}}$ such that for $i \in \{0, \cdots, 7\}$, $\mathsf{tab_shri}_{\text{LSN}}[i]$ = `(i<<4)>>K & 0xf` and for $i \in \{8, \cdots, 15\}$, $\mathsf{tab_shri}_{\text{LSN}}[i]$ = `((i&0xf0)<<4)>>K & 0xf`
- $\mathsf{tab_shri}_{\text{MSN}}$ such that for $i \in \{0, \cdots, 7\}$, $\mathsf{tab_shri}_{\text{MSN}}[i]$ = `(i>>K) & 0xf` and for $i \in \{8, \cdots, 15\}$, $\mathsf{tab_shri}_{\text{MSN}}[i]$ = `((i&0xf0)>>K) & 0xf`

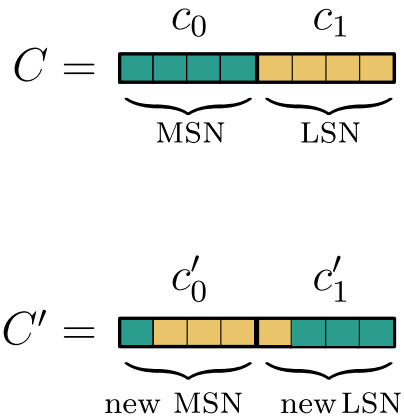


Figure B.1: New MSN and LSN of an 8-bit encrypted value rotated by three to the left ($C' = \mathsf{UROLi}(C, 3)$).

- $\mathsf{tab_ushri}_{\text{MSN}}$ such that for $i \in \{0, \cdots, 15\}$, $\mathsf{tab_ushri}_{\text{MSN}}[i] = (i \texttt{>>} K)$ `& 0xf`

SHRi

// takes a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and an 8-bit immediate value K
$\bar{c_0} = \mathsf{SimpleBoot}(c_0, \mathsf{tab_shri}_{\text{MSN}})$
$c_t = \mathsf{SimpleBoot}(c_1, \mathsf{tab_ushri}_{\text{MSN}})$
$c_u = \mathsf{SimpleBoot}(c_0, \mathsf{tab_shri}_{\text{LSN}})$
$\bar{c_1} = \mathsf{LweAdd}(c_t, c_u)$

B.1.2 Rotations (ROLi, RORi)

With rotations, bits that are "shifted out" are reinserted at the end or beginning of the word, depending on the shift direction (see Figure B.1). To implement cleartext-ciphertext rotations, it is sufficient to use the same tables as for the unsigned shifts. For instance, here is the left rotation (ROLi).

ROLi

// takes a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and an 8-bit immediate value K
$c_t = \mathsf{SimpleBoot}(c_1, \mathsf{tab_ushli}_{\text{MSN}})$
$c_u = \mathsf{SimpleBoot}(c_0, \mathsf{tab_ushli}_{\text{LSN}})$
$\bar{c_1} = \mathsf{LweAdd}(c_t, c_u)$
$c_t = \mathsf{SimpleBoot}(c_0, \mathsf{tab_ushli}_{\text{MSN}})$
$c_u = \mathsf{SimpleBoot}(c_1, \mathsf{tab_ushli}_{\text{LSN}})$
$\bar{c_0} = \mathsf{LweAdd}(c_t, c_u)$

B.1.3 Univariate Bitwise Operations

All univariate bitwise operators can be homomorphically evaluated using the same tools LUTeval and MVLUTeval as before. But in this specific case, it is not the most efficient way. Indeed, regarding bitwise operands, MSN and LSN parts are independent. Thus, two simple bootstrappings are sufficient to compute the resulting MSN and LSN, there is no need for a tree-based method. For instance, to homomorphically compute the bitwise operation XORi of a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ with an 8-bit immediate value $T = 16u + v$, we only need to create two 16-elements tables. The first one, corresponding to the evaluation of the MSN is $\mathsf{tab}_{\mathsf{XORi}_u}$ such that for $j \in \{0, \cdots, 15\}$, $\mathsf{tab}_{\mathsf{XORi}_u}[j] = j \oplus u$. The second, corresponding to the LSN is $\mathsf{tab}_{\mathsf{XORi}_v}$ such that for $j \in \{0, \cdots, 15\}$, $\mathsf{tab}_{\mathsf{XORi}_v}[j] = j \oplus v$. Then, the evaluation of XORi is as follows.

XORi

// takes a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and an 8-bit immediate value T
// $\mathsf{tab}_{\mathrm{MSN}}$ and $\mathsf{tab}_{\mathrm{LSN}}$ are created on the fly depending on T
$\bar{c_0} = \mathsf{SimpleBoot}(c_0, \mathsf{tab}_{\mathrm{MSN}})$
$\bar{c_1} = \mathsf{SimpleBoot}(c_1, \mathsf{tab}_{\mathrm{LSN}})$

The final result is $\bar{C} = (\bar{c_0}, \bar{c_1}) = \mathsf{XORi}(C, T) \in [\![u \oplus h]\!] \times [\![v \oplus l]\!]$. All other univariate bitwise operations (including CDUPi) are similarly implemented.

B.1.4 Other Exceptions

Other cleartext-ciphertext operations that have a different structure to XOP include EQi, which tests whether a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ is an encryption of a plaintext $T = 16u + v$. Only one 256-element table, depending on the immediate value T and straightforward to compute, is needed. This table is tab such that for $i \in \{0, \cdots, 255\}$, $\mathsf{tab}[i] = (i == T)$. Then, the instruction is implemented as follows:

EQi

// takes a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and an 8-bit immediate value T
// tab is created on the fly depending on T
$\bar{c_1} = \mathsf{LUTeval}(c_0, c_1, \mathsf{tab})$

Depending on the programmer choice, $\bar{c_0}$ is either an encryption of 0, a plaintext value 0, or $\perp$. This choice is left to the programmer's discretion since the value of $\bar{c_0}$ will not be used in subsequent calculations (the boolean result is encrypted only in $\bar{c_1}$).

B.2 Bivariate Operations

Unlike univariate operators, there is no generic way of efficiently handling bivariate operators. The number of tables and calls to LUTeval and MVLUTeval will depend on the type of operation required, which is why we provide more details for numerous types of operations.

B.2.1 Bitwise Operators

For clarity's sake, let us denote $\circledast$ any bitwise operator (such as XOR, AND, NOR, etc.) or any composition of these operators. All bitwise bivariate operators can be homomorphically evaluated using one 256-element table and MVLUTeval. Indeed, the LUT table corresponding to the bitwise operator $\circledast$ is $\mathsf{tab}_{\circledast}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{tab}_{\circledast}[16i + j] = i \circledast j$. Note that the coefficients of $\mathsf{tab}_{\circledast}$ are in $\{0, \cdots, 15\}$. Then, the homomorphic computation of the $\circledast$ bitwise operand on two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$ only costs two calls to LUTeval. For instance, the XOR instruction is computed as follows.

XOR

// takes two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$
$\bar{c_0} = \mathsf{LUTeval}(c_0, c'_0; \mathsf{tab}_{\mathsf{XOR}})$
$\bar{c_1} = \mathsf{LUTeval}(c_1, c'_1; \mathsf{tab}_{\mathsf{XOR}})$

The final result is $\bar{C} = (\bar{c_0}, \bar{c_1}) = \mathsf{XOR}(C, C') \in [\![h \oplus h']\!] \times [\![l \oplus l']\!]$. All other bivariate bitwise operations are similarly implemented.

B.2.2 Addition (ADD, SUB)

With 8-bit messages decomposed into two nibbles, the addition over $\mathbb{Z}_{256}$ is not completely straightforward. Indeed, to sum $M = 16h + l$ and $M' = 16h' + l'$, we first have to sum the two least significant nibbles of the messages. That is to say to compute $L = l + l' = 16u + v$. As the sum of l and l' may be greater than 15, we have to compute not only v, but also the carry u. Then we can compute $H = h + h' + u$, but in this case we do not compute the carry as we work modulo 256.
To proceed to these computations in the homomorphic domain, we thus need two tables:

- tab_add which computes the LSN part of the addition of two nibbles. That is to say for $i, j \in \{0, \cdots, 15\}$, $\mathsf{tab_add}[16i + j] = (i + j) \pmod{16}$.
- add_carry which computes the carry corresponding to the addition of two nibbles. That is to say for $i, j \in \{0, \cdots, 15\}$, $\mathsf{add_carry}[16i + j] = \lfloor \frac{i+j}{16} \rfloor$.

Using these two tables, we implement the homomorphic addition the following way:

ADD

// takes two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$
$(\bar{c_1}, c_r) = \mathsf{MVLUTeval}(c_1; c'_1, c'_1; \mathsf{tab_add}, \mathsf{add_carry})$
$c_v = \mathsf{LUTeval}(c_0, c'_0, \mathsf{tab_add})$
$\bar{c_0} = \mathsf{LUTeval}(c_v, c_r, \mathsf{tab_add})$

The final result is $\bar{C} = (\bar{c_0}, \bar{c_1}) = \mathsf{ADD}(C, C') \in [\![\lfloor \frac{16h+l+16h'+l'}{16} \rfloor]\!] \times [\![l + l' \pmod{16}]\!]$. Note that the substraction modulo 256 (SUB) works similarly, only the LUTs are different.

The Case of the Addition by Zero (ADDZ)

If the user knows that at least one of the two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ or $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$ is an encryption of zero, then he should use a less expensive operator than the one described above. Indeed, if one of the two ciphertexts encrypts zero, then the addition with any other ciphertext will not produce any carry. It is thus more efficient to compute such an addition as follows:

ADDZ

// takes two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$
$\bar{c_0} = \mathsf{LUTeval}(c_0, c'_0, \mathsf{tab_add})$
$\bar{c_1} = \mathsf{LUTeval}(c_1, c'_1, \mathsf{tab_add})$

This specific operator can be needed in several cases: for instance a homomorphic array assignment (Algo 8), a bubble sort (Algo 5), or the computation of the Sigmoid (Section 5.6.2).

B.2.3 Multiplication (MUL(M))

The multiplication of two 8-bit messages each decomposed into two nibbles relies on the same principle as the addition: we progress nibble by nibble and propagate the carry. We note $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$ two ciphertexts respectively encrypting $T = 16h + l$ and $T' = 16h' + l'$. Then, the following relation stands for all $T, T' \in \mathcal{M}$.

$$T \times T' \;=\; (16h + l) \times (16h' + l') \;=\; 16^2 hh' \;+\; 16(hl' + lh') \;+\; ll'$$

Since we work with 8-bit messages, we do not need to compute the term in 16^2. Thus, we only have to compute $16(hl' + lh') \;+\; ll'$, which involves three multiplications and two additions over $\mathbb{Z}_{256}$. To compute the additions, we use the tables created in Section B.2.2. For the multiplication, we create two new tables:

- tab_mul which computes the LSN part of the multiplication of two nibbles. That is to say for $i, j \in \{0, \cdots, 15\}$, $\mathsf{tab_mul}[16i + j] = (i \times j) \pmod{16}$.

- mul_carry which computes the carry corresponding to the addition of two nibbles. That is to say for $i, j \in \{0, \cdots, 15\}$, $\mathsf{mul_carry}[16i + j] = \lfloor \frac{i \times j}{16} \rfloor$.

Note that we only need to compute the LSN parts of hl' and lh', as well as the LSN of the sum of these two terms (the MSN parts will be multiples of 16^2 modulo 256). For the same reason, the MSN of ll' will be added with regards only to the least significant bits of the result. Finally, multiplication can then be achieved as follows:

MUL

$(\bar{c_1}, c_t, c_u) = \mathsf{MVLUTeval}(c_1; c'_1, c'_1, c'_0; \mathsf{tab_mul}, \mathsf{mul_carry}, \mathsf{tab_mul})$
$//c_t$ encrypts the carry of ll', c_u encrypts the LSN of lh'
$c_v = \mathsf{LUTeval}(c_0, c'_1; \mathsf{tab_mul})$
$c_w = \mathsf{LUTeval}(c_u, c_v; \mathsf{tab_add})$
$\bar{c_0} = \mathsf{LUTeval}(c_w, c_t; \mathsf{tab_add})$

The final result is $\bar{C} = (\bar{c_0}, \bar{c_1}) = \mathsf{MUL}(C, C') \in [\![\lfloor \frac{(16h+l)\times(16h'+l')}{16} \rfloor]\!] \times [\![ll' \pmod{16}]\!]$.

To Obtain the Most Significant Byte

When we multiply two 8-bit integers, we obtain a result on 16 bits. By working with an 8-bit processor, we need a different operation than the multiplication modulo 256 to obtain the Most Significant Byte. To do so, we have no choice but to compute the whole operation without using the ADD instruction, which only works modulo 256. That is to say, to obtain the Most Significant Byte of the multiplication of a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ with a ciphertext $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$, we have to homomorphically compute

$$T \times T' = (16h + l) \times (16h' + l') = 16^2 hh' + 16(hl' + lh') + ll'.$$

with $T = 16h + l$ and $T' = 16h' + l'$ the corresponding two 8-bit plaintexts. The only unnecessary computation is the one giving the LSN of the term ll'. Indeed, all the other operations need to be computed in order to propagate the carry. We can summarize this method as follows:

MULM

$(c_u, c_v, c_w, c_x) = \mathsf{MVLUTeval}(c_0; c'_0, c'_0, c'_1, c'_1; \mathsf{tab_mul}, \mathsf{tab_mul}, \mathsf{tab_mul}, \mathsf{tab_mul})$
$(c_t, c_y, c_z) = \mathsf{MVLUTeval}(c_1; c'_0, c'_0, c'_1; \mathsf{mul_carry}, \mathsf{tab_mul}, \mathsf{mul_carry})$
$(c_x, c_y) = \mathsf{MVLUTeval}(c_x; c_y, c_y; \mathsf{add_carry}, \mathsf{tab_add})$
$(c_t, c_w) = \mathsf{MVLUTeval}(c_w; c_t, c_t; \mathsf{add_carry}, \mathsf{tab_add})$
$(c_c, c_s) = \mathsf{MVLUTeval}(c_x; c_w, c_w; \mathsf{add_carry}, \mathsf{tab_add})$
$c_z = \mathsf{LUTeval}(c_y, c_z, \mathsf{add_carry})$
$(c_s, c_z) = \mathsf{MVLUTeval}(c_s; c_z, c_z, \mathsf{add_carry}, \mathsf{tab_add})$
$c_c = \mathsf{LUTeval}(c_c, c_s, \mathsf{add_carry})$
$(c_0, \bar{c_1}) = \mathsf{ADD}((c_u, c_v), (c_c, c_z))$

The result is $\bar{C} = (\bar{c_0}, \bar{c_1}) = \mathsf{MULM}(C, C') \in [\![\lfloor (16h + l) \times (16h' + l')/4096 \rfloor]\!] \times [\![\lfloor (16h + l) \times (16h' + l')/256 \rfloor \pmod{16}]\!]$.

B.2.4 Minimum and Maximum (MIN, MAX)

Without loss of generality, we describe here the homomorphic computation of the minimum of two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$. The computation of the maximum is very similar and can easily be inferred from that of the minimum.
To evaluate the MIN instruction with our MVLUTeval and LUTeval tools, several tables are needed:

- $\mathsf{tab_min}$, such that for $i, j \in \{0, 1, \cdots, 15\}$, $\mathsf{tab_min}[16i + j] = \min(i, j)$
- $\mathsf{is_inf}$ such that for $i, j \in \{0, 1, \cdots, 15\}$, $\mathsf{is_inf}[16i + j] = (i < j)$.
- $\mathsf{is_sup}$ such that for $i, j \in \{0, 1, \cdots, 15\}$, $\mathsf{is_sup}[16i + j] = (i > j)$.
- $\mathsf{is_eq}$ such that for $i, j \in \{0, 1, \cdots, 15\}$, $\mathsf{is_eq}[16i + j] = (i == j)$.

Then, we can obtain $c_x = \mathsf{LUTeval}(c_0, c'_0, \mathsf{is_inf}) \in [\![h < h']\!] \subset \mathcal{C}$, $c_y = \mathsf{LUTeval}(c_0, c'_0, \mathsf{is_sup}) \in [\![h > h']\!] \subset \mathcal{C}$, and $c_z = \mathsf{LUTeval}(c_0, c'_0, \mathsf{is_eq}) \in [\![h == h']\!] \subset \mathcal{C}$ with a factorized call to

$$\mathsf{MVLUTeval}(c_0; c'_0, c'_0, c'_0; \mathsf{is_inf}, \mathsf{is_sup}, \mathsf{is_eq}).$$

Then, the result of the evaluation of the minimum of two ciphertexts C and C' is

$$\bar{C} = \mathsf{MIN}(C, C') = (\mathsf{LUTeval}(c_0, c'_0, \mathsf{min_tab}),\ c_x \cdot c_1 + c_y \cdot c'_1 + c_z \cdot \mathsf{LUTeval}(c_1, c'_1, \mathsf{min_tab})).$$

Note that the required homomorphic multiplications always involve a ciphertext that encrypts 0 or 1 (because c_x, c_y, and c_z are encryptions of booleans). This means that these multiplications will not produce any carry. It is thus sufficient to only compute the LSN result of these multiplications. The same goes for the required additions: only one of the three terms will be positive, and the others will encrypt zero.

MIN

$(c_x, c_y, c_z, \bar{c_0})$ = MVLUTeval$(c_0; c'_0, c'_0, c'_0, c'_0;$ is_inf, is_sup, is_eq, tab_min$)$
$(c_y, \tilde{c_1})$ = MVLUTeval$(c'_1; c_y, c_1,$ tab_mul, tab_min$)$
c_z = LUTeval$(c_z, \tilde{c_1},$ tab_mul$)$
c_x = LUTeval$(c_x, c_1,$ tab_mul$)$
c_{xy} = LUTeval$(c_x, c_y,$ tab_add$)$
$\bar{c_1}$ = LUTeval$(c_{xy}, c_z,$ tab_add$)$

B.2.5 Division by a 4-bit Ciphertext (DIV4)

Following the presentation of the homomorphic division in Section 5.3.4, we use the following tables to implement the division by a ciphertext $C_k \in [\![0]\!] \times [\![k]\!]$:

- $\mathsf{div}_{\mathrm{MSN}_1}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{div}_{\mathrm{MSN}_1}[16i + j] = \lfloor \frac{16i}{j} \rfloor / 16$
- $\mathsf{div}_{\mathrm{MSN}_2}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{div}_{\mathrm{MSN}_2}[16i + j] = \lfloor \frac{16i}{j} \rfloor \% 16$
- $\mathsf{div}_{\mathrm{LSN}}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{div}_{\mathrm{LSN}}[16i + j] = \lfloor \frac{i}{j} \rfloor$
- $\mathsf{mod}_{\mathrm{LSN}}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{mod}_{\mathrm{LSN}}[16i + j] = i \% j$
- $\mathsf{mod}_{\mathrm{MSN}}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{mod}_{\mathrm{MSN}}[16i + j] = 16i \% j$

DIV4

$(c_u, c_v, c_w, c_x, c_y)$ = MVLUTeval$(c_k; c_0, c_0, c_1, c_0, c_1; \mathsf{div}_{\mathrm{MSN}_1}, \mathsf{div}_{\mathrm{MSN}_2}, \mathsf{div}_{\mathrm{LSN}}, \mathsf{mod}_{\mathrm{MSN}}, \mathsf{mod}_{\mathrm{LSN}})$
(c_t, c_z) = MVLUTeval$(c_x; c_y, c_y,$ add_carry, tab_add$)$
c_t = LUTeval$(c_t, c_z,$ tab_add$)$
c_s = LUTeval$(c_t, c_w,$ tab_add$)$
$(c_s, \bar{c_1})$ = MVLUTeval$(c_v; c_s, c_s;$ add_carry, tab_add$)$
$\bar{c_0}$ = LUTeval$(c_u, c_s,$ tab_add$)$

Details on division by an 8-bit encrypted value $C'_k = (c_{k_0}, c_{k_1}) \in [\![k_0]\!] \times [\![k_1]\!]$ are given in Section 5.3.4, as well as the pseudo code of DIV.

B.2.6 Modulo

MOD4

Let us say that we want to compute the homomorphic modulo of one 8-bit encrypted value $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ corresponding to the encryption of $T = 16h + l$ by the 4-bit encrypted value $c_k \in [\![k]\!]$ (note that if $c_k \in [\![0]\!]$, the instruction should return an error, but we choose to return encryptions of 0). This means we have to homomorphically compute

$$16h + l \pmod{k} = (16h \pmod{k} + l \pmod{k}) \pmod{k}.$$

But, to do so, computing $16h \pmod k + l \pmod k$ is not sufficient and could result in a ciphertext encrypting a value superior to k. For instance, given $k = 15$, $h = 14$ and $l = 14$ we obtain $16h \pmod k + l \pmod k = 28 > k$. This example also highlights that $16h \pmod k + l \pmod k$ may require two nibbles to hold the result.

However, we have:

$$\begin{aligned} 16h \pmod k + l \pmod k &\leq k - 1 + k - 1 \\ &\leq 2k - 2 \end{aligned}$$

If we note $16h \pmod k + l \pmod k = 16h' + l'$ with $h', l' \in \{0, ..., 15\}$, then

$$16h' \pmod k + l' \pmod k \leq k - 1$$

Indeed, if $h' = 0$, then $16h' \pmod k + l' \pmod k = l' \pmod k < k$, and if $h' > 0$, then $16h' > k$ so $16h' \pmod k \leq 16h' - k$ and $16h' \pmod k + l' \pmod k < 2k - 2 - k = k - 2$. Thus computing $16h' \pmod k + l' \pmod k$ gives us a ciphertext encrypting a value inferior to k, that is the smallest positive representative of the class of $T \pmod k$. To implement it with our LUT evaluation tools, we need two tables, $\mathsf{mod16}$ and mod, defined by $\mathsf{mod16}[16i + j] = 16i\%j$ and $\mathsf{mod}[16i + j] = i\%j$ with $i, j \in \{0, 1, \cdots, 15\}$.

MOD4

$(c_r, c_u) = \mathsf{MVLUTeval}(c_k; c_0, c_1; \mathsf{mod16}, \mathsf{mod})$
$(c_t, c_z) = \mathsf{MVLUTeval}(c_r; c_u, c_u, \mathsf{add_carry}, \mathsf{tab_add})$
$c_s = \mathsf{MVLUTeval}(c_k; c_t, c_z, \mathsf{mod16}, \mathsf{mod})$
$\bar{c_1} = \mathsf{LUTeval}(c_t, c_s, \mathsf{tab_add})$

Depending on the user's choice, $\bar{c_0}$ is either an encryption of 0, a plaintext value 0, or $\perp$.

MOD

Now, let us say that a user wants to compute the homomorphic modulo of a ciphertext $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ by another ciphertext $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$ respectively encrypting the 8-bit values $T = 16h + l$ and $T' = 16h' + l'$. This computation is similar to the computation of DIV, we recall here how it can be computed.

MOD

$c_d = \mathsf{LUTeval}(c_0, c'_1, \mathsf{div}_{\text{LSN}})$
$c_s = \mathsf{LUTeval}(c'_0, c_d, \mathsf{tab})$
$c_t = \mathsf{LUTeval}(c_s, c'_1, \mathsf{mul_carry})$
$c_d = \mathsf{LUTeval}(c_0, c_t, \mathsf{tab_sub})$
// Computation of the MSN part of the result is over, we now compute the LSN
$\tilde{C} = (c_d, c_1)$
For $i = 3$ to 0
$C_m = (c_{l0}, c_{l1}) = \mathsf{SHLi}(C', i)$
$c_g = \mathsf{GTE}(\tilde{C}, C_m)$
$c_b = \mathsf{LUTeval}(c'_0, c_g, \mathsf{tab_and_mulm_zero})$
$C_s = \mathsf{MVLUTeval}(c_b; c_{l0}, c_{l1}; \mathsf{tab_mul}, \mathsf{tab_tab_mul})$
$\tilde{C} = \mathsf{SUB}(\tilde{C}, C_s)$

With $\tilde{C}$ the final result. tab is such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{tab}[16i+j] = (i == 0) \times j$. Other tables are defined in Section 5.3.4.

B.2.7 Comparisons

In this section, we discuss the case of several ciphertext-ciphertext comparisons that return an encrypted boolean. Depending on the programmer choice, $\bar{c_0}$ is either an encryption of 0, a plaintext value 0, or $\perp$. This choice is left to the programmer's discretion since the value of $\bar{c_0}$ will not be used in subsequent calculations (the boolean result is encrypted in $\bar{c_1}$).

Equality Test of Two Ciphertext (EQ)

To compute the homomorphic equality test of two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$ two methods are available. The first one is to homomorphically compute the subtraction of one of the two 8-bit ciphertexts by the other and then homomorphically test if the result ciphertext is an encryption of zero. We have already created all the tools required for these operations (see previous sections). So, it is only a question of reusing what has already been made rather than creating new tables. The other method involves creating a new table for evaluating comparisons of 4-bit ciphertexts. Indeed, we have to precompute a 16×16 equality table $\mathsf{tab_eq}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{tab_eq}[16i + j] = (i == j)$. Then we implement the EQ operator the following way:

EQ

$c_t = \mathsf{LUTeval}(c_0, c'_0, \mathsf{tab_eq})$
$c_s = \mathsf{LUTeval}(c_1, c'_1, \mathsf{tab_eq})$
$\bar{c_1} = \mathsf{LUTeval}(c_t, c_s, \mathsf{tab_mul})$

Greater/Less Than (GT(E)/LT(E))

Given two 8-bit ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$ respectively encrypting $T = 16u + v$ and $T' = 16u' + v'$, we want to homomorphically determine either if one of them is greater or smaller than the other. To do so, we have to create new tables for evaluating comparisons of 4-bit ciphertexts. Indeed, for instance, for the "Greater Than" operator, we have $T > T'$ if and only if $h > h'$ or $h == h'$ and $l > l'$. So we need a table that gives the evaluation "Greater Than" and one that gives the evaluation "Equal To". The first one we call $\mathsf{tab_greater_than}$, and is computed such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{tab_greater_than}[16i + j] = (i > j)$. Similarly, as seen in the previous section, we create $\mathsf{tab_eq}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{tab_eq}[16i + j] = (i == j)$. Instructions GTE, LT and LTE are implemented in a similar way.

GT

$(c_t, c_s) = \mathsf{MVLUTeval}(c_0; c'_0, c'_0; \mathsf{tab_greater_than}, \mathsf{tab_eq})$
$c_u = \mathsf{LUTeval}(c_1, c'_1, \mathsf{tab_greater_than})$
$c_b = \mathsf{LUTeval}(c_s, c_u, \mathsf{tab_mul})$
$\bar{c_1} = \mathsf{LUTeval}(c_t, c_b, \mathsf{tab_mul})$

B.2.8 Rotations and Shifts

Shifts ((U)SHL, (U)SHR)

In the case of a shift of an 8-bit encrypted input $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ by an encrypted 4-bit index $c_k \in [\![k]\!]$,as the offset index is encrypted, one cannot proceed by considering different cases based on the value of the offset to optimize the FHE calculations. We thus have to create LUTs encoding the result of the shift and evaluate them with our LUTeval and MVLUTeval tools to be able to compute the operation. Without loss of generality, let us say that we want to compute a homomorphic arithmetic signed shift to the right SHR. Several tables are needed:

- $\mathsf{shr}_{\text{MSN}}$ such that for $i, j \in \{0, \cdots, 15\}$, $\mathsf{shr}_{\text{MSN}}[16i + j] = (i\texttt{>>}j)\texttt{\&0xf}$
- $\mathsf{ashr}_{\text{MSN}}$ such that for $i \in \{0, \cdots, 7\}$ and $j \in \{0, \cdots, 15\}$, $\mathsf{ashr}_{\text{MSN}}[16i+j] = \mathsf{shr}_{\text{MSN}}[16i+j]$ and for $i \in \{8, \cdots, 15\}$ and $j \in \{0, \cdots, 15\}$, $\mathsf{ashr}_{\text{MSN}}[16i + j] = ((i\texttt{\&0xf0})\texttt{>>}j)\texttt{\&0xf}$

- $\mathsf{ashr}_{\text{LSN}}$ such that for $i \in \{0,\cdots,7\}$ and $j \in \{0,\cdots,15\}$, $\mathsf{ashr}_{\text{LSN}}[16i+j] = ((i\texttt{<<}4)\texttt{>>}j)\texttt{\&0xf}$ and for $i \in \{8,\cdots,15\}$ and $j \in \{0,\cdots,15\}$, $\mathsf{ashr}_{\text{LSN}}[16i+j] = (((i\texttt{\&0xf0}) \texttt{<<} 4)\texttt{>>}j)\texttt{\&0xf}$

Then, SHR can be implemented as follows:

SHR

$(\bar{c_0}, c_s, c_u) = \mathsf{MVLUTeval}(c_k; c_0, c_0; c_1\mathsf{ashr}_{\text{MSN}}, \mathsf{ashr}_{\text{LSN}}, \mathsf{shr}_{\text{MSN}})$
$\bar{c_1} = \mathsf{LUTeval}(c_u, c_s, \mathsf{tab_add})$

Other shifts are computed in a similar way with their corresponding tables.

Rotations (ROL,ROR)

Rotations are computed using the same logic as the logical shift. The only difference is that instead of inserting zeros (or signed bit) to fill the space left by the bits that have been pushed out, we recover these bits and reinsert them. The LUTs are identical to the one needed for logical shift. ROR instruction is computed in a symmetric way.

ROL

$(c_t, c_s, c_u, c_x) = \mathsf{MVLUTeval}(c_k; c_0, c_0, c_1; c_1; \mathsf{shl}_{\text{MSN}}, \mathsf{shl}_{\text{LSN}}, \mathsf{shl}_{\text{MSN}}, \mathsf{shl}_{\text{LSN}})$
$\bar{c_0} = \mathsf{LUTeval}(c_t, c_x, \mathsf{tab_add})$
$\bar{c_1} = \mathsf{LUTeval}(c_u, c_s, \mathsf{tab_add})$

B.2.9 Conditional Assignment

We propose two conditional assignment operators that we call CDUP and NCDUP. They each take two ciphertexts $C = (c_0, c_1) \in [\![h]\!] \times [\![l]\!]$ and $C' = (c'_0, c'_1) \in [\![h']\!] \times [\![l']\!]$ as inputs. The first one is an encryption of a boolean value (meaning $h = 0$ and $l \in \{0, 1\}$), and the second one can be an encryption of any 8-bit value. We define these instructions so that $\mathsf{CDUP}(C, C') = \bar{C} \in [\![h \times l']\!] \times [\![l \times l']\!]$ and $\mathsf{NCDUP}(C, C') = \bar{C} \in [\![h \times (1-l')]\!] \times [\![l \times (1-l')]\!]$. To implement these, we either need the tab_mul table used to compute the LSN of the multiplication of two encrypted nibbles (CDUP) or a modified tab_mul called tab_mul_spe such that for $i, j \in \{0,\cdots,15\}$, $\mathsf{tab_mul_spe}[16i + j] = (j == 0) \times i$ (NCDUP).

(N)CDUP

//Depending on the instruction to be computed, we use tab which is either tab_mul
//or tab_mul_spe
$(\bar{c_0}, \bar{c_1}) = \mathsf{MVLUTeval}(c'_1; c_0, c_1; \mathsf{tab}, \mathsf{tab})$